# WEATHER MONITOR

*An Intelligent Android Weather Application with Adaptive Weather Ambience*

A Project Report submitted to

**KURUKSHETRA UNIVERSITY, KURUKSHETRA**

in partial fulfillment of the requirements for the award of the degree of

**[Bachelor of Technology / BCA / MCA] in [Computer Science & Engineering]**

*[ Insert University / College Logo Here ]*

*Submitted by*

**RAGHAV SUTHAR**

Enrollment No. / LMS ID: 11056166

*Under the supervision of*

**[SUPERVISOR NAME], [Designation]**

**[Department Name]**

**[College / Institute Name], Kurukshetra University**

Academic Year [2025-2026]

# Declaration

I, RAGHAV SUTHAR, bearing Enrollment No. / LMS ID 11056166, hereby declare that the project report entitled “Weather Monitor – An Intelligent Android Weather Application with Adaptive Weather Ambience” submitted to Kurukshetra University, Kurukshetra, is a record of original work carried out by me under the supervision of [SUPERVISOR NAME], [Designation].

I further declare that the work reported in this project has not been submitted, either in part or in full, to any other university or institute for the award of any degree or diploma, and that all external libraries, data sources, and references used in this work have been duly acknowledged.

Place: Kurukshetra

Date: _____

**RAGHAV SUTHAR**

Enrollment No. / LMS ID: 11056166

# Certificate from the Supervisor

This is to certify that the project report entitled “Weather Monitor – An Intelligent Android Weather Application with Adaptive Weather Ambience” is a bonafide record of the project work carried out by RAGHAV SUTHAR (Enrollment No. / LMS ID: 11056166) under my supervision and guidance, in partial fulfillment of the requirements for the award of the degree of [Degree] in [Branch] at [College / Institute Name], Kurukshetra University, during the academic year [2025-2026].

To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

_____

**[SUPERVISOR NAME]**
[Designation], [Department Name]
[College / Institute Name]

# Acknowledgement

The successful completion of this project would not have been possible without the guidance, support, and encouragement of many individuals, and I take this opportunity to express my sincere gratitude to all of them.

First and foremost, I am deeply indebted to my supervisor, [SUPERVISOR NAME], [Designation], for the invaluable guidance, constant encouragement, and constructive feedback offered throughout the course of this project. Their insights and patience were instrumental in shaping both the work and my understanding of it.

I extend my sincere thanks to [HOD NAME], Head of the Department of [Department Name], and to all the faculty members of [College / Institute Name], Kurukshetra University, for providing the academic environment, laboratory facilities, and resources that made this project possible. I am also grateful to the university library and its staff for access to reference material that supported this work.

I gratefully acknowledge the open-source community — in particular the maintainers of Open-Meteo, the Jetpack Compose toolkit, and the many libraries used in this project — whose freely shared work formed the technical foundation of this application.

Finally, I wish to thank my family and friends for their unwavering support and motivation throughout the duration of this project.

**RAGHAV SUTHAR**

# List of Abbreviations, Figures and Tables

## List of Abbreviations

| Abbreviation | Full Form |
|---|---|
| **AQI** | Air Quality Index |
| **API** | Application Programming Interface |
| **APK** | Android Package Kit |
| **DI** | Dependency Injection |
| **DFD** | Data Flow Diagram |
| **GPS** | Global Positioning System |
| **HTTP(S)** | Hyper Text Transfer Protocol (Secure) |
| **IDE** | Integrated Development Environment |
| **JSON** | JavaScript Object Notation |
| **MVVM** | Model–View–ViewModel |
| **OS** | Operating System |
| **RSS** | Really Simple Syndication |
| **SDK** | Software Development Kit |
| **UI / UX** | User Interface / User Experience |
| **UV** | Ultraviolet (Index) |
| **WMO** | World Meteorological Organization |

## List of Figures

| Figure | Description |
|---|---|

## List of Tables

# Table of Contents

Open in Word, then right-click here and choose 'Update Field' to build the index with page numbers.

# Chapter 1: Introduction

## 1.1 Introduction

Weather influences countless everyday decisions — what to wear, whether to carry an umbrella, when to travel, and how to plan outdoor activities. Weather Monitor is a native Android application, developed in Kotlin using Google's modern Jetpack Compose toolkit, that delivers accurate, real-time weather information in a clean, visually rich, and easy-to-understand format. Rather than simply displaying numbers, the application interprets weather data into practical guidance and introduces an immersive audio layer that makes checking the weather a calm and pleasant experience.

This report documents the conception, design, implementation, and testing of the application, carried out as a project under Kurukshetra University, Kurukshetra. It describes the motivation behind the project, the tools and methods used, the challenges encountered, and the skills and knowledge gained during its development.

## 1.2 Purpose of the Report

The purpose of this report is to present a complete and structured account of the Weather Monitor project. Specifically, the report aims to: explain why the project was undertaken and the problem it addresses; describe the objectives set at the outset and the extent to which they were met; document the methodology, system design, and implementation in sufficient detail to be reproducible; record the testing performed and the results obtained; and reflect on the learning outcomes, challenges, and skills developed during the project. The report also serves as the formal academic submission for evaluation by the university.

## 1.3 Objectives of the Project

- **Real-time data:** fetch live and forecast weather from reliable, free, and open APIs.
- **Actionable insight:** convert raw weather metrics into a concise, practical ‘smart summary’.
- **Environmental awareness:** present air quality (AQI), UV index, and pollen alongside standard data.

- **Immersive experience:** play adaptive ambient sound matching the current weather condition.
- **Reliability:** function fully offline using cached data and refresh in the background.
- **Usability and privacy:** deliver a clean, ad-free, account-free interface that respects user privacy.
- **Sound engineering practice:** apply the MVVM architecture, dependency injection, and modern Android components.

## 1.4 Scope of the Project

The project delivers a complete, installable Android application targeting devices running Android 8.0 (API level 26) and above. Its scope includes location-based and search-based weather, hourly and multi-day forecasts, environmental metrics, weather news, notifications, a home-screen widget, configurable settings, offline caching, and the adaptive weather ambience feature. The application consumes existing third-party weather and news services rather than hosting a custom backend, which keeps the system lightweight, free to operate, and privacy-friendly.

# Chapter 2: Organization Overview

## 2.1 About Kurukshetra University

Kurukshetra University, Kurukshetra (KUK) is a premier public state university located in the historic city of Kurukshetra in the state of Haryana, India. The university was established in 1956, and its foundation stone was laid by Dr. Rajendra Prasad, the first President of India. Spread over a large, green campus on the banks of the sacred Brahma Sarovar, the university has grown into one of the leading centres of higher education and research in northern India.

Kurukshetra University has been accredited with the highest 'A++' grade by the National Assessment and Accreditation Council (NAAC) in the fourth cycle of assessment, with a cumulative score of 3.56 out of 4, making it the first government university in Haryana to achieve the highest grade. The university has also been placed among the top state universities of the country (Category-I) by the Ministry of Education and has been granted academic autonomy.

## 2.2 Vision and Mission

The vision of Kurukshetra University is to be a globally recognised centre of academic excellence that nurtures knowledge, research, and human values. Its mission is to impart quality higher education, promote research and innovation, and develop socially responsible, skilled, and employable graduates who contribute meaningfully to society and the nation.

## 2.3 Organizational Structure and Key Departments

The university is headed by the Vice-Chancellor and supported by the Registrar, Deans of Faculties, and Heads of Departments. Academic work is organised into several faculties — including Sciences, Engineering and Technology, Arts and Languages, Commerce and Management, and Social Sciences — each comprising multiple teaching departments. The university also maintains affiliated colleges across the region.

This project was carried out within the domain of computer science and software engineering, under the [Department Name] at [College / Institute Name] affiliated to /

constituent of Kurukshetra University. The department focuses on programming, software development, data structures, databases, computer networks, and emerging technologies such as mobile and cloud computing, providing the academic foundation on which this project was built.

# Chapter 3: Project Activities

## 3.1 Roles and Responsibilities

As this was an individual project, I was responsible for the entire software development life cycle of the application, performing the combined roles of analyst, designer, developer, and tester. The key tasks and responsibilities undertaken were:

- **Requirement analysis:** studying existing weather applications, identifying their limitations, and defining the functional and non-functional requirements of the proposed system.
- **System design:** designing the overall MVVM architecture, the data flow, the user-interaction model, and the data models of the application.
- **UI/UX design:** designing clean, accessible screens in Jetpack Compose, including light and dark themes and animated weather scenes.
- **Development:** implementing all features in Kotlin — data fetching, state management, the adaptive weather-ambience audio engine, offline caching, the home-screen widget, and notifications.
- **Integration:** integrating third-party services (Open-Meteo, WeatherAPI.com, and a news feed) and libraries (Koin, OkHttp, Lottie, WorkManager, DataStore).
- **Testing:** writing unit and instrumentation tests and performing manual testing across online and offline scenarios.
- **Documentation:** preparing this project report, including diagrams, screenshots, and the source-code listing.

## 3.2 Projects and Assignments

### 3.2.1 Project Objectives

The principal assignment was the design and development of the Weather Monitor application. Its objectives were to provide accurate real-time weather and forecasts, to translate raw data into actionable insight, to integrate environmental health metrics, to guarantee offline reliability, and to introduce an innovative adaptive weather-ambience feature, all within a clean, privacy-first, ad-free product.

To position the proposed system against existing solutions, a comparative study was carried out, summarised in Table 3.1.

*Table 3.1: Comparison – Existing vs Proposed System*

| Aspect | Typical Existing Apps | Weather Monitor (Proposed) |
|---|---|---|
| Cost / Ads | Ads or paid tiers | Free and completely ad-free |
| Account | Often required | No account or sign-in |
| Data interpretation | Raw numbers | Smart one-line actionable summary |
| Environmental metrics | Partial / separate | AQI, UV and pollen in one view |
| Offline support | Limited | Full offline via local cache |
| Privacy | Tracking / analytics | Privacy-first, no trackers |
| Audio experience | None | Adaptive layered weather ambience |
| UI technology | Mixed / legacy | 100% Jetpack Compose, animated |

### 3.2.2 Methodology and System Design

The project followed an iterative, incremental development methodology. Requirements were refined continuously, features were built and tested in small increments, and the design evolved through successive refinements. The application adopts the Model–View–ViewModel (MVVM) architectural pattern, which cleanly separates the user interface from business logic and data access, making the codebase easier to test, maintain, and extend.

The overall architecture is organised into three layers — the Presentation layer (Jetpack Compose screens), the State/Logic layer (the ViewModel and the audio engine), and the Data layer (repositories and the local cache) — which communicate with external weather and news services, as shown in Figure 3.1.

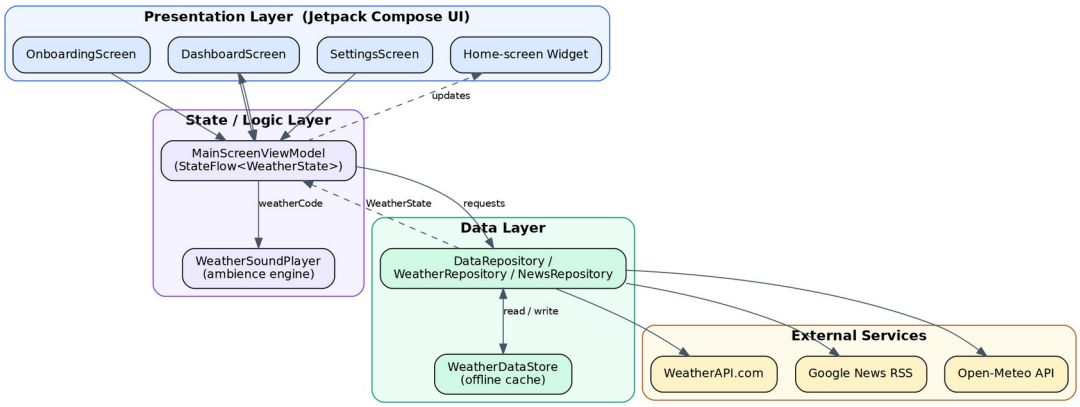


*Figure 3.1: System Architecture (MVVM layered design)*

The flow of information through the system — from capturing the user's location or search query, through fetching and parsing data, generating insights and ambience, and finally rendering the dashboard and dispatching notifications — is depicted in the data flow diagram (Figure 3.2).

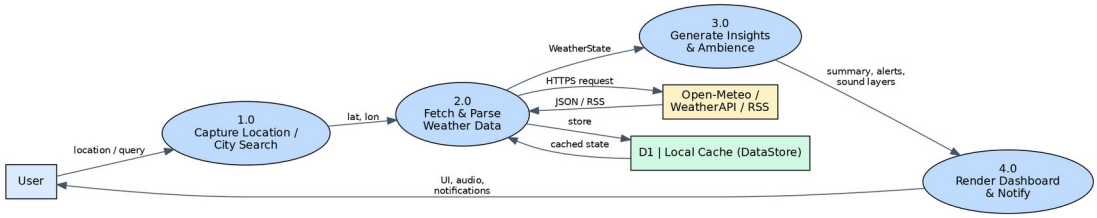


*Figure 3.2: Data Flow Diagram (Level-1)*

The primary interactions between the user and the system are captured in the use case diagram (Figure 3.3), while the principal data models and their relationships are shown in the entity / class diagram (Figure 3.4).

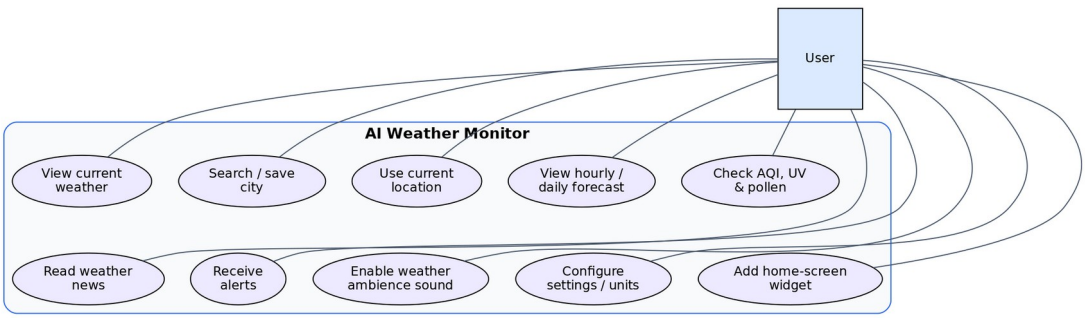


*Figure 3.3: Use Case Diagram*

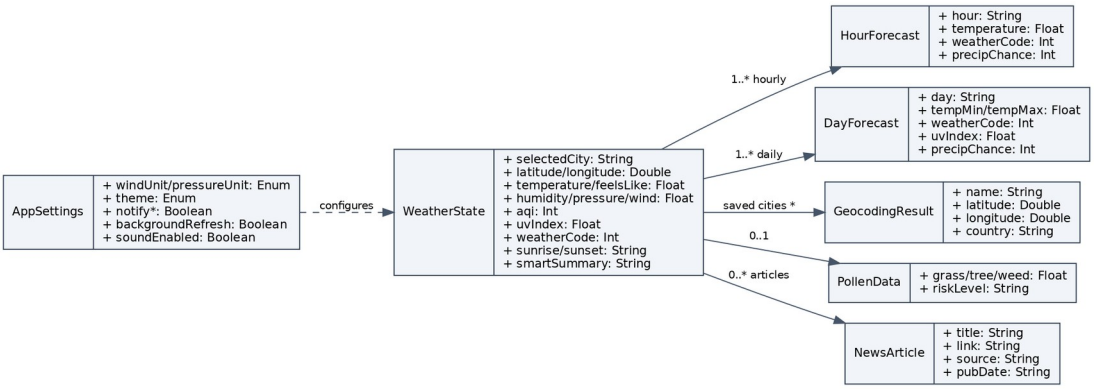


*Figure 3.4: Entity / Class Relationship Diagram*

### 3.2.3 Implementation

Weather data is fetched from the Open-Meteo API using OkHttp and parsed with kotlinx.serialization into strongly-typed Kotlin data classes that populate a single immutable WeatherState object. The ViewModel exposes this state as an observable StateFlow, so that whenever new data arrives or a setting changes, the Compose UI recomposes automatically. A built-in insight generator derives a plain-language ‘smart summary’ from the state (for example, advising an umbrella when rain is likely or sunscreen when the UV index is high).

Open-Meteo returns standard WMO weather codes; a central mapping converts each code into a textual description, a Lottie animation, and a set of sound layers, keeping the textual, visual, and audio representations consistent. The distinctive adaptive weather-ambience feature plays layered, royalty-free ambient sound that matches the current condition, as summarised in Table 3.6. Its engine manages a separate looping audio player per layer, controls per-layer volume, cross-fades between conditions, and is lifecycle-aware — pausing in the background and releasing resources when closed. The feature is disabled by default and controlled from Settings; all ambience audio is generated procedurally, avoiding any copyright concerns.

Every successful fetch is cached using Jetpack DataStore, so the application launches instantly with the last known state and remains fully usable offline, while a WorkManager job refreshes data in the background to keep the widget and notifications current. The hardware and software environment used for development

is given in Tables 3.2 and 3.3, the technology stack in Table 3.4, the application's modules in Table 3.5, and the external data sources in Table 3.7.

*Table 3.2: Hardware Requirements*

| Component | Minimum Specification |
|---|---|
| **Processor (development)** | Intel i3 / equivalent or higher |
| **RAM (development)** | 8 GB (16 GB recommended for Android Studio) |
| **Storage** | 10 GB free for SDK, tools and project |
| **Target device** | Android smartphone, Android 8.0 (API 26) or above |
| **Connectivity** | GPS / network location (optional), internet access |

*Table 3.3: Software Requirements*

| Software / Tool | Version / Detail |
|---|---|
| **Operating System** | Windows / macOS / Linux (development) |
| **IDE** | Android Studio (latest stable) |
| **Language** | Kotlin |
| **UI Toolkit** | Jetpack Compose with Material 3 |
| **Build System** | Gradle (Kotlin DSL) |
| **Min / Target SDK** | API 26 / latest |
| **Runtime** | Android 8.0+ device or emulator |

*Table 3.4: Technology Stack*

| Layer / Concern | Technology Used |
|---|---|
| **Programming language** | Kotlin |
| **User interface** | Jetpack Compose, Material 3, custom |

| | |
|---|---|
| | components |
| **Animations** | Lottie (Airbnb) |
| **Architecture** | MVVM with unidirectional state (StateFlow) |
| **Dependency injection** | Koin |
| **Networking** | OkHttp |
| **Serialization** | kotlinx.serialization (JSON) |
| **Persistence / cache** | Jetpack DataStore |
| **Background work** | WorkManager |
| **Audio** | Android MediaPlayer (multi-layer mixer) |
| **Data sources** | Open-Meteo, WeatherAPI.com, Google News RSS |

*Table 3.5: Module Description*

| Module | Responsibility |
|---|---|
| **DashboardScreen** | Renders current weather, forecasts, AQI/UV/pollen, news and smart summary. |
| **MainScreenViewModel** | Holds and updates WeatherState; coordinates data, settings and sound. |
| **WeatherRepository / DataRepository** | Fetches and merges weather, AQI, UV and pollen from APIs. |
| **NewsRepository** | Retrieves and parses weather / local news from the RSS feed. |
| **WeatherDataStore** | Caches the latest state and settings for offline use. |
| **WeatherSound / WeatherSoundPlayer** | Maps weather codes to layered soundscapes and plays them. |
| **WeatherRefreshWorker** | Performs scheduled background refresh via |

| | |
|---|---|
| | WorkManager. |
| **WeatherWidgetProvider** | Supplies and updates the home-screen widget. |
| **NotificationHelper** | Builds and dispatches severe-weather, rain, briefing and UV alerts. |

*Table 3.6: Weather Condition to Sound-Layer Mapping*

| Weather Condition | Ambient Sound Layers |
|---|---|
| **Clear / Sunny** | Birdsong + faint breeze |
| **Partly cloudy / Cloudy** | Gentle wind + birds |
| **Fog** | Soft, low wind |
| **Drizzle / Rain / Showers** | Rainfall + light breeze |
| **Snow** | Hushed gentle wind |
| **Thunderstorm** | Rain + rolling thunder + wind |

*Table 3.7: APIs and Data Sources*

| Service | Purpose | Key Required |
|---|---|---|
| **Open-Meteo** | Current weather, hourly & daily forecast, AQI, UV | No |
| **WeatherAPI.com** | Optional enrichment / alerts | Yes (optional) |
| **Open-Meteo Pollen** | Grass, tree and weed pollen levels | No |
| **Google News RSS** | Weather and local news headlines | No |

### 3.2.4 Testing and Outcomes

The application was validated using a combination of unit tests (for data parsing, state transformation, and the smart-summary logic), instrumentation tests (for navigation,

UI rendering, and the widget), and manual testing across real-world scenarios. A representative set of test cases and their results is given in Table 3.8.

*Table 3.8: Sample Test Cases and Results*

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| **T-01** | Launch with valid cache | Cached weather shown instantly, then refreshed | Pass |
| **T-02** | Search for a city | Matching cities listed; selection loads weather | Pass |
| **T-03** | Use current location | Weather for GPS location fetched and displayed | Pass |
| **T-04** | Lose network connectivity | Offline banner shown; cached data usable | Pass |
| **T-05** | View hourly & daily forecast | Forecast lists render with correct values | Pass |
| **T-06** | Enable Weather Ambience | Correct layered sound plays for the condition | Pass |
| **T-07** | Change weather condition | Sound cross-fades to the new soundscape | Pass |
| **T-08** | Send app to background | Ambient sound pauses; resumes on return | Pass |
| **T-09** | Toggle theme & units | UI updates and preference persists | Pass |
| **T-10** | Add home-screen widget | Widget shows current weather and updates | Pass |

The outcome of the project is a complete, working Android application that meets all of its stated objectives. All sample test cases passed, and the application behaved

correctly across online and offline conditions, handled permission denial gracefully, and delivered the adaptive ambience feature reliably, including correct pause/resume behaviour and smooth transitions between soundscapes. Screenshots of the working application are provided in Appendix A.

## 3.3 Skills and Tools Used

The project required and developed a broad range of technical skills and made use of the tools and technologies listed in Table 3.9.

*Table 3.9: Skills and Tools Used*

| Category | Skills / Tools |
|---|---|
| **Programming** | Kotlin, object-oriented and functional programming, coroutines |
| **UI development** | Jetpack Compose, Material 3, state hoisting, animations (Lottie) |
| **Architecture** | MVVM, unidirectional data flow, dependency injection (Koin) |
| **Networking & data** | REST APIs, OkHttp, JSON parsing (kotlinx.serialization), RSS |
| **Persistence** | Jetpack DataStore, offline caching strategies |
| **Background & system** | WorkManager, notifications, app widgets, audio (MediaPlayer) |
| **Tooling** | Android Studio, Gradle (Kotlin DSL), Git, emulator/device debugging |
| **Testing** | JUnit, MockK, Compose UI testing |

# Chapter 4: Learning Experience

## 4.1 Challenges Faced

Several challenges were encountered during the project, each of which became an opportunity to learn:

- **Adaptive audio without copyright:** real ambient recordings carry licensing restrictions. This was solved by generating all sound procedurally from filtered noise and synthesised tones, producing royalty-free, seamless loops.
- **Lifecycle-correct audio playback:** background audio can leak resources or continue inappropriately. A lifecycle-aware engine was built to pause playback when the app is backgrounded and release all players when the screen is closed.
- **Smooth audio transitions:** abruptly switching soundscapes was jarring; a cross-fade mechanism with independent per-layer volume control was implemented to make transitions seamless.
- **Offline reliability:** network failures could break the experience. A cache-first strategy using DataStore ensures the last known forecast is always available, with a clear offline indicator.
- **Consistent state across UI, animation and sound:** keeping the textual, visual, and audio representations in sync was achieved by mapping every WMO weather code through a single source of truth.
- **Asynchronous data handling:** combining several network calls (weather, AQI, pollen, news) was managed cleanly using Kotlin coroutines and a repository layer.

## 4.2 Skills Acquired

Through building the application, I acquired and strengthened a number of technical and professional skills, including: proficiency in Kotlin and the Jetpack Compose declarative UI paradigm; practical application of the MVVM architecture and reactive state management with StateFlow; dependency injection using Koin; consuming and parsing REST and RSS data with OkHttp and kotlinx.serialization; implementing offline caching with DataStore and background work with WorkManager; audio programming with the Android MediaPlayer API; and writing unit and

instrumentation tests. Beyond coding, I developed skills in requirement analysis, system design, debugging, technical documentation, and time and task management.

## 4.3 Knowledge Gained

The project deepened my understanding of how a modern Android application is structured and delivered end to end — from consuming public APIs and modelling data, through designing a responsive and accessible interface, to handling the Android application lifecycle, background processing, and system integration such as widgets and notifications. I gained insight into meteorological data (WMO weather codes, AQI, UV, and pollen), the importance of privacy-first design, and the value of a clean, layered architecture for building maintainable and testable software. I also learned how thoughtful use of design, animation, and sound can meaningfully improve the user experience.

# Chapter 5: Analysis and Discussion

## 5.1 Performance Evaluation

The project was evaluated both by my supervisor and through self-assessment. From the supervisor's perspective, the work was assessed on the clarity of the problem definition, the soundness of the design, the quality and completeness of the implementation, the rigour of testing, and the quality of documentation. [Supervisor's formal evaluation/remarks to be added.]

My own self-assessment of the project across key parameters is summarised in Table 5.1.

*Table 5.1: Self-Assessment of Performance*

| Parameter | Self-Assessment | Remarks |
|---|---|---|
| **Requirement analysis** | Strong | Clear problem definition and scope. |
| **Design quality** | Strong | Clean MVVM separation; documented with diagrams. |
| **Implementation** | Strong | All planned features delivered and working. |
| **Innovation** | Very strong | Novel adaptive weather-ambience feature. |
| **Testing** | Good | Unit, instrumentation and manual testing performed. |
| **Documentation** | Strong | Comprehensive report with full source listing. |
| **Time management** | Good | Delivered within the planned schedule. |

## 5.2 Comparison with Expectations

At the outset, I expected to build a functional weather app that could fetch and display current conditions and a forecast. The actual outcome exceeded these initial expectations in several ways. Beyond core forecasting, the application integrates environmental health metrics, generates human-readable insights, works fully offline, and includes a home-screen widget and notifications. Most notably, the adaptive weather-ambience feature — which was not part of the original plan — added a genuinely novel dimension to the project. I also underestimated the importance of lifecycle management and audio resource handling, which turned out to be valuable learning areas. Overall, the experience was more challenging and more rewarding than anticipated.

## 5.3 Recommendations

Based on this experience, the following recommendations are offered for future students undertaking similar projects and for enhancing the application itself:

- Begin with a clear, layered architecture (such as MVVM); it pays off in testability and maintainability as the project grows.
- Prefer free and open data sources (such as Open-Meteo) to avoid cost and key-management overhead during academic projects.
- Test early and often on real devices, especially for lifecycle-sensitive features like audio and background work.
- For the application, future enhancements could include a foreground service so ambience can continue in the background, on-device AI ‘nowcasting’, richer user-selectable soundscapes, Wear OS support, animated precipitation radar, voice briefings, and multi-language localisation.

# Chapter 6: Conclusion

The Weather Monitor project set out to build a weather application that is accurate, informative, reliable, and pleasant to use, and it succeeded in meeting all of these goals. By translating raw meteorological data into clear, actionable insight, integrating environmental health metrics, guaranteeing offline reliability, and introducing the innovative adaptive weather-ambience feature, the application addresses the most common shortcomings of existing weather apps while remaining free, ad-free, and privacy-respecting.

As a learning experience, the project provided comprehensive, hands-on exposure to modern Android development — the MVVM architecture, Jetpack Compose, reactive state management, dependency injection, background processing, audio programming, and testing — within a single, cohesive product. It strengthened my technical abilities, improved my problem-solving and design thinking, and gave me confidence in independently planning and delivering a complete software project. The project has contributed significantly to my personal and professional growth and has laid a strong foundation for further work in mobile application development.

# References


1. Android Developers. (2025). Guide to app architecture. Google. https://developer.android.com/topic/architecture
2. Android Developers. (2025). Jetpack Compose. Google. https://developer.android.com/jetpack/compose
3. Android Developers. (2025). Jetpack DataStore. Google. https://developer.android.com/topic/libraries/architecture/datastore
4. Android Developers. (2025). MediaPlayer overview. Google. https://developer.android.com/media/platform/mediaplayer
5. Android Developers. (2025). WorkManager. Google. https://developer.android.com/topic/libraries/architecture/workmanager
6. Airbnb. (2025). Lottie for Android. https://airbnb.io/lottie
7. JetBrains. (2025). Kotlin programming language documentation. https://kotlinlang.org/docs
8. Koin. (2025). Koin – Kotlin dependency injection framework. https://insert-koin.io
9. Kurukshetra University. (2025). About the university. https://kuk.ac.in
10. Material Design. (2025). Material Design 3 guidelines. Google. https://m3.material.io
11. Open-Meteo. (2025). Open-Meteo free weather API documentation. https://open-meteo.com
12. WeatherAPI.com. (2025). Weather API documentation. https://www.weatherapi.com
13. World Meteorological Organization. (2019). Manual on codes (WMO-No. 306). WMO.

# Appendix A: Application Screenshots

This appendix presents screenshots of the implemented application running on a physical Android device, illustrating the complete user experience.

## A.1 Onboarding

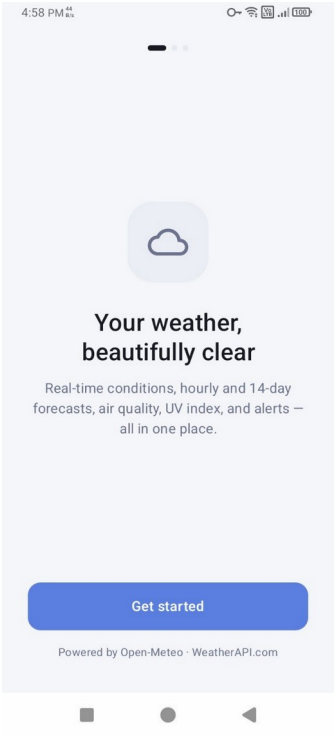


*Figure A.1: Onboarding - Welcome*

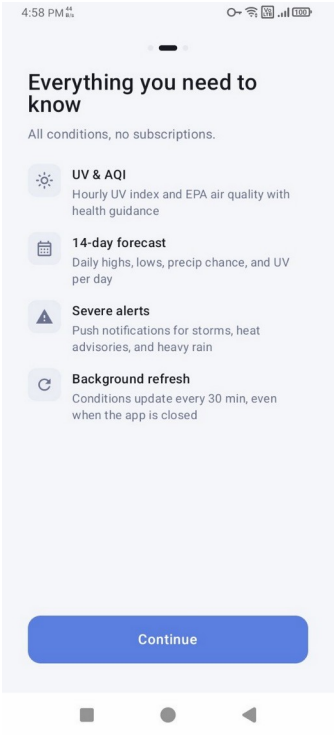


*Figure A.2: Onboarding - Features*

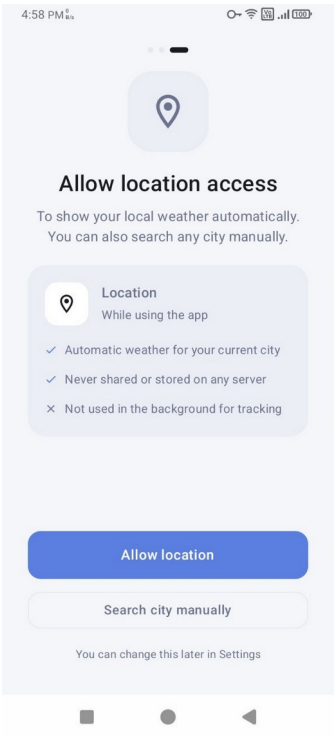


*Figure A.3: Onboarding - Location permission*

## A.2 Dashboard and Forecasts

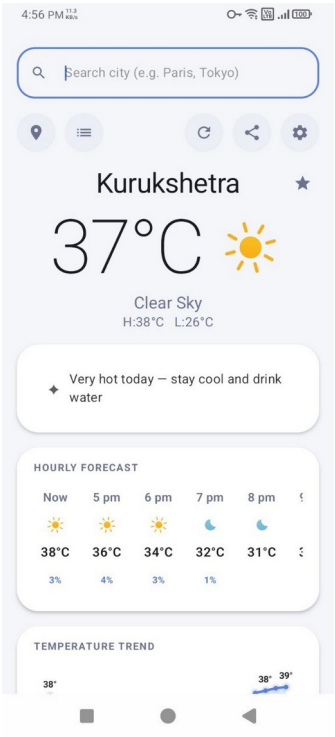


*Figure A.4: Main Dashboard & smart summary*

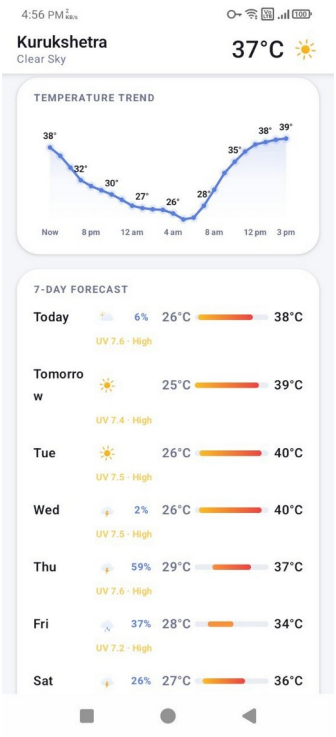


*Figure A.5: Temperature Trend & 7-Day Forecast*


*Figure A.6: Extended 14-Day Forecast*

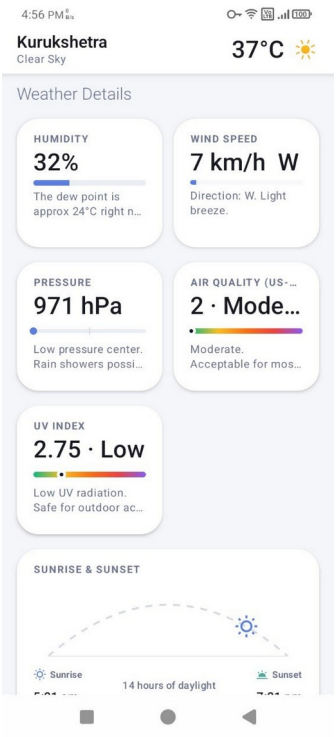


*Figure A.7: Weather Details (AQI, UV, wind)*

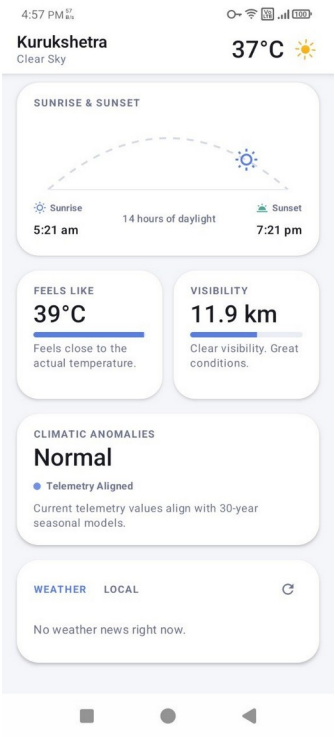


*Figure A.8: Sunrise/Sunset, Feels-like, Visibility*

## A.3 Search, Saved Cities and News


*Figure A.9: City Search with Autocomplete*

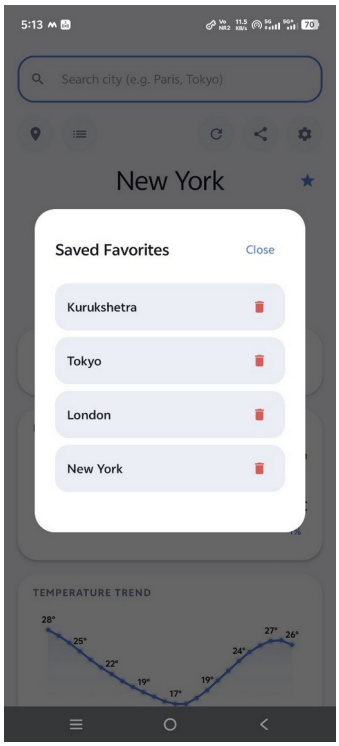


*Figure A.10: Saved Favourite Cities*

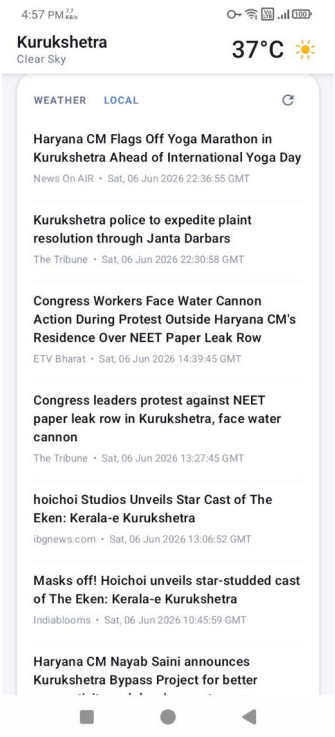


***Figure A.11: Weather & Local News Feed***

## A.4 Settings and Dialogs

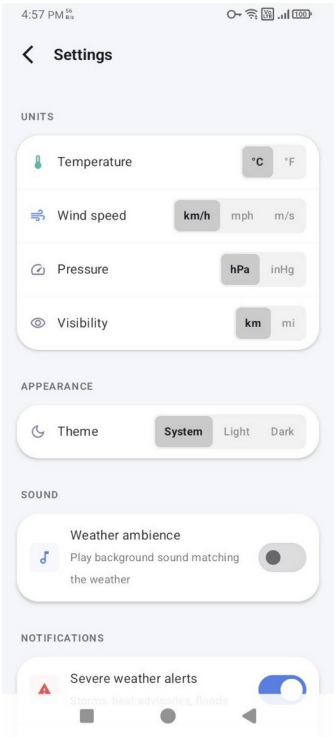


*Figure A.12: Settings - Units, Appearance & Ambience*

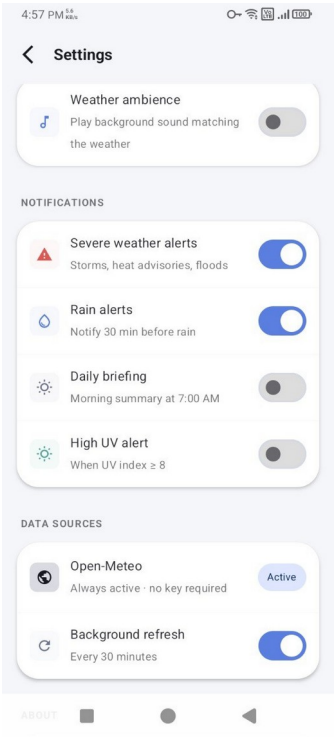


*Figure A.13: Settings - Notifications & Data Sources*

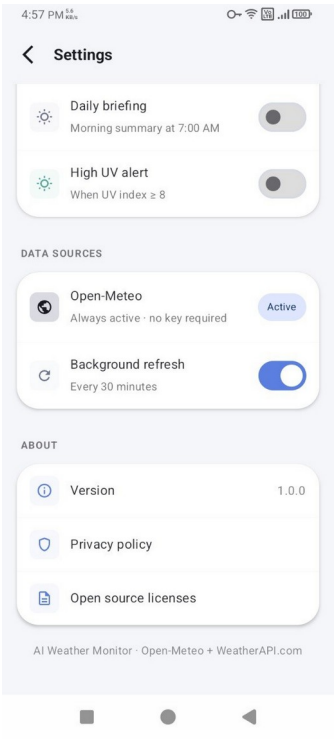


*Figure A.14: Settings - Data Sources & About*

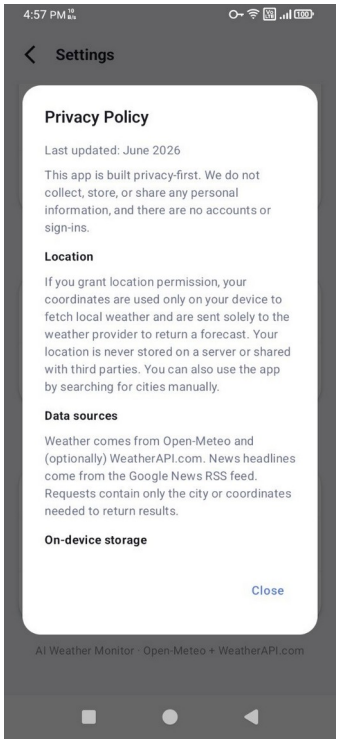


*Figure A.15: Privacy Policy*

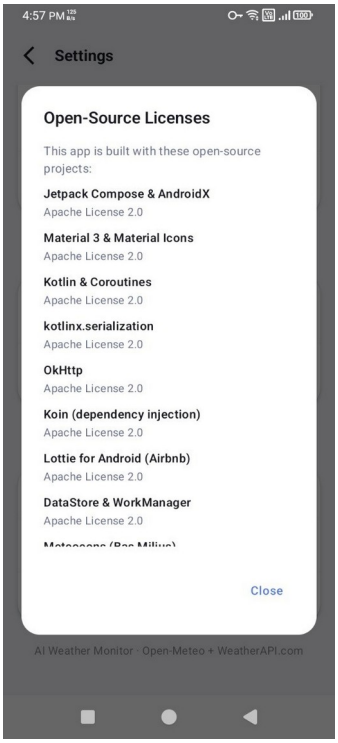


*Figure A.16: Open-Source Licenses*

## A.5 Dark Theme and Widget

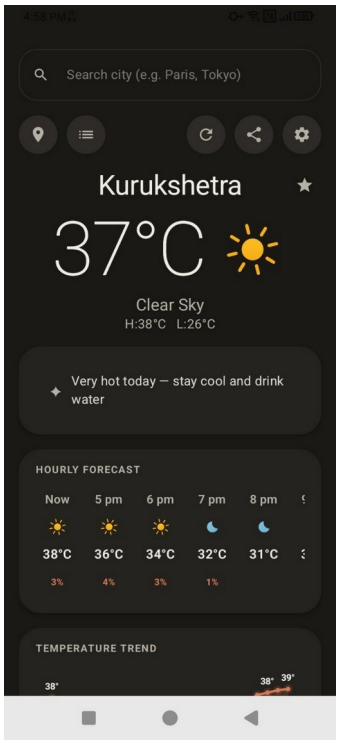


*Figure A.17: Dark Mode - Dashboard*

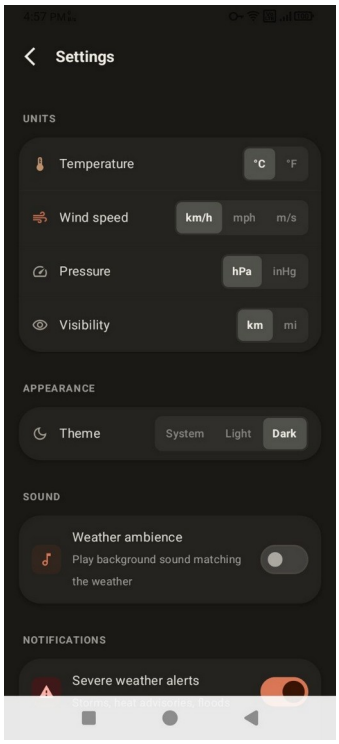


*Figure A.18: Dark Mode - Settings*

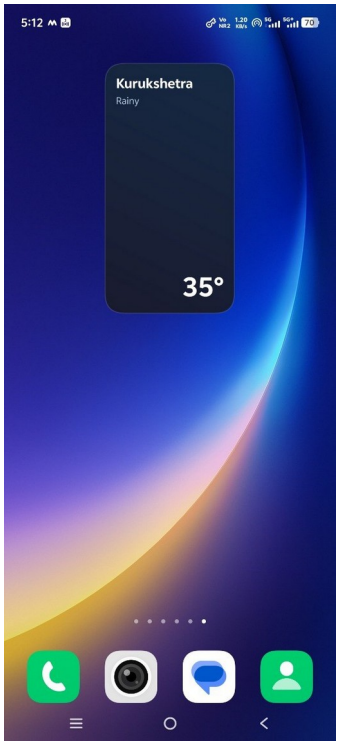


*Figure A.19: Home-screen Widget*

# Appendix B: Source Code Listing

The complete Kotlin source code of the application's main module (app/src/main/java) is reproduced below. The code is presented in Courier New for readability. Files are listed alphabetically by package and file name.

**File: MainActivity.kt**

```
package com.example.aiweathermonitor

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Surface
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import com.example.aiweathermonitor.theme.AIWeatherMonitorTheme
import com.example.aiweathermonitor.ui.AppTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        enableEdgeToEdge(
            statusBarStyle = androidx.activity.SystemBarStyle.auto(
                android.graphics.Color.TRANSPARENT,
                android.graphics.Color.TRANSPARENT
            ),
            navigationBarStyle = androidx.activity.SystemBarStyle.auto(
                android.graphics.Color.TRANSPARENT,
                android.graphics.Color.TRANSPARENT
            )
        )

        setContent {
            val viewModel:
com.example.aiweathermonitor.ui.main.MainScreenViewModel =
                org.koin.androidx.compose.koinViewModel()
            val settings by viewModel.appSettings.collectAsState()

            val darkTheme = when (settings.theme) {
                AppTheme.DARK -> true
                AppTheme.LIGHT -> false
                else -> isSystemInDarkTheme()   // AppTheme.SYSTEM
            }
            AIWeatherMonitorTheme(darkTheme = darkTheme) {
                Surface(modifier =
androidx.compose.ui.Modifier.fillMaxSize()) {
                    MainNavigation()
                }
            }
        }
    }
}
```

**File: Navigation.kt**

```
package com.example.aiweathermonitor

import android.Manifest
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.core.content.ContextCompat
import androidx.navigation3.runtime.NavKey
import androidx.navigation3.runtime.entryProvider
import androidx.navigation3.runtime.rememberNavBackStack
import androidx.navigation3.ui.NavDisplay
import com.example.aiweathermonitor.ui.LocalWeatherCode
import com.example.aiweathermonitor.util.OnboardingPrefs
import com.example.aiweathermonitor.ui.OnboardingScreen
import com.example.aiweathermonitor.ui.SettingsScreen
import com.example.aiweathermonitor.ui.main.MainScreen
import com.example.aiweathermonitor.ui.main.MainScreenViewModel
import org.koin.androidx.compose.koinViewModel

@Composable
fun MainNavigation() {
    val context = LocalContext.current
    val viewModel: MainScreenViewModel = koinViewModel()
    val state by viewModel.weatherState.collectAsState()
    val settings by viewModel.appSettings.collectAsState()

    // Determine start destination: show onboarding only on first launch
    val startDestination: NavKey = remember {
        if (OnboardingPrefs.hasSeenOnboarding(context)) Main else Onboarding
    }

    val backStack = rememberNavBackStack(startDestination)

    NavDisplay(
        backStack = backStack,
        onBack = { backStack.removeLastOrNull() },
        entryProvider = entryProvider {

            // ── Onboarding
──────────────────────────────────────────────────

            entry<Onboarding> {
                val locationLauncher = rememberLauncherForActivityResult(
                    ActivityResultContracts.RequestMultiplePermissions()
                ) { perms ->
                    val granted =
perms[Manifest.permission.ACCESS_FINE_LOCATION] == true ||

perms[Manifest.permission.ACCESS_COARSE_LOCATION] == true
                    // Mark onboarding done regardless of permission outcome
                    OnboardingPrefs.markOnboardingSeen(context)
                    if (granted)
viewModel.fetchWeatherForCurrentLocation(context)
                    // Navigate to Main, removing Onboarding from back stack
                    backStack.removeLastOrNull()
                    backStack.add(Main)
                }

                OnboardingScreen(
                    onRequestLocation = {
                        val hasFine =
ContextCompat.checkSelfPermission(context,
```

```
Manifest.permission.ACCESS_FINE_LOCATION) ==
PackageManager.PERMISSION_GRANTED
                        val hasCoarse =
ContextCompat.checkSelfPermission(context,
Manifest.permission.ACCESS_COARSE_LOCATION) ==
PackageManager.PERMISSION_GRANTED
                        if (hasFine || hasCoarse) {
                            // Already granted — mark done and go to Main
                            OnboardingPrefs.markOnboardingSeen(context)

viewModel.fetchWeatherForCurrentLocation(context)
                            backStack.removeLastOrNull()
                            backStack.add(Main)
                        } else {
                            locationLauncher.launch(
                                arrayOf(

Manifest.permission.ACCESS_FINE_LOCATION,

Manifest.permission.ACCESS_COARSE_LOCATION
                                )
                            )
                        }
                    },
                    onSkipToSearch = {
                        OnboardingPrefs.markOnboardingSeen(context)
                        backStack.removeLastOrNull()
                        backStack.add(Main)
                    }
                )
            }

            // ── Main
────────────────────────────────────────────────────────
            entry<Main> {
                MainScreen(
                    modifier = Modifier.fillMaxSize(),
                    settings = settings,
                    onNavigateToSettings = { backStack.add(Settings) }
                )
            }

            // ── Settings
──────────────────────────────────────────────────
            entry<Settings> {
                CompositionLocalProvider(LocalWeatherCode provides
state.weatherCode) {
                    SettingsScreen(
                        state = state,
                        settings = settings,
                        onSettingsChange = { viewModel.updateSettings(it) },
                        onSaveWeatherApiKey = { key ->
                            viewModel.updateWeatherState(
                                state.copy(weatherApiKey = key,
showKeyDialog = false)
                            )
                        },
                        onNavigateBack = { backStack.removeLastOrNull() }
                    )
                }
            }
        }
    )
}
```

**File: NavigationKeys.kt**

```
package com.example.aiweathermonitor

import androidx.navigation3.runtime.NavKey
import kotlinx.serialization.Serializable

@Serializable data object Onboarding : NavKey
@Serializable data object Main       : NavKey
@Serializable data object Settings   : NavKey
```

**File: NetworkConnectivityObserver.kt**

```
package com.example.aiweathermonitor

import android.content.Context
import android.net.ConnectivityManager
import android.net.Network
import android.net.NetworkCapabilities
import android.net.NetworkRequest
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.flow.distinctUntilChanged

/**
 * Observes device network connectivity state changes in real-time.
 *
 * Monitors internet connectivity through ConnectivityManager callbacks and
emits
 * boolean values indicating whether the device has internet access.
 *
 * **Usage:**
 * ```kotlin
 * val observer = NetworkConnectivityObserver(context)
 * observer.isConnected.collect { isOnline ->
 *     Log.d("Network", "Connected: $isOnline")
 * }
 * ```
 *
 * **Features:**
 * - Real-time connectivity updates
 * - Filters duplicate states with `distinctUntilChanged()`
 * - Sends initial connectivity state on subscription
 * - Properly unregisters callback when flow collection stops
 *
 * @param context Application context used to access ConnectivityManager
system service
 *
 * @throws IllegalStateException if ConnectivityManager service is
unavailable
 *
 * **Thread Safety:** Safe to collect from any coroutine dispatcher
 *
 * **Requires Permissions:**
 * - `android.permission.ACCESS_NETWORK_STATE`
 */
class NetworkConnectivityObserver(context: Context) {
    private val connectivityManager =
context.getSystemService(Context.CONNECTIVITY_SERVICE) as
ConnectivityManager

    /**
     * Check if ACCESS_NETWORK_STATE permission is granted.
     * Required for Android 6.0+ (API 23+)
     */
    private fun hasNetworkStatePermission(context: Context): Boolean {
        return if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.M) {
            val permission =
android.Manifest.permission.ACCESS_NETWORK_STATE
            context.checkSelfPermission(permission) ==
android.content.pm.PackageManager.PERMISSION_GRANTED
        } else {
            true  // Permissions auto-granted on pre-Marshmallow
        }
    }

    /**
```

```
     * Observable Flow of network connectivity state.
     *
     * **Emissions:**
     * - `true`: Device has internet connectivity
     * - `false`: Device has no internet connectivity
     *
     * **Behavior:**
     * - Emits initial state immediately on subscription
     * - Updates whenever connectivity changes
     * - Filters consecutive duplicate values
     * - Thread-safe: subscriptions can be made from any dispatcher
     *
     * **Lifecycle:**
     * - Starts monitoring when first collector subscribes
     * - Stops monitoring when all collectors unsubscribe
     */
    val isConnected: Flow<Boolean> = callbackFlow {
        // Get initial state BEFORE registering callback to avoid race
condition
        val activeNetwork = connectivityManager.activeNetwork
        val capabilities =
connectivityManager.getNetworkCapabilities(activeNetwork)
        val isInitiallyConnected =
capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET) ==
true

        val callback = object : ConnectivityManager.NetworkCallback() {
            override fun onAvailable(network: Network) {
                super.onAvailable(network)
                trySend(true)
            }

            override fun onLost(network: Network) {
                super.onLost(network)
                trySend(false)
            }
        }

        val request = NetworkRequest.Builder()
            .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
            .build()

        connectivityManager.registerNetworkCallback(request, callback)

        // Send initial state after registering to ensure we catch any
changes
        trySend(isInitiallyConnected)

        awaitClose {
            connectivityManager.unregisterNetworkCallback(callback)
        }
    }.distinctUntilChanged()
}
```

**File: NotificationHelper.kt**

```
package com.example.aiweathermonitor

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat

object NotificationHelper {

    // ── Channel IDs ──────────────────────────────────────────────
    const val CHANNEL_SEVERE    = "severe_weather"
    const val CHANNEL_RAIN      = "rain_alert"
    const val CHANNEL_DAILY     = "daily_briefing"
    const val CHANNEL_UV        = "uv_alert"

    // ── Notification IDs ─────────────────────────────────────────
    private const val NOTIF_SEVERE  = 1001
    private const val NOTIF_RAIN    = 1002
    private const val NOTIF_DAILY   = 1003
    private const val NOTIF_UV      = 1004

    // ── Channel setup (call once from Application) ───────────────
    fun createChannels(context: Context) {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
        val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_SEVERE, "Severe weather alerts",
NotificationManager.IMPORTANCE_HIGH).apply {
                description = "Storm warnings, heat advisories, and flood
alerts"
            }
        )
        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_RAIN, "Rain alerts",
NotificationManager.IMPORTANCE_DEFAULT).apply {
                description = "Notifications when rain is expected soon"
            }
        )
        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_DAILY, "Daily briefing",
NotificationManager.IMPORTANCE_DEFAULT).apply {
                description = "Morning weather summary"
            }
        )
        nm.createNotificationChannel(
            NotificationChannel(CHANNEL_UV, "UV alerts",
NotificationManager.IMPORTANCE_DEFAULT).apply {
                description = "Alerts when UV index is very high or extreme"
            }
        )
    }

    // ── Builder helper ───────────────────────────────────────────
    private fun baseBuilder(context: Context, channelId: String):
NotificationCompat.Builder {
        val tapIntent = context.packageManager
```

```
            .getLaunchIntentForPackage(context.packageName)
            ?.apply { flags = Intent.FLAG_ACTIVITY_SINGLE_TOP }
        val tapPi = PendingIntent.getActivity(
            context, 0, tapIntent,
            PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
        )
        return NotificationCompat.Builder(context, channelId)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setAutoCancel(true)
            .setContentIntent(tapPi)
    }

    // ── 1. Severe weather
─────────────────────────────────────────────────
    fun notifySevere(context: Context, event: String, headline: String) {
        if (!hasPermission(context)) return
        val n = baseBuilder(context, CHANNEL_SEVERE)
            .setContentTitle("⚠ $event")
            .setContentText(headline)
            .setStyle(NotificationCompat.BigTextStyle().bigText(headline))
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .build()
        NotificationManagerCompat.from(context).notify(NOTIF_SEVERE, n)
    }

    // ── 2. Rain in ~30 min
──────────────────────────────────────────────
    fun notifyRainSoon(context: Context, cityName: String, precipChance:
Int, hour: String) {
        if (!hasPermission(context)) return
        val n = baseBuilder(context, CHANNEL_RAIN)
            .setContentTitle("Rain expected in $cityName")
            .setContentText("$precipChance% chance of rain around $hour —
carry an umbrella")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()
        NotificationManagerCompat.from(context).notify(NOTIF_RAIN, n)
    }

    // ── 3. Daily briefing
─────────────────────────────────────────────────
    fun notifyDailyBriefing(context: Context, cityName: String, summary:
String) {
        if (!hasPermission(context)) return
        val n = baseBuilder(context, CHANNEL_DAILY)
            .setContentTitle("Good morning · $cityName")
            .setContentText(summary)
            .setStyle(NotificationCompat.BigTextStyle().bigText(summary))
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()
        NotificationManagerCompat.from(context).notify(NOTIF_DAILY, n)
    }

    // ── 4. High UV
──────────────────────────────────────────────────
    fun notifyHighUv(context: Context, uvIndex: Float) {
        if (!hasPermission(context)) return
        val label = uvIndex.getUvDescription()
        val n = baseBuilder(context, CHANNEL_UV)
            .setContentTitle("High UV index: ${uvIndex.toInt()} ($label)")
            .setContentText("Apply SPF 30+ sunscreen and seek shade between
10 AM – 4 PM")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .build()
        NotificationManagerCompat.from(context).notify(NOTIF_UV, n)
    }
```

```
    // ── Permission guard (Android 13+) ──────────────────────────
    private fun hasPermission(context: Context): Boolean {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU) return
true
        return androidx.core.content.ContextCompat.checkSelfPermission(
            context, android.Manifest.permission.POST_NOTIFICATIONS
        ) == android.content.pm.PackageManager.PERMISSION_GRANTED
    }
}
```

**File: WeatherApplication.kt**

```
package com.example.aiweathermonitor

import android.app.Application
import com.example.aiweathermonitor.ui.main.MainScreenViewModel
import com.example.aiweathermonitor.util.HttpClientFactory
import com.example.aiweathermonitor.data.WeatherRepository
import com.example.aiweathermonitor.data.DefaultWeatherRepository
import com.example.aiweathermonitor.data.NewsRepository
import com.example.aiweathermonitor.data.DefaultNewsRepository
import kotlinx.serialization.json.Json
import org.koin.android.ext.koin.androidContext
import org.koin.androidx.viewmodel.dsl.viewModel
import org.koin.core.context.startKoin
import org.koin.dsl.module

val appModule = module {
    // Shared OkHttpClient (timeouts/retry centralised in HttpClientFactory)
    single { HttpClientFactory.create() }

    // Json parser as a singleton
    single { Json { ignoreUnknownKeys = true } }

    // Weather data layer (network + parsing + mapping)
    single<WeatherRepository> { DefaultWeatherRepository(get(), get()) }

    // News data layer (RSS fetch + XML parsing)
    single<NewsRepository> { DefaultNewsRepository(get()) }

    // MainScreenViewModel with injected dependencies
    viewModel { MainScreenViewModel(get(), get(), get()) }
}

class WeatherApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidContext(this@WeatherApplication)
            modules(appModule)
        }
        NotificationHelper.createChannels(this)
        WeatherRefreshWorker.schedule(this)
    }
}
```

**File: WeatherRefreshWorker.kt**

```
package com.example.aiweathermonitor

import android.content.Context
import androidx.work.CoroutineWorker
import androidx.work.ExistingPeriodicWorkPolicy
import androidx.work.PeriodicWorkRequestBuilder
import androidx.work.WorkManager
import androidx.work.WorkerParameters
import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.data.models.OpenMeteoResponse
import com.example.aiweathermonitor.data.models.WeatherApiForecastResponse
import com.example.aiweathermonitor.util.AppLogger
import com.example.aiweathermonitor.util.UrlBuilder
import kotlinx.serialization.json.Json
import okhttp3.OkHttpClient
import okhttp3.Request
import java.text.SimpleDateFormat
import java.util.*
import java.util.concurrent.TimeUnit

private const val TAG = "WeatherRefreshWorker"
private const val WORK_NAME = "weather_periodic_refresh"

class WeatherRefreshWorker(
    appContext: Context,
    params: WorkerParameters
) : CoroutineWorker(appContext, params) {

    private val client = OkHttpClient()
    private val json = Json { ignoreUnknownKeys = true }

    override suspend fun doWork(): Result {
        return try {
            val prefs = applicationContext.getSharedPreferences(
                WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
Context.MODE_PRIVATE
            )

            val cachedJson =
prefs.getString(WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY, null)
                ?: return Result.success()

            val state = try {
                json.decodeFromString(WeatherState.serializer(), cachedJson)
            } catch (e: Exception) {
                AppLogger.warning("WeatherRefreshWorker: corrupt cache,
skipping", TAG, e)
                return Result.success()
            }

            val lat = state.latitude.toFloat()
            val lon = state.longitude.toFloat()
            val apiKey = state.weatherApiKey.ifBlank {
                try { BuildConfig.WEATHER_API_KEY } catch (e: Exception)
{ "" }
            }

            // ── Fetch
──────────────────────────────────────────────────

            val waData: WeatherApiForecastResponse? = if
(apiKey.isNotBlank()) {
                runCatching {
                    val url = UrlBuilder.weatherApiForecast(lat, lon,
apiKey)

client.newCall(Request.Builder().url(url).build()).execute().use { r ->
```

```
                        if (!r.isSuccessful) null
                        else r.body?.string()?.let
{ json.decodeFromString<WeatherApiForecastResponse>(it) }
                    }
                }.getOrNull()
            } else null

            val omData: OpenMeteoResponse? = runCatching {
                val url = UrlBuilder.openMeteoForecast(lat, lon, hourly =
true, daily = true)

client.newCall(Request.Builder().url(url).build()).execute().use { r ->
                    if (!r.isSuccessful) null
                    else r.body?.string()?.let
{ json.decodeFromString<OpenMeteoResponse>(it) }
                }
            }.getOrNull()

            if (waData == null && omData == null) return Result.retry()

            // ── Persist minimal update
────────────────────────────────────
            val newTemp = waData?.current?.tempC ?:
omData?.current?.temperature2m ?: state.temperature
            val newCode = omData?.current?.weatherCode ?: state.weatherCode
            val newUv = waData?.current?.uv ?: omData?.current?.uv_index ?:
state.uvIndex
            val newWind = waData?.current?.windKph ?: state.windSpeed

            val updatedState = state.copy(
                temperature = newTemp,
                weatherCode = newCode,
                uvIndex = newUv,
                windSpeed = newWind,
                lastRefreshedTime = SimpleDateFormat("h:mm a",
Locale.getDefault()).format(Date()),
                smartSummary = generateSmartSummary(state)
            )

            prefs.edit().putString(
                WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY,
                json.encodeToString(WeatherState.serializer(), updatedState)
            ).apply()

            // ── Widget update
─────────────────────────────────────────────
            val widgetIntent = android.content.Intent(applicationContext,
WeatherWidgetProvider::class.java).apply {
                action =
android.appwidget.AppWidgetManager.ACTION_APPWIDGET_UPDATE
                val ids =
android.appwidget.AppWidgetManager.getInstance(applicationContext)
                    .getAppWidgetIds(android.content.ComponentName(applicati
onContext, WeatherWidgetProvider::class.java))

putExtra(android.appwidget.AppWidgetManager.EXTRA_APPWIDGET_IDS, ids)
                setPackage(applicationContext.packageName)
            }
            applicationContext.sendBroadcast(widgetIntent) // NOSONAR

            // ── Notifications
─────────────────────────────────────────────
            val cal = Calendar.getInstance()
            val hour = cal.get(Calendar.HOUR_OF_DAY)
            val todayStr = SimpleDateFormat("yyyy-MM-dd",
Locale.getDefault()).format(Date())
            val cityName = state.selectedCity.ifBlank { "your location" }
```

```
            // 1. Severe weather alert (from WeatherAPI alerts)
            val notifySevere = prefs.getBoolean("notif_severe", true)
            if (notifySevere) {
                val alertEvent = waData?.alerts?.alert?.firstOrNull()
                if (alertEvent != null && !alertEvent.event.isNullOrBlank())
{
                    val lastAlert = prefs.getString("last_alert_event",
null)
                    if (lastAlert != alertEvent.event) {
                        NotificationHelper.notifySevere(
                            applicationContext,
                            alertEvent.event,
                            alertEvent.headline ?: alertEvent.desc ?:
"Severe weather in your area"
                        )
                        prefs.edit().putString("last_alert_event",
alertEvent.event).apply()
                    }
                }
            }

            // 2. Rain in next ~30 min (first hourly slot ≥40% precip
chance)
            val notifyRain = prefs.getBoolean("notif_rain", true)
            if (notifyRain) {
                val rainSlot =
updatedState.hourlyForecast.take(3).firstOrNull { it.precipChance >= 40 }
                if (rainSlot != null) {
                    val lastRainNotifDate =
prefs.getString("last_rain_notif_date", null)
                    if (lastRainNotifDate != todayStr) {
                        NotificationHelper.notifyRainSoon(
                            applicationContext, cityName,
rainSlot.precipChance, rainSlot.hour
                        )
                        prefs.edit().putString("last_rain_notif_date",
todayStr).apply()
                    }
                }
            }

            // 3. Daily briefing at 7 AM (once per day)
            val notifyBriefing = prefs.getBoolean("notif_briefing", false)
            if (notifyBriefing && hour == 7) {
                val lastBriefingDate = prefs.getString("last_briefing_date",
null)
                if (lastBriefingDate != todayStr) {
                    val summary = updatedState.smartSummary.ifBlank {
                        "Today: ${newTemp.toInt()}° · $
{newCode.getWeatherCodeDescription()}"
                    }

NotificationHelper.notifyDailyBriefing(applicationContext, cityName,
summary)
                    prefs.edit().putString("last_briefing_date",
todayStr).apply()
                }
            }

            // 4. High UV alert (once per day when UV ≥ 8)
            val notifyUv = prefs.getBoolean("notif_uv", false)
            if (notifyUv && newUv >= 8f) {
                val lastUvDate = prefs.getString("last_uv_notif_date", null)
                if (lastUvDate != todayStr && hour in 9..11) {
                    NotificationHelper.notifyHighUv(applicationContext,
newUv)
                    prefs.edit().putString("last_uv_notif_date",
todayStr).apply()
```

```
                }
            }

            Result.success()
        } catch (e: Exception) {
            AppLogger.error("WeatherRefreshWorker failed", TAG, e)
            Result.retry()
        }
    }

    companion object {
        fun schedule(context: Context) {
            val request =
PeriodicWorkRequestBuilder<WeatherRefreshWorker>(30, TimeUnit.MINUTES)
                .build()
            WorkManager.getInstance(context).enqueueUniquePeriodicWork(
                WORK_NAME,
                ExistingPeriodicWorkPolicy.KEEP,
                request
            )
        }

        fun cancel(context: Context) {
            WorkManager.getInstance(context).cancelUniqueWork(WORK_NAME)
        }
    }
}
```

**File: WeatherState.kt**

```
package com.example.aiweathermonitor

import com.example.aiweathermonitor.data.models.PollenData
import com.example.aiweathermonitor.data.models.NewsArticle
import kotlinx.serialization.Serializable

// ── Weather code → description ──────────────────────────────────

fun Int.getWeatherCodeDescription(): String = when (this) {
    0 -> "Clear Sky"
    1, 2, 3 -> "Partly Cloudy"
    45, 48 -> "Fog / Rime Fog"
    51, 53, 55 -> "Drizzle"
    61, 63, 65 -> "Rainy"
    71, 73, 75 -> "Snowy"
    80, 81, 82 -> "Rain Showers"
    95, 96, 99 -> "Thunderstorm"
    else -> "Unstable / Varied"
}

// ── UV index → description ──────────────────────────────────────

fun Float.getUvDescription(): String = when {
    this < 3f  -> "Low"
    this < 6f  -> "Moderate"
    this < 8f  -> "High"
    this < 11f -> "Very High"
    else       -> "Extreme"
}

// ── AQI (US EPA 1-6) → description ──────────────────────────────

fun Int.getAqiDescription(): String = when (this) {
    1 -> "Good"
    2 -> "Moderate"
    3 -> "Unhealthy for Sensitive Groups"
    4 -> "Unhealthy"
    5 -> "Very Unhealthy"
    6 -> "Hazardous"
    else -> "Unknown"
}

// ── Data models ─────────────────────────────────────────────────

@Serializable
data class GeocodingResult(
    val name: String = "",
    val latitude: Double = 0.0,
    val longitude: Double = 0.0,
    val country: String? = null,
    val admin1: String? = null
)

fun createGeocodingResult(
    name: String?,
    latitude: Double?,
    longitude: Double?,
    country: String? = null,
    admin1: String? = null
): GeocodingResult = GeocodingResult(
    name = name ?: "",
    latitude = latitude ?: 0.0,
```

```
    longitude = longitude ?: 0.0,
    country = country,
    admin1 = admin1
)

@Serializable
data class HourForecast(
    val hour: String,
    val temperature: Float,
    val weatherCode: Int,
    val precipChance: Int = 0        // % chance of rain (0-100)
)

@Serializable
data class DayForecast(
    val day: String,
    val tempMin: Float,
    val tempMax: Float,
    val weatherCode: Int,
    val uvIndex: Float = 0f,         // peak UV for the day
    val precipChance: Int = 0        // % chance of rain (0-100)
)

@Serializable
data class WeatherState(
    // Geocoding & City
    val searchQuery: String = "",
    val selectedCity: String = "New York",
    val latitude: Double = 40.7128,
    val longitude: Double = -74.0060,
    val searchResults: List<GeocodingResult> = emptyList(),
    val isLoading: Boolean = false,
    val isSearching: Boolean = false,
    val errorMessage: String? = null,

    // WeatherAPI.com key — runtime override; build-time default from
BuildConfig
    val weatherApiKey: String = "",
    val showKeyDialog: Boolean = false,

    // Saved Cities & Units
    val savedCities: List<GeocodingResult> = emptyList(),
    val isCelsius: Boolean = true,
    val showSavedCitiesScreen: Boolean = false,

    // Current conditions
    val temperature: Float = 22.0f,
    val feelsLike: Float = 22.0f,
    val humidity: Float = 50.0f,
    val pressure: Float = 1013.0f,
    val windSpeed: Float = 12.0f,
    val windDir: String = "",        // e.g. "NNE"
    val visibility: Float = 10.0f,   // km
    val aqi: Int = 0,                // US EPA index 1-6 (0 = unknown)
    val uvIndex: Float = 3f,
    val weatherCode: Int = 0,
    val sunrise: String = "6:00 AM",
    val sunset: String = "6:00 PM",
    val sunriseMinutes: Int = 360,
    val sunsetMinutes: Int = 1080,

    // Forecasts
    val hourlyForecast: List<HourForecast> = emptyList(),
    val dailyForecast: List<DayForecast> = emptyList(),

    // Connectivity & cache
    val isOnline: Boolean = true,
    val lastRefreshedTime: String = "",
```

```
    val lastRefreshedEpoch: Long = 0L,   // System.currentTimeMillis() at
last successful fetch
    val smartSummary: String = "",      // Generated one-line briefing
    @kotlinx.serialization.Transient
    val pollenData: PollenData? = null,   // Pollen concentrations (not
persisted)

    // Alerts
    val alertEvent: String? = null,
    val alertHeadline: String? = null,
    val alertDescription: String? = null,

    // News (RSS) — not persisted; refreshed from the feed on each launch
    @kotlinx.serialization.Transient
    val newsArticles: List<NewsArticle> = emptyList(),        // weather
news for the city
    @kotlinx.serialization.Transient
    val localNewsArticles: List<NewsArticle> = emptyList(),   //
general/local news for the city
    @kotlinx.serialization.Transient
    val isNewsLoading: Boolean = false,
    @kotlinx.serialization.Transient
    val newsError: String? = null
)

// ── Smart summary generator
──────────────────────────────────────────────

fun generateSmartSummary(state: WeatherState): String {
    val parts = mutableListOf<String>()

    // Rain alert: find first hourly slot with precip >= 40%
    val rainHour = state.hourlyForecast.firstOrNull { it.precipChance >=
40 }
    if (rainHour != null) {
        parts.add("Carry an umbrella — rain likely around ${rainHour.hour}")
    }

    // UV peak warning
    if (state.uvIndex >= 8f) {
        parts.add("UV peaks at ${state.uvIndex.toInt()} — apply sunscreen")
    } else if (state.uvIndex >= 6f) {
        parts.add("Moderate UV today — sunscreen recommended")
    }

    // Temperature extremes
    when {
        state.temperature >= 38f -> parts.add("Extreme heat — stay hydrated
and limit outdoor activity")
        state.temperature >= 35f -> parts.add("Very hot today — stay cool
and drink water")
        state.temperature <= 0f  -> parts.add("Below zero — ice risk, drive
carefully")
        state.temperature <= 5f  -> parts.add("Near-freezing temps — dress
in layers")
    }

    // Wind advisory
    if (state.windSpeed >= 50f) {
        parts.add("Strong winds (${state.windSpeed.toInt()} km/h) — secure
loose objects outdoors")
    }

    // Air quality
    when (state.aqi) {
        4 -> parts.add("Air quality is unhealthy — limit prolonged outdoor
exertion")
```

```
        5, 6 -> parts.add("Very poor air quality — avoid outdoor activity if
possible")
    }

    // Visibility
    if (state.visibility < 2f) {
        parts.add("Dense fog — drive with caution")
    }

    // Good day
    if (parts.isEmpty()) {
        val code = state.weatherCode
        parts.add(when {
            code == 0 -> "Clear skies all day — great conditions"
            code in 1..2 -> "Mostly clear with light clouds — pleasant day
ahead"
            code in 3..45 -> "Overcast and calm — comfortable for outdoor
plans"
            else -> "Conditions look manageable today"
        })
    }
    return parts.take(3).joinToString("  •  ")
}
```

**File: WeatherWidgetProvider.kt**

```
package com.example.aiweathermonitor

import android.app.PendingIntent
import android.appwidget.AppWidgetManager
import android.appwidget.AppWidgetProvider
import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.widget.RemoteViews
import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.util.formatTempShort
import kotlinx.serialization.json.Json

class WeatherWidgetProvider : AppWidgetProvider() {

    override fun onUpdate(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetIds: IntArray
    ) {
        val state = loadState(context)
        for (id in appWidgetIds) {
            updateWidget(context, appWidgetManager, id, state)
        }
    }

    override fun onAppWidgetOptionsChanged(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetId: Int,
        newOptions: Bundle
    ) {
        updateWidget(context, appWidgetManager, appWidgetId,
loadState(context))
    }

    private fun loadState(context: Context): WeatherState? {
        val prefs =
context.getSharedPreferences(WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
Context.MODE_PRIVATE)
        val json =
prefs.getString(WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY, null) ?:
return null
        return try {
            Json { ignoreUnknownKeys =
true }.decodeFromString(WeatherState.serializer(), json)
        } catch (e: Exception) {
            null
        }
    }

    private fun updateWidget(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetId: Int,
        state: WeatherState?
    ) {
        val options = appWidgetManager.getAppWidgetOptions(appWidgetId)
        val minWidth =
options.getInt(AppWidgetManager.OPTION_APPWIDGET_MIN_WIDTH, 0)
        val use4x2 = minWidth >= 250  // ~4 cells wide

        val views = if (use4x2) build4x2Views(context, state) else
build2x2Views(context, state)

        // Tap → open app
```

```
        val tapIntent = PendingIntent.getActivity(
            context, 0,
            Intent(context, MainActivity::class.java).apply { flags =
Intent.FLAG_ACTIVITY_SINGLE_TOP },
            PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
        )
        val containerId = if (use4x2) R.id.widget_container_4x2 else
R.id.widget_container
        views.setOnClickPendingIntent(containerId, tapIntent)

        appWidgetManager.updateAppWidget(appWidgetId, views)
    }

    // ── 2×2 compact layout
──────────────────────────────────────────────────
    private fun build2x2Views(context: Context, state: WeatherState?):
RemoteViews {
        val views = RemoteViews(context.packageName,
R.layout.weather_widget_layout)
        if (state == null) {
            views.setTextViewText(R.id.widget_temp, "--°")
            views.setTextViewText(R.id.widget_city, "Sunnyside")
            views.setTextViewText(R.id.widget_description, "Open app to load
weather")
            views.setTextViewText(R.id.widget_online_status, "")
            return views
        }
        val tempStr = formatTempShort(state.temperature, state.isCelsius)
        views.setTextViewText(R.id.widget_temp, tempStr)
        views.setTextViewText(R.id.widget_city, state.selectedCity)
        views.setTextViewText(R.id.widget_description,
state.weatherCode.getWeatherCodeDescription())
        views.setTextViewText(R.id.widget_online_status, if (state.isOnline)
"" else "Offline")
        return views
    }

    // ── 4×2 wide layout
──────────────────────────────────────────────────────
    private fun build4x2Views(context: Context, state: WeatherState?):
RemoteViews {
        val views = RemoteViews(context.packageName,
R.layout.weather_widget_4x2_layout)
        if (state == null) {
            views.setTextViewText(R.id.widget_temp_4x2, "--°")
            views.setTextViewText(R.id.widget_city_4x2, "Sunnyside")
            views.setTextViewText(R.id.widget_desc_4x2, "Open app to load
weather")
            views.setTextViewText(R.id.widget_hl_4x2, "")
            views.setTextViewText(R.id.widget_uv_4x2, "")
            return views
        }

        // Header
        views.setTextViewText(R.id.widget_temp_4x2,
formatTempShort(state.temperature, state.isCelsius))
        views.setTextViewText(R.id.widget_city_4x2, state.selectedCity)
        views.setTextViewText(R.id.widget_desc_4x2,
state.weatherCode.getWeatherCodeDescription())

        val today = state.dailyForecast.firstOrNull()
        if (today != null) {
            val hiStr = formatTempShort(today.tempMax, state.isCelsius)
            val loStr = formatTempShort(today.tempMin, state.isCelsius)
            views.setTextViewText(R.id.widget_hl_4x2, "H: $hiStr · L:
$loStr")
        } else {
```

```
            views.setTextViewText(R.id.widget_hl_4x2, "")
        }

        val uvLabel = if (state.uvIndex > 0f) "UV ${state.uvIndex.toInt()} ·
${state.uvIndex.getUvDescription()}" else ""
        views.setTextViewText(R.id.widget_uv_4x2, uvLabel)

        // Hourly strip — up to 5 slots
        val hourlySlots = listOf(
            Triple(R.id.widget_hour_0_time, R.id.widget_hour_0_temp,
R.id.widget_hour_0_precip) to R.id.widget_hour_0_icon,
            Triple(R.id.widget_hour_1_time, R.id.widget_hour_1_temp,
R.id.widget_hour_1_precip) to R.id.widget_hour_1_icon,
            Triple(R.id.widget_hour_2_time, R.id.widget_hour_2_temp,
R.id.widget_hour_2_precip) to R.id.widget_hour_2_icon,
            Triple(R.id.widget_hour_3_time, R.id.widget_hour_3_temp,
R.id.widget_hour_3_precip) to R.id.widget_hour_3_icon,
            Triple(R.id.widget_hour_4_time, R.id.widget_hour_4_temp,
R.id.widget_hour_4_precip) to R.id.widget_hour_4_icon
        )

        hourlySlots.forEachIndexed { i, (ids, iconId) ->
            val hour = state.hourlyForecast.getOrNull(i)
            if (hour != null) {
                views.setTextViewText(ids.first, if (i == 0) "Now" else
hour.hour)
                views.setTextViewText(ids.second,
formatTempShort(hour.temperature, state.isCelsius))
                views.setTextViewText(ids.third, if (hour.precipChance > 0)
"${hour.precipChance}%" else "")
                views.setTextViewText(iconId,
weatherCodeToEmoji(hour.weatherCode))
            } else {
                views.setTextViewText(ids.first, "")
                views.setTextViewText(ids.second, "")
                views.setTextViewText(ids.third, "")
                views.setTextViewText(iconId, "")
            }
        }

        return views
    }

    private fun weatherCodeToEmoji(code: Int): String = when (code) {
        0 -> "☀"
        1, 2 -> "▫"
        3 -> "☁"
        45, 48 -> "▫"
        51, 53, 55, 56, 57 -> "▫"
        61, 63, 65, 66, 67 -> "▫"
        71, 73, 75, 77 -> "❄"
        80, 81, 82 -> "▫"
        85, 86 -> "▫"
        95 -> "▫"
        96, 99 -> "▫"
        else -> "▫"
    }
}
```

**File: audio/WeatherSound.kt**

```
package com.example.aiweathermonitor.audio

import androidx.annotation.RawRes
import com.example.aiweathermonitor.R

/**
 * A single ambient audio layer: a looping raw resource played at [volume]
(0f..1f).
 * Multiple layers are mixed together at runtime to build a soundscape.
 */
data class SoundLayer(@RawRes val res: Int, val volume: Float)

/**
 * Maps a WMO weather code to a set of layered ambient sounds.
 *
 * This mirrors [com.example.aiweathermonitor.ui.getWeatherLottieRes] so the
audio
 * always matches the animated scene on screen. All clips are procedurally
 * generated, royalty-free loops bundled in res/raw.
 */
fun getWeatherSoundscape(code: Int): List<SoundLayer> = when (code) {
    // 0 — Clear sky: birdsong over a faint breeze (the "sunny" ambience)
    0 -> listOf(
        SoundLayer(R.raw.amb_birds, 0.70f),
        SoundLayer(R.raw.amb_wind, 0.18f)
    )

    // 1,2,3 — Partly cloudy / cloudy: gentle wind layered with birds
    1, 2, 3 -> listOf(
        SoundLayer(R.raw.amb_wind, 0.55f),
        SoundLayer(R.raw.amb_birds, 0.45f)
    )

    // 45,48 — Fog / rime fog: soft, low wind only
    45, 48 -> listOf(
        SoundLayer(R.raw.amb_wind, 0.40f)
    )

    // 51..82 — Drizzle / rain / rain showers: rain bed with a faint breeze
    51, 53, 55, 61, 63, 65, 80, 81, 82 -> listOf(
        SoundLayer(R.raw.amb_rain, 0.95f),
        SoundLayer(R.raw.amb_wind, 0.25f)
    )

    // 71,73,75 — Snow: hushed, gentle wind
    71, 73, 75 -> listOf(
        SoundLayer(R.raw.amb_wind, 0.45f)
    )

    // 95,96,99 — Thunderstorm: rain + rolling thunder + wind
    95, 96, 99 -> listOf(
        SoundLayer(R.raw.amb_rain, 0.80f),
        SoundLayer(R.raw.amb_thunder, 1.0f),
        SoundLayer(R.raw.amb_wind, 0.40f)
    )

    // Fallback (varied / unknown): wind + birds
    else -> listOf(
        SoundLayer(R.raw.amb_wind, 0.50f),
        SoundLayer(R.raw.amb_birds, 0.40f)
    )
}
```

**File: audio/WeatherSoundPlayer.kt**

```
package com.example.aiweathermonitor.audio

import android.content.Context
import android.media.AudioAttributes
import android.media.MediaPlayer
import com.example.aiweathermonitor.util.AppLogger
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.cancel
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import java.util.concurrent.ConcurrentHashMap

/**
 * Plays layered, looping ambient weather sound that matches the current
weather code.
 *
 * Each weather condition resolves (via [getWeatherSoundscape]) to one or
more looping
 * [SoundLayer]s that are mixed together. Switching conditions crossfades
layers in and
 * out so transitions are smooth.
 *
 * Lifecycle: the owner is responsible for calling [pause]/[resume] when the
app goes to
 * the background/foreground and [release] when done (e.g. from a Compose
DisposableEffect).
 *
 * Always construct with the *application* context to avoid leaking an
Activity.
 */
class WeatherSoundPlayer(context: Context) {

    private val TAG = "WeatherSoundPlayer"
    private val appContext: Context = context.applicationContext
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    private class Active(
        val player: MediaPlayer,
        var current: Float = 0f,
        var target: Float = 0f,
        var job: Job? = null
    )

    /** res id -> currently playing layer */
    private val active = ConcurrentHashMap<Int, Active>()

    @Volatile private var masterVolume: Float = 1f
    @Volatile private var currentCode: Int? = null
    @Volatile private var paused: Boolean = false

    /** Overall volume multiplier (0f..1f) applied on top of each layer's
own volume. */
    fun setMasterVolume(v: Float) {
        masterVolume = v.coerceIn(0f, 1f)
        active.values.forEach { applyVolume(it) }
    }

    /**
     * Switch the soundscape to match [code], crossfading layers that
change.
     * Safe to call repeatedly; redundant calls for the same code are
ignored.
     */
```

```
    fun play(code: Int) {
        if (code == currentCode && !paused && active.isNotEmpty()) return
        currentCode = code
        paused = false

        val target = getWeatherSoundscape(code).associate { it.res to
it.volume }

        // Fade out + release any layer no longer needed
        active.keys.filter { it !in target }.forEach { res ->
fadeOutAndRelease(res) }

        // Add new layers or re-target existing ones
        target.forEach { (res, vol) ->
            val a = active[res]
            if (a == null) startLayer(res, vol) else fadeTo(a, vol)
        }
    }

    /** Pause all layers (keeps them ready to resume from the same
position). */
    fun pause() {
        paused = true
        active.values.forEach { a ->
            try {
                if (a.player.isPlaying) a.player.pause()
            } catch (e: IllegalStateException) {
                AppLogger.error("pause failed", TAG, e)
            }
        }
    }

    /** Resume all layers previously paused. */
    fun resume() {
        if (currentCode == null) return
        paused = false
        active.values.forEach { a ->
            try {
                if (!a.player.isPlaying) a.player.start()
            } catch (e: IllegalStateException) {
                AppLogger.error("resume failed", TAG, e)
            }
        }
    }

    /** Stop and release every layer (the player can still be reused via
[play]). */
    fun stop() {
        currentCode = null
        paused = false
        active.keys.toList().forEach { res ->
            active.remove(res)?.let { a ->
                a.job?.cancel()
                releasePlayer(a.player)
            }
        }
    }

    /** Permanently release all resources. The instance must not be used
afterwards. */
    fun release() {
        stop()
        scope.cancel()
    }

    // ── internals
```

```
    private fun startLayer(res: Int, targetVol: Float) {
        scope.launch {
            try {
                // Build manually so audio attributes are set BEFORE
prepare().
                // MediaPlayer.create() prepares immediately, which would
make a
                // later setAudioAttributes() call throw on some Android
versions.
                val mp = MediaPlayer()
                mp.setAudioAttributes(
                    AudioAttributes.Builder()
                        .setUsage(AudioAttributes.USAGE_MEDIA)
                        .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
                        .build()
                )
                appContext.resources.openRawResourceFd(res).use { afd ->
                    mp.setDataSource(afd.fileDescriptor, afd.startOffset,
afd.length)
                }
                mp.isLooping = true
                mp.setVolume(0f, 0f)
                mp.prepare()

                // We may have been superseded (stopped / code changed)
while preparing.
                val stillWanted = currentCode?.let { code ->
                    getWeatherSoundscape(code).any { it.res == res }
                } ?: false
                if (!stillWanted) {
                    releasePlayer(mp)
                    return@launch
                }

                val a = Active(mp, current = 0f, target = targetVol)
                active[res] = a
                if (!paused) {
                    try {
                        mp.start()
                    } catch (e: IllegalStateException) {
                        AppLogger.error("start failed", TAG, e)
                    }
                }
                fadeTo(a, targetVol, fromZero = true)
            } catch (e: Exception) {
                AppLogger.error("startLayer($res) failed", TAG, e)
            }
        }
    }

    private fun fadeTo(a: Active, target: Float, durationMs: Long = 1200,
fromZero: Boolean = false) {
        a.target = target
        a.job?.cancel()
        a.job = scope.launch {
            val steps = 24
            val start = if (fromZero) 0f else a.current
            for (i in 1..steps) {
                val f = i.toFloat() / steps
                setLayerVol(a, start + (target - start) * f)
                delay(durationMs / steps)
            }
            setLayerVol(a, target)
        }
    }

    private fun fadeOutAndRelease(res: Int) {
        val a = active.remove(res) ?: return
```

```
            a.job?.cancel()
            a.job = scope.launch {
                val steps = 18
                val start = a.current
                for (i in 1..steps) {
                    val f = i.toFloat() / steps
                    setLayerVol(a, start * (1f - f))
                    delay(700L / steps)
                }
                releasePlayer(a.player)
            }
        }

        private fun setLayerVol(a: Active, v: Float) {
            a.current = v.coerceIn(0f, 1f)
            applyVolume(a)
        }

        private fun applyVolume(a: Active) {
            val vol = (a.current * masterVolume).coerceIn(0f, 1f)
            try {
                a.player.setVolume(vol, vol)
            } catch (e: IllegalStateException) {
                AppLogger.error("setVolume failed", TAG, e)
            }
        }

        private fun releasePlayer(mp: MediaPlayer) {
            try {
                if (mp.isPlaying) mp.stop()
            } catch (e: IllegalStateException) {
                // already stopped / not started — ignore
            }
            try {
                mp.release()
            } catch (e: Exception) {
                AppLogger.error("release failed", TAG, e)
            }
        }
    }
```

**File: config/WeatherApiConfig.kt**

```
package com.example.aiweathermonitor.config

/**
 * Centralized configuration for Weather API endpoints, timing, and
defaults.
 * Prevents magic numbers and strings scattered throughout the codebase.
 */
object WeatherApiConfig {
    // Timing Configuration
    const val SEARCH_DEBOUNCE_MS = 400
    const val SEARCH_MIN_CHARS = 2
    const val SEARCH_RESULT_LIMIT = 10
    const val API_TIMEOUT_SECONDS = 30L
    const val FORECAST_HOURS_LIMIT = 24
    const val FORECAST_DAYS_LIMIT = 14
    const val REFRESH_INTERVAL_MINUTES = 30
    const val CACHE_EXPIRATION_MS = 3600000L  // 1 hour in milliseconds

    // WeatherAPI.com free tier supports up to 3-day forecast
    const val WEATHER_API_FORECAST_DAYS = 3

    // News (RSS) configuration — how many headlines to keep from the feed
    const val NEWS_ITEM_LIMIT = 15

    // API Endpoints
    object Endpoints {
        const val OPEN_METEO_GEOCODING = "https://geocoding-api.open-
meteo.com/v1/search"
        const val OPEN_METEO_FORECAST     =
"https://api.open-meteo.com/v1/forecast"
        const val OPEN_METEO_AIR_QUALITY  = "https://air-quality-api.open-
meteo.com/v1/air-quality"
        const val WEATHER_API_FORECAST =
"https://api.weatherapi.com/v1/forecast.json"
        const val WEATHER_API_SEARCH   =
"https://api.weatherapi.com/v1/search.json"

        // Google News RSS — weather-related top stories. HTTPS so it
satisfies
        // the app's network-security config (no cleartext allowed).
        const val NEWS_RSS_FEED =
            "https://news.google.com/rss/search?q=weather&hl=en-
US&gl=US&ceid=US:en"
    }

    // Default Values
    object Defaults {
        const val DEFAULT_CITY = "New York"
        const val DEFAULT_LATITUDE = 40.7128
        const val DEFAULT_LONGITUDE = -74.0060
        const val DEFAULT_SUNRISE = "6:00 AM"
        const val DEFAULT_SUNSET = "6:00 PM"
        const val DEFAULT_SUNRISE_MINUTES = 360
        const val DEFAULT_SUNSET_MINUTES = 1080
        const val DEFAULT_TEMPERATURE = 22f
        const val DEFAULT_HUMIDITY = 50f
        const val DEFAULT_PRESSURE = 1013f
        const val DEFAULT_WIND_SPEED = 12f
        const val DEFAULT_AQI = 45
        const val DEFAULT_UV_INDEX = 3f
    }

    // Error Messages
    object ErrorMessages {
        const val ERROR_NO_INTERNET = "No internet connection - using cached
data"
```

```
        const val ERROR_INVALID_API_KEY = "Invalid API key configuration"
        const val ERROR_LOCATION_PERMISSION = "Location permission required"
        const val ERROR_TIMEOUT = "Request timeout - please try again"
        const val ERROR_SERVER = "Server error - please try again later"
        const val ERROR_INVALID_DATA = "Invalid data received from server"
        const val ERROR_SEARCH_FAILED = "Search failed - please try another
query"
    }

    // Shared Preferences Keys
    object SharedPrefsKeys {
        const val PREFS_NAME = "weather_cache_prefs"
        const val WEATHER_STATE_KEY = "cached_weather_state"
        const val LAST_REFRESH_KEY = "last_refresh_time"
        const val API_KEY_KEY = "weatherapi_key"
    }

    // Logging Tags
    object LogTags {
        const val VIEW_MODEL = "MainScreenViewModel"
        const val NETWORK = "NetworkConnectivity"
        const val REPOSITORY = "WeatherRepository"
        const val WIDGET = "WeatherWidget"
    }
}
```

**File: data/DataRepository.kt**

```
package com.example.aiweathermonitor.data

import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow

/**
 * Repository interface for managing application data access.
 * Provides a reactive interface for consuming data through Flows.
 */
interface DataRepository {
    /**
     * Reactive stream of data updates.
     * Emits a list of strings representing application data.
     * Collectors will receive updates whenever data changes.
     */
    val data: Flow<List<String>>
}

/**
 * Default implementation of [DataRepository].
 * Currently provides a simple static list of data.
 *
 * @see DataRepository
 */
class DefaultDataRepository : DataRepository {
    /**
     * Static data flow that emits a single list containing "Android".
     * In a production app, this would fetch data from network/database.
     */
    override val data: Flow<List<String>> = flow {
        emit(listOf("Android"))
    }
}
```

**File: data/NewsRepository.kt**

```
package com.example.aiweathermonitor.data

import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.data.models.NewsArticle
import com.example.aiweathermonitor.util.AppLogger
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.Request
import org.xmlpull.v1.XmlPullParser
import java.io.StringReader

private const val TAG = "NewsRepository"

/**
 * Data-access layer for news headlines. Owns the RSS fetch + XML parsing so
the
 * ViewModel stays focused on UI state. Pure and side-effect free (no
Android
 * Context), making it unit-testable with a mock [OkHttpClient].
 */
interface NewsRepository {
    /**
     * Fetches and parses the configured RSS feed, returning up to
     * [WeatherApiConfig.NEWS_ITEM_LIMIT] articles. Honours an optional
[feedUrl]
     * override (defaults to the weather news feed in config).
     */
    suspend fun fetchNews(
        feedUrl: String = WeatherApiConfig.Endpoints.NEWS_RSS_FEED
    ): Result<List<NewsArticle>>
}

class DefaultNewsRepository(
    private val client: OkHttpClient,
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO
) : NewsRepository {

    override suspend fun fetchNews(feedUrl: String):
Result<List<NewsArticle>> =
        withContext(ioDispatcher) {
            try {
                val request = Request.Builder()
                    .url(feedUrl)
                    .header("User-Agent", "AIWeatherMonitor/1.0 (Android)")
                    .build()
                val xml = client.newCall(request).execute().use { response
->
                    if (!response.isSuccessful) {
                        return@withContext Result.failure(
                            Exception("RSS feed error: ${response.code}")
                        )
                    }
                    response.body?.string()
                        ?: return@withContext
Result.failure(Exception("Empty RSS response"))
                }
                Result.success(parseRss(xml))
            } catch (e: Exception) {
                AppLogger.error("Failed to fetch news", TAG, e)
                Result.failure(e)
            }
        }

    /**
```

```
     * Parses an RSS 2.0 document into [NewsArticle]s using a pull parser.
     * Reads `<title>`, `<link>`, `<pubDate>`, `<description>`, and
`<source>`
     * inside each `<item>`. Skips the channel-level metadata.
     */
    private fun parseRss(xml: String): List<NewsArticle> {
        val articles = mutableListOf<NewsArticle>()
        val parser = android.util.Xml.newPullParser().apply {
            setFeature(XmlPullParser.FEATURE_PROCESS_NAMESPACES, false)
            setInput(StringReader(xml))
        }

        var inItem = false
        var title = ""
        var link = ""
        var pubDate = ""
        var description = ""
        var source = ""

        var event = parser.eventType
        while (event != XmlPullParser.END_DOCUMENT) {
            when (event) {
                XmlPullParser.START_TAG -> {
                    when (parser.name?.lowercase()) {
                        "item" -> {
                            inItem = true
                            title = ""; link = ""; pubDate = ""; description
= ""; source = ""
                        }
                        "title"       -> if (inItem) title =
parser.nextTextSafe()
                        "link"        -> if (inItem) link =
parser.nextTextSafe()
                        "pubdate"     -> if (inItem) pubDate =
parser.nextTextSafe()
                        "description" -> if (inItem) description =
stripHtml(parser.nextTextSafe())
                        "source"      -> if (inItem) source =
parser.nextTextSafe()
                    }
                }
                XmlPullParser.END_TAG -> {
                    if (parser.name?.lowercase() == "item" && inItem) {
                        inItem = false
                        if (title.isNotBlank() && link.isNotBlank()) {
                            articles.add(
                                NewsArticle(
                                    title = cleanTitle(title),
                                    link = link.trim(),
                                    source = source.trim(),
                                    pubDate = pubDate.trim(),
                                    description = description.trim()
                                )
                            )
                        }
                    }
                }
            }
            event = parser.next()
        }
        return sortAndLimit(articles)
    }

    /** RFC-1123 pubDate (e.g. "Wed, 03 Jun 2026 10:17:35 GMT") → epoch
millis; 0 if missing/unparseable. */
    private fun parsePubMillis(pubDate: String): Long = runCatching {
        if (pubDate.isBlank()) 0L
```

```
        else java.text.SimpleDateFormat("EEE, dd MMM yyyy HH:mm:ss zzz",
java.util.Locale.US)
            .parse(pubDate)?.time ?: 0L
    }.getOrDefault(0L)

    /**
     * Newest-first ordering. Drops clearly-stale items (older than 30 days)
so old
     * Google-News matches (some from years ago) don't surface — but falls
back to
     * the full sorted list if nothing recent is left.
     */
    private fun sortAndLimit(articles: List<NewsArticle>): List<NewsArticle>
{
        val dated = articles.map { it to parsePubMillis(it.pubDate) }
            .sortedByDescending { it.second }
        val cutoff = System.currentTimeMillis() - 30L * 24 * 60 * 60 * 1000
        val recent = dated.filter { it.second == 0L || it.second >= cutoff }
        return (if (recent.isNotEmpty()) recent else dated)
            .take(WeatherApiConfig.NEWS_ITEM_LIMIT)
            .map { it.first }
    }

    /** Safely reads the text content of the current element, returning ""
on any issue. */
    private fun XmlPullParser.nextTextSafe(): String = runCatching {
        if (next() == XmlPullParser.TEXT) text?.trim().orEmpty() else ""
    }.getOrDefault("")

    /** Google News titles are "Headline - Publisher"; trim a trailing
publisher suffix. */
    private fun cleanTitle(raw: String): String {
        val decoded = stripHtml(raw)
        val idx = decoded.lastIndexOf(" - ")
        return if (idx > 0 && idx > decoded.length - 60)
decoded.substring(0, idx).trim()
        else decoded
    }

    /** Removes HTML tags and decodes a few common entities — keeps the repo
Android-UI free. */
    private fun stripHtml(input: String): String =
        input.replace(Regex("<[^>]*>"), " ")
            .replace(" ", " ")
            .replace("&amp;", "&")
            .replace("&quot;", "\"")
            .replace("&#39;", "'")
            .replace("&apos;", "'")
            .replace("&lt;", "<")
            .replace("&gt;", ">")
            .replace(Regex("\\s+"), " ")
            .trim()
}
```

**File: data/WeatherRepository.kt**

```
package com.example.aiweathermonitor.data

import com.example.aiweathermonitor.DayForecast
import com.example.aiweathermonitor.GeocodingResult
import com.example.aiweathermonitor.HourForecast
import com.example.aiweathermonitor.WeatherState
import com.example.aiweathermonitor.createGeocodingResult
import com.example.aiweathermonitor.generateSmartSummary
import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.data.models.OpenMeteoGeocodingResponse
import com.example.aiweathermonitor.data.models.OpenMeteoResponse
import com.example.aiweathermonitor.data.models.PollenData
import com.example.aiweathermonitor.data.models.PollenResponse
import com.example.aiweathermonitor.data.models.WeatherApiForecastResponse
import com.example.aiweathermonitor.data.models.WeatherApiSearchResult
import com.example.aiweathermonitor.data.models.mapWeatherApiCodeToWmo
import com.example.aiweathermonitor.util.AppLogger
import com.example.aiweathermonitor.util.UrlBuilder
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import kotlinx.serialization.json.Json
import okhttp3.OkHttpClient
import okhttp3.Request
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

private const val TAG = "WeatherRepository"
private const val TIME_DEFAULT_SUNRISE = "6:00 AM"
private const val TIME_DEFAULT_SUNSET = "6:00 PM"

/**
 * Data-access layer for weather. Owns all networking, JSON parsing, and the
 * WeatherAPI.com <-> Open-Meteo merge so the ViewModel can stay focused on
UI
 * state. Pure and side-effect free (no notifications, no Android Context)
which
 * makes every method unit-testable with a mock [OkHttpClient].
 */
interface WeatherRepository {

    /**
     * Searches for cities. Tries Open-Meteo geocoding first, then falls
back to
     * WeatherAPI.com search when [apiKey] is present and Open-Meteo is
empty.
     */
    suspend fun searchCities(query: String, apiKey: String):
Result<List<GeocodingResult>>

    /**
     * Fetches current conditions + forecast for the given coordinates and
merges
     * the result into [base], returning a fully populated [WeatherState]
     * (isLoading = false, smartSummary regenerated, pollen attached).
     */
    suspend fun getWeather(
        base: WeatherState,
        cityName: String,
        latitude: Float,
        longitude: Float,
        apiKey: String
    ): Result<WeatherState>
}
```

```
class DefaultWeatherRepository(
    private val client: OkHttpClient,
    private val jsonParser: Json = Json { ignoreUnknownKeys = true },
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO
) : WeatherRepository {

    // ── City search ──────────────────────────────────────────────────

    override suspend fun searchCities(
        query: String,
        apiKey: String
    ): Result<List<GeocodingResult>> = withContext(ioDispatcher) {
        try {
            // Primary: Open-Meteo geocoding
            val omResults = runCatching {
                val url = UrlBuilder.openMeteoSearch(query,
WeatherApiConfig.SEARCH_RESULT_LIMIT)
                val body =
client.newCall(Request.Builder().url(url).build()).execute()
                    .use { r -> if (!r.isSuccessful) null else
r.body?.string() }
                if (!body.isNullOrBlank()) {

jsonParser.decodeFromString<OpenMeteoGeocodingResponse>(body)
                        .results
                        ?.map {
                            createGeocodingResult(
                                name = it.name,
                                latitude = it.latitude,
                                longitude = it.longitude,
                                country = it.country,
                                admin1 = it.admin1
                            )
                        } ?: emptyList()
                } else emptyList()
            }.getOrElse { emptyList() }

            if (omResults.isNotEmpty()) {
                return@withContext Result.success(omResults)
            }

            // Fallback: WeatherAPI search (only if a key is configured)
            if (apiKey.isBlank()) return@withContext
Result.success(emptyList())
            val waResults = runCatching {
                val url = UrlBuilder.weatherApiSearch(query, apiKey)
                val body =
client.newCall(Request.Builder().url(url).build()).execute()
                    .use { r -> if (!r.isSuccessful) null else
r.body?.string() }
                if (!body.isNullOrBlank()) {

jsonParser.decodeFromString<List<WeatherApiSearchResult>>(body)
                        .map {
                            createGeocodingResult(
                                name = it.name,
                                latitude = it.lat,
                                longitude = it.lon,
                                country = it.country,
                                admin1 = it.region
                            )
                        }
                } else emptyList()
            }.getOrElse { emptyList() }
            Result.success(waResults)
        } catch (e: Exception) {
            AppLogger.error("City search failed", TAG, e)
```

```
            Result.failure(e)
        }
    }

    // ── Weather fetch orchestration ──────────────────────────────

    override suspend fun getWeather(
        base: WeatherState,
        cityName: String,
        latitude: Float,
        longitude: Float,
        apiKey: String
    ): Result<WeatherState> = withContext(ioDispatcher) {
        try {
            var state: WeatherState? = null

            // 1. WeatherAPI.com (richer current data) + Open-Meteo extended
forecast
            if (apiKey.isNotBlank()) {
                try {
                    val waData = fetchWeatherApiData(apiKey, latitude,
longitude)
                    val omData = runCatching
{ fetchOpenMeteoWeather(latitude, longitude) }
                        .onFailure { AppLogger.warning("Open-Meteo extended
unavailable: ${it.message}", TAG) }
                        .getOrNull()
                    state = mapWeatherApiToState(base, waData, omData,
cityName)
                } catch (e: Exception) {
                    AppLogger.warning("WeatherAPI.com failed, falling back
to Open-Meteo: ${e.message}", TAG)
                }
            }

            // 2. Open-Meteo only fallback
            if (state == null) {
                val data = fetchOpenMeteoWeather(latitude, longitude)
                state = mapOpenMeteoToState(base, data, cityName)
            }

            // Regenerate smart summary + attach pollen after a successful
fetch
            val pollen = runCatching { fetchPollenData(latitude,
longitude) }.getOrNull()
            val finalState = state.copy(
                smartSummary = generateSmartSummary(state),
                lastRefreshedEpoch = System.currentTimeMillis(),
                pollenData = pollen
            )
            Result.success(finalState)
        } catch (e: Exception) {
            AppLogger.error("Failed to fetch weather", TAG, e)
            Result.failure(e)
        }
    }

    // ── Network fetchers ─────────────────────────────────────────

    private suspend fun fetchWeatherApiData(
        apiKey: String,
        latitude: Float,
        longitude: Float
    ): WeatherApiForecastResponse = withContext(ioDispatcher) {
        val url = UrlBuilder.weatherApiForecast(latitude, longitude, apiKey)
```

```
        client.newCall(Request.Builder().url(url).build()).execute().use
{ response ->
            when (response.code) {
                401, 403 -> throw Exception("Invalid WeatherAPI key. Check
Settings.")
                else -> if (!response.isSuccessful) throw
Exception("WeatherAPI.com error: ${response.code}")
            }
            val body = response.body?.string() ?: throw Exception("Empty
WeatherAPI.com response")
            jsonParser.decodeFromString<WeatherApiForecastResponse>(body)
        }
    }

    private suspend fun fetchOpenMeteoWeather(
        latitude: Float,
        longitude: Float
    ): OpenMeteoResponse = withContext(ioDispatcher) {
        val url = UrlBuilder.openMeteoForecast(latitude, longitude, hourly =
true, daily = true)
        client.newCall(Request.Builder().url(url).build()).execute().use
{ response ->
            if (!response.isSuccessful) throw Exception("Open-Meteo error: $
{response.code}")
            val body = response.body?.string() ?: throw Exception("Empty
Open-Meteo response")
            jsonParser.decodeFromString<OpenMeteoResponse>(body)
        }
    }

    private suspend fun fetchPollenData(latitude: Float, longitude: Float):
PollenData? =
        withContext(ioDispatcher) {
            runCatching {
                val url = UrlBuilder.openMeteoPollen(latitude, longitude)

client.newCall(Request.Builder().url(url).build()).execute().use { r ->
                    if (!r.isSuccessful) return@withContext null
                    val body = r.body?.string() ?: return@withContext null
                    val resp =
jsonParser.decodeFromString<PollenResponse>(body)
                    val h = resp.hourly ?: return@withContext null
                    // Average the first 8 hours (daytime today)
                    fun avg(list: List<Float?>?): Float =
                        list?.take(8)?.filterNotNull()?.average()?.toFloat()
?: 0f
                    PollenData(
                        alder   = avg(h.alderPollen),
                        birch   = avg(h.birchPollen),
                        grass   = avg(h.grassPollen),
                        mugwort = avg(h.mugwortPollen),
                        olive   = avg(h.olivePollen),
                        ragweed = avg(h.ragweedPollen)
                    )
                }
            }.getOrNull()
        }

    // ── State mappers (side-effect free)
──────────────────────────────────────

    private fun mapWeatherApiToState(
        currentState: WeatherState,
        waData: WeatherApiForecastResponse,
        omData: OpenMeteoResponse?,
        cityName: String
    ): WeatherState {
        val current = waData.current
```

```
        val forecastDays = waData.forecast?.forecastday ?: emptyList()
        val todayDay = forecastDays.firstOrNull()

        val tempVal      = current?.tempC ?: 0f
        val feelsLikeVal = current?.feelsLikeC ?: tempVal
        val humidityVal  = current?.humidity?.toFloat() ?: 0f
        val pressureVal  = current?.pressureMb ?: 1013f
        val windSpeedVal = current?.windKph ?: 0f
        val windDirVal   = current?.windDir ?: ""
        val visibilityVal = current?.visKm ?: 10f
        val uvVal        = current?.uv ?: todayDay?.day?.uv ?: 0f
        val aqiVal       = current?.airQuality?.usEpaIndex ?: 0
        val wmoCode      = mapWeatherApiCodeToWmo(current?.condition?.code)

        val astro = todayDay?.astro
        val parsedSunrise = astro?.sunrise?.trim() ?: TIME_DEFAULT_SUNRISE
        val parsedSunset  = astro?.sunset?.trim()  ?: TIME_DEFAULT_SUNSET
        val sunriseMinutes = parseFormattedTimeToMins(parsedSunrise)
        val sunsetMinutes  = parseFormattedTimeToMins(parsedSunset)

        val allHours = forecastDays.flatMap { it.hour ?: emptyList() }
        val nowHourStr = SimpleDateFormat("yyyy-MM-dd HH",
Locale.getDefault()).format(Date())
        var startIdx = allHours.indexOfFirst { h ->
            h.time?.substring(0, minOf(13, h.time.length)) == nowHourStr
        }
        if (startIdx == -1) startIdx = 0

        val hourlyList = mutableListOf<HourForecast>()
        for (i in startIdx until minOf(startIdx +
WeatherApiConfig.FORECAST_HOURS_LIMIT, allHours.size)) {
            val h = allHours[i]
            hourlyList.add(
                HourForecast(
                    hour = if (i == startIdx) "Now" else parseHour(h.time ?:
""),
                    temperature = h.tempC ?: 0f,
                    weatherCode = mapWeatherApiCodeToWmo(h.condition?.code),
                    precipChance = h.chanceOfRain ?: 0
                )
            )
        }

        val dailyList = mutableListOf<DayForecast>()
        forecastDays.forEachIndexed { i, fd ->
            dailyList.add(
                DayForecast(
                    day = parseDay(fd.date ?: "", i),
                    tempMin = fd.day?.minTempC ?: 20f,
                    tempMax = fd.day?.maxTempC ?: 30f,
                    weatherCode =
mapWeatherApiCodeToWmo(fd.day?.condition?.code),
                    uvIndex = fd.day?.uv ?: 0f,
                    precipChance = fd.day?.chanceOfRain ?: 0
                )
            )
        }

        if (omData != null) {
            val omDaily = omData.daily
            val omTimes = omDaily?.time ?: emptyList()
            val lastWaDate = forecastDays.lastOrNull()?.date ?: ""
            val startOmIdx = omTimes.indexOfFirst { it > lastWaDate }.takeIf
{ it >= 0 } ?: omTimes.size
            for (i in startOmIdx until omTimes.size) {
                val idx = dailyList.size
                dailyList.add(
                    DayForecast(
```

```
                        day = parseDay(omTimes[i], idx),
                        tempMin = omDaily?.temperature2mMin?.getOrElse(i)
{ 20f } ?: 20f,
                        tempMax = omDaily?.temperature2mMax?.getOrElse(i)
{ 30f } ?: 30f,
                        weatherCode = omDaily?.weatherCode?.getOrElse(i) { 0
} ?: 0,
                        uvIndex = omDaily?.uvIndexMax?.getOrElse(i)
{ 0f } ?: 0f,
                        precipChance =
omDaily?.precipProbabilityMax?.getOrElse(i) { 0 } ?: 0
                    )
                )
            }
        }

        val firstAlert = waData.alerts?.alert?.firstOrNull()

        return currentState.copy(
            // Honour the city the user actually chose; WeatherAPI's
location.name
            // can resolve to a nearby district (e.g. London → "Strand").
            selectedCity = cityName.ifBlank { waData.location?.name ?: "" },
            temperature = tempVal,
            feelsLike = feelsLikeVal,
            humidity = humidityVal,
            pressure = pressureVal,
            windSpeed = windSpeedVal,
            windDir = windDirVal,
            visibility = visibilityVal,
            weatherCode = wmoCode,
            aqi = aqiVal,
            uvIndex = uvVal,
            sunrise = parsedSunrise,
            sunset = parsedSunset,
            sunriseMinutes = sunriseMinutes,
            sunsetMinutes = sunsetMinutes,
            hourlyForecast = hourlyList,
            dailyForecast = dailyList,
            isLoading = false,
            errorMessage = null,
            alertEvent = firstAlert?.event,
            alertHeadline = firstAlert?.headline,
            alertDescription = firstAlert?.desc,
            lastRefreshedTime = SimpleDateFormat("h:mm a",
Locale.getDefault()).format(Date())
        )
    }

    private fun mapOpenMeteoToState(
        currentState: WeatherState,
        data: OpenMeteoResponse,
        cityName: String
    ): WeatherState {
        val current = data.current ?: throw Exception("Current block missing
in Open-Meteo response")

        val hourlyList = mutableListOf<HourForecast>()
        val hourlyData = data.hourly
        var currentVisibilityKm = 10f
        if (hourlyData != null) {
            val timeList = hourlyData.time ?: emptyList()
            if (timeList.isNotEmpty()) {
                val nowHourStr = SimpleDateFormat("yyyy-MM-dd'T'HH",
Locale.getDefault()).format(Date())
                var startIdx = timeList.indexOfFirst
{ it.startsWith(nowHourStr) }
                if (startIdx == -1) startIdx = 0
```

```
                hourlyData.visibility?.getOrNull(startIdx)?.let
{ currentVisibilityKm = it / 1000f }
                for (i in startIdx until minOf(startIdx +
WeatherApiConfig.FORECAST_HOURS_LIMIT, timeList.size)) {
                    hourlyList.add(
                        HourForecast(
                            hour = if (i == startIdx) "Now" else
parseHour(timeList[i].replace('T', ' ')),
                            temperature =
hourlyData.temperature2m?.getOrElse(i) { 20f } ?: 20f,
                            weatherCode =
hourlyData.weatherCode?.getOrElse(i) { 0 } ?: 0,
                            precipChance =
hourlyData.precipitationProbability?.getOrElse(i) { 0 } ?: 0
                        )
                    )
                }
            }
        }

        val dailyList = mutableListOf<DayForecast>()
        var parsedSunrise = TIME_DEFAULT_SUNRISE
        var parsedSunset = TIME_DEFAULT_SUNSET
        var parsedSunriseMinutes = 360
        var parsedSunsetMinutes = 1080
        val dailyData = data.daily
        if (dailyData != null) {
            val dayTimes = dailyData.time ?: emptyList()
            if (dayTimes.isNotEmpty()) {
                parsedSunrise =
parseIsoTime(dailyData.sunrise?.firstOrNull() ?: "")
                parsedSunset =
parseIsoTime(dailyData.sunset?.firstOrNull() ?: "")
                parsedSunriseMinutes =
parseFormattedTimeToMins(parsedSunrise)
                parsedSunsetMinutes = parseFormattedTimeToMins(parsedSunset)
                for (i in dayTimes.indices) {
                    dailyList.add(
                        DayForecast(
                            day = parseDay(dayTimes[i], i),
                            tempMin =
dailyData.temperature2mMin?.getOrElse(i) { 20f } ?: 20f,
                            tempMax =
dailyData.temperature2mMax?.getOrElse(i) { 30f } ?: 30f,
                            weatherCode =
dailyData.weatherCode?.getOrElse(i) { 0 } ?: 0,
                            uvIndex = dailyData.uvIndexMax?.getOrElse(i)
{ 0f } ?: 0f,
                            precipChance =
dailyData.precipProbabilityMax?.getOrElse(i) { 0 } ?: 0
                        )
                    )
                }
            }
        }

        return currentState.copy(
            selectedCity = cityName,
            temperature = current.temperature2m ?: 0f,
            feelsLike = current.apparentTemperature ?: current.temperature2m
?: 0f,
            humidity = current.relativeHumidity2m ?: 0f,
            pressure = current.surfacePressure ?: 1013f,
            windSpeed = current.windSpeed10m ?: 0f,
            windDir = current.windDirection10m?.let { degreesToCompass(it) }
?: "",
            visibility = currentVisibilityKm,
            uvIndex = current.uv_index ?: 0f,
```

```
            weatherCode = current.weatherCode ?: 0,
            sunrise = parsedSunrise,
            sunset = parsedSunset,
            sunriseMinutes = parsedSunriseMinutes,
            sunsetMinutes = parsedSunsetMinutes,
            hourlyForecast = hourlyList,
            dailyForecast = dailyList,
            isLoading = false,
            errorMessage = null,
            lastRefreshedTime = SimpleDateFormat("h:mm a",
Locale.getDefault()).format(Date())
        )
    }

    // ── Formatting / parsing helpers ──────────────────────────

    private fun parseHour(raw: String): String {
        if (raw.isBlank()) return ""
        return runCatching {
            val hour = raw.replace('T', ' ').substringAfter(' ').trim()
                .substringBefore(':').toInt()
            val cal = java.util.Calendar.getInstance().apply {
                set(java.util.Calendar.HOUR_OF_DAY, hour)
                set(java.util.Calendar.MINUTE, 0)
            }
            SimpleDateFormat("h a", Locale.getDefault()).format(cal.time)
        }.getOrDefault(raw)
    }

    private fun parseDay(dateStr: String, index: Int): String {
        if (index == 0) return "Today"
        if (index == 1) return "Tomorrow"
        return runCatching {
            val d = SimpleDateFormat("yyyy-MM-dd",
Locale.getDefault()).parse(dateStr)
            SimpleDateFormat("EEE", Locale.getDefault()).format(d!!)
        }.getOrDefault(dateStr)
    }

    private fun parseIsoTime(iso: String): String {
        if (iso.isBlank()) return TIME_DEFAULT_SUNRISE
        return runCatching {
            val d = SimpleDateFormat("yyyy-MM-dd'T'HH:mm",
Locale.getDefault()).parse(iso)
            SimpleDateFormat("h:mm a", Locale.getDefault()).format(d!!)
        }.getOrDefault(iso)
    }

    private fun parseFormattedTimeToMins(formatted: String): Int =
runCatching {
        val d = SimpleDateFormat("h:mm a",
Locale.getDefault()).parse(formatted.trim())
        val cal = java.util.Calendar.getInstance().apply { time = d!! }
        cal.get(java.util.Calendar.HOUR_OF_DAY) * 60 +
cal.get(java.util.Calendar.MINUTE)
    }.getOrDefault(WeatherApiConfig.Defaults.DEFAULT_SUNRISE_MINUTES)

    private fun degreesToCompass(deg: Float): String {
        val dirs = arrayOf(
            "N", "NNE", "NE", "ENE", "E", "ESE", "SE", "SSE",
            "S", "SSW", "SW", "WSW", "W", "WNW", "NW", "NNW"
        )
        var idx = (((deg % 360f) / 22.5f) + 0.5f).toInt() % 16
        if (idx < 0) idx += 16
        return dirs[idx]
    }
}
```

**File: data/models/NewsModels.kt**

```
package com.example.aiweathermonitor.data.models

import kotlinx.serialization.Serializable

/**
 * A single news headline parsed from an RSS 2.0 feed `<item>` element.
 *
 * Side-effect free and serializable so it can be cached alongside the rest
of
 * the app state if ever needed. [link] is opened in the browser when
tapped.
 */
@Serializable
data class NewsArticle(
    val title: String = "",
    val link: String = "",
    val source: String = "",          // Publisher / feed source name
    val pubDate: String = "",         // Raw RFC-822 date string from the
feed
    val description: String = ""      // Plain-text summary (HTML stripped)
)
```

**File: data/models/OpenMeteoModels.kt**

```
package com.example.aiweathermonitor.data.models

import kotlinx.serialization.Serializable
import kotlinx.serialization.SerialName

/**
 * Open-Meteo Weather API response models
 * Separated for better organization and maintainability
 */

@Serializable
data class OpenMeteoCurrentWeather(
    val temperature: Float? = null,
    @SerialName("windspeed")
    val windSpeed: Float? = null,
    @SerialName("weathercode")
    val weatherCode: Int? = null,
    @SerialName("is_day")
    val isDay: Int? = null,
    val time: String? = null
)

@Serializable
data class OpenMeteoHourly(
    val time: List<String>? = null,
    @SerialName("temperature_2m")
    val temperature2m: List<Float>? = null,
    @SerialName("weather_code")
    val weatherCode: List<Int>? = null,
    @SerialName("relative_humidity_2m")
    val relativeHumidity2m: List<Int>? = null,
    @SerialName("apparent_temperature")
    val apparentTemperature: List<Float>? = null,
    @SerialName("weather_description")
    val weatherDescription: List<String>? = null,
    @SerialName("precipitation_probability")
    val precipitationProbability: List<Int>? = null,
    val visibility: List<Float>? = null
)

@Serializable
data class OpenMeteoDaily(
    val time: List<String>? = null,
    @SerialName("weather_code")
    val weatherCode: List<Int>? = null,
    @SerialName("temperature_2m_max")
    val temperature2mMax: List<Float>? = null,
    @SerialName("temperature_2m_min")
    val temperature2mMin: List<Float>? = null,
    val sunrise: List<String>? = null,
    val sunset: List<String>? = null,
    @SerialName("weather_description")
    val weatherDescription: List<String>? = null,
    @SerialName("uv_index_max")
    val uvIndexMax: List<Float>? = null,
    @SerialName("precipitation_probability_max")
    val precipProbabilityMax: List<Int>? = null
)

@Serializable
data class OpenMeteoCurrent(
    val time: String? = null,
    val interval: Int? = null,
    @SerialName("temperature_2m")
    val temperature2m: Float? = null,
    @SerialName("relative_humidity_2m")
```

```
    val relativeHumidity2m: Float? = null,
    @SerialName("apparent_temperature")
    val apparentTemperature: Float? = null,
    @SerialName("surface_pressure")
    val surfacePressure: Float? = null,
    @SerialName("wind_speed_10m")
    val windSpeed10m: Float? = null,
    @SerialName("wind_direction_10m")
    val windDirection10m: Float? = null,
    @SerialName("weather_code")
    val weatherCode: Int? = null,
    @SerialName("uv_index")
    val uv_index: Float? = null
)

@Serializable
data class OpenMeteoForecastResponse(
    val latitude: Double? = null,
    val longitude: Double? = null,
    @SerialName("generationtime_ms")
    val generationTimeMs: Double? = null,
    @SerialName("utc_offset_seconds")
    val utcOffsetSeconds: Int? = null,
    val timezone: String? = null,
    @SerialName("timezone_abbreviation")
    val timezoneAbbreviation: String? = null,
    val elevation: Float? = null,
    @SerialName("current_weather")
    val currentWeather: OpenMeteoCurrentWeather? = null,
    val current: OpenMeteoCurrent? = null,
    val hourly: OpenMeteoHourly? = null,
    val daily: OpenMeteoDaily? = null
)

@Serializable
data class OpenMeteoGeocodingResult(
    val id: Int? = null,
    val name: String? = null,
    val latitude: Double? = null,
    val longitude: Double? = null,
    val elevation: Float? = null,
    @SerialName("feature_code")
    val featureCode: String? = null,
    @SerialName("country_code")
    val countryCode: String? = null,
    val country: String? = null,
    val admin1: String? = null,
    val admin2: String? = null,
    val timezone: String? = null,
    val population: Int? = null,
    val postcodes: List<String>? = null
)

@Serializable
data class OpenMeteoGeocodingResponse(
    val results: List<OpenMeteoGeocodingResult>? = null,
    @SerialName("generationtime_ms")
    val generationTimeMs: Double? = null
)

/**
 * The rest of the codebase refers to the forecast payload simply as
 * [OpenMeteoResponse]; keep that name as an alias of the concrete model.
 */
typealias OpenMeteoResponse = OpenMeteoForecastResponse
```

**File: data/models/PollenModels.kt**

```
package com.example.aiweathermonitor.data.models

import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
data class PollenResponse(
    val latitude: Double? = null,
    val longitude: Double? = null,
    val hourly: PollenHourly? = null
)

@Serializable
data class PollenHourly(
    val time: List<String>? = null,
    @SerialName("alder_pollen")    val alderPollen:    List<Float?>? = null,
    @SerialName("birch_pollen")    val birchPollen:    List<Float?>? = null,
    @SerialName("grass_pollen")    val grassPollen:    List<Float?>? = null,
    @SerialName("mugwort_pollen")  val mugwortPollen:  List<Float?>? = null,
    @SerialName("olive_pollen")    val olivePollen:    List<Float?>? = null,
    @SerialName("ragweed_pollen")  val ragweedPollen:  List<Float?>? = null
)

// ── Domain model ─────────────────────────────────────────────────────


data class PollenData(
    val alder: Float = 0f,
    val birch: Float = 0f,
    val grass: Float = 0f,
    val mugwort: Float = 0f,
    val olive: Float = 0f,
    val ragweed: Float = 0f
) {
    /** Highest single pollen count across all types */
    val peak: Float get() = maxOf(alder, birch, grass, mugwort, olive,
ragweed)

    /** Human-readable risk level based on peak count (grains/m³) */
    val riskLevel: String get() = when {
        peak < 10f   -> "Low"
        peak < 30f   -> "Moderate"
        peak < 80f   -> "High"
        else         -> "Very High"
    }

    /** List of types with meaningful concentration (>5 grains/m³) */
    val activeTypes: List<String> get() = buildList {
        if (grass   > 5f) add("Grass")
        if (birch   > 5f) add("Birch")
        if (ragweed > 5f) add("Ragweed")
        if (alder   > 5f) add("Alder")
        if (mugwort > 5f) add("Mugwort")
        if (olive   > 5f) add("Olive")
    }
}
```

**File: data/models/WeatherApiModels.kt**

```
package com.example.aiweathermonitor.data.models

import kotlinx.serialization.Serializable
import kotlinx.serialization.SerialName

/**
 * WeatherAPI.com response models
 * Endpoint: https://api.weatherapi.com/v1/forecast.json
 * Docs: https://www.weatherapi.com/docs/
 */

@Serializable
data class WeatherApiCondition(
    val text: String? = null,
    val icon: String? = null,
    val code: Int? = null
)

@Serializable
data class WeatherApiCurrent(
    @SerialName("temp_c") val tempC: Float? = null,
    @SerialName("temp_f") val tempF: Float? = null,
    @SerialName("feelslike_c") val feelsLikeC: Float? = null,
    @SerialName("is_day") val isDay: Int? = null,
    val humidity: Int? = null,
    @SerialName("wind_kph") val windKph: Float? = null,
    @SerialName("wind_mph") val windMph: Float? = null,
    @SerialName("wind_dir") val windDir: String? = null,
    @SerialName("pressure_mb") val pressureMb: Float? = null,
    @SerialName("vis_km") val visKm: Float? = null,
    val uv: Float? = null,
    @SerialName("air_quality") val airQuality: WeatherApiAirQuality? = null,
    val condition: WeatherApiCondition? = null
)

@Serializable
data class WeatherApiAirQuality(
    @SerialName("pm2_5") val pm25: Float? = null,
    @SerialName("pm10") val pm10: Float? = null,
    @SerialName("us-epa-index") val usEpaIndex: Int? = null,
    @SerialName("gb-defra-index") val gbDefraIndex: Int? = null
)

@Serializable
data class WeatherApiAstro(
    val sunrise: String? = null,
    val sunset: String? = null,
    @SerialName("moon_phase") val moonPhase: String? = null
)

@Serializable
data class WeatherApiDayDetail(
    @SerialName("maxtemp_c") val maxTempC: Float? = null,
    @SerialName("mintemp_c") val minTempC: Float? = null,
    @SerialName("maxtemp_f") val maxTempF: Float? = null,
    @SerialName("mintemp_f") val minTempF: Float? = null,
    @SerialName("avghumidity") val avgHumidity: Float? = null,
    @SerialName("daily_chance_of_rain") val chanceOfRain: Int? = null,
    val uv: Float? = null,
    val condition: WeatherApiCondition? = null
)

@Serializable
data class WeatherApiHour(
    val time: String? = null,
    @SerialName("temp_c") val tempC: Float? = null,
```

```
    @SerialName("temp_f") val tempF: Float? = null,
    val humidity: Int? = null,
    @SerialName("chance_of_rain") val chanceOfRain: Int? = null,
    val condition: WeatherApiCondition? = null,
    @SerialName("is_day") val isDay: Int? = null
)

@Serializable
data class WeatherApiForecastDay(
    val date: String? = null,
    val day: WeatherApiDayDetail? = null,
    val astro: WeatherApiAstro? = null,
    val hour: List<WeatherApiHour>? = null
)

@Serializable
data class WeatherApiForecast(
    val forecastday: List<WeatherApiForecastDay>? = null
)

@Serializable
data class WeatherApiLocation(
    val name: String? = null,
    val region: String? = null,
    val country: String? = null,
    val lat: Double? = null,
    val lon: Double? = null,
    val localtime: String? = null
)

@Serializable
data class WeatherApiAlert(
    val headline: String? = null,
    val msgtype: String? = null,
    val severity: String? = null,
    val urgency: String? = null,
    val event: String? = null,
    val desc: String? = null,
    val instruction: String? = null
)

@Serializable
data class WeatherApiAlerts(
    val alert: List<WeatherApiAlert>? = null
)

@Serializable
data class WeatherApiForecastResponse(
    val location: WeatherApiLocation? = null,
    val current: WeatherApiCurrent? = null,
    val forecast: WeatherApiForecast? = null,
    val alerts: WeatherApiAlerts? = null
)

/**
 * Maps a WeatherAPI.com condition code to a WMO weather code
 * used throughout the app for icon/description display.
 * Reference: https://www.weatherapi.com/docs/weather_conditions.json
 */
fun mapWeatherApiCodeToWmo(code: Int?): Int = when (code) {
    1000 -> 0   // Sunny / Clear
    1003 -> 2   // Partly cloudy
    1006 -> 3   // Cloudy
    1009 -> 3   // Overcast
    1030 -> 45  // Mist
    1063, 1180, 1183 -> 61   // Patchy / light rain
    1066, 1210, 1213 -> 71   // Patchy / light snow
    1069, 1204, 1207 -> 73   // Sleet
```

```
    1072, 1168, 1171 -> 51   // Drizzle / freezing drizzle
    1087 -> 95  // Thundery outbreaks
    1114, 1117 -> 75  // Blowing / blizzard
    1135, 1147 -> 45  // Fog
    1150, 1153 -> 51  // Light drizzle
    1189, 1192 -> 63  // Moderate / heavy rain
    1195 -> 65  // Heavy rain
    1198, 1201 -> 61  // Light / heavy freezing rain
    1216, 1219 -> 73  // Moderate / heavy snow
    1222, 1225 -> 75  // Heavy snow
    1237 -> 75  // Ice pellets
    1240 -> 80  // Light rain shower
    1243, 1246 -> 82  // Moderate / heavy rain shower
    1249, 1252 -> 81  // Sleet showers
    1255, 1258 -> 81  // Snow showers
    1261, 1264 -> 82  // Ice pellet showers
    1273, 1276 -> 95  // Thunderstorm with rain
    1279, 1282 -> 95  // Thunderstorm with snow
    else -> 0
}

// ── Search endpoint models ──────────────────────────────────

@Serializable
data class WeatherApiSearchResult(
    val id: Int? = null,
    val name: String? = null,
    val region: String? = null,
    val country: String? = null,
    val lat: Double? = null,
    val lon: Double? = null,
    val url: String? = null
)
```

**File: data/persistence/WeatherDataStore.kt**

```
package com.example.aiweathermonitor.data.persistence

import android.content.Context
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import com.example.aiweathermonitor.WeatherState
import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.util.AppLogger
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.flow.map
import kotlinx.serialization.json.Json

/**
 * DataStore instance for weather app preferences.
 * Replaces deprecated SharedPreferences with a more modern approach.
 */
private val Context.weatherDataStore by preferencesDataStore(
    name = "weather_prefs"
)

/**
 * Utility for managing weather data persistence using DataStore.
 * Thread-safe and supports Flow-based reactive updates.
 *
 * DataStore provides:
 * - Type-safe key-value storage
 * - ACID transactions
 * - Coroutine-based API
 * - Migration from SharedPreferences
 *
 * @param context Application context
 * @param json JSON serializer for complex objects
 */
class WeatherDataStore(
    private val context: Context,
    private val json: Json
) {
    private val TAG = "WeatherDataStore"

    private val WEATHER_STATE_KEY = stringPreferencesKey(
        WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY
    )
    private val LAST_UPDATED_KEY = stringPreferencesKey(
        WeatherApiConfig.SharedPrefsKeys.LAST_REFRESH_KEY
    )
    private val API_KEY_KEY = stringPreferencesKey(
        WeatherApiConfig.SharedPrefsKeys.API_KEY_KEY
    )

    /**
     * Observes cached weather state as a Flow.
     * Emits current value immediately, then any updates.
     */
    val cachedWeatherState: Flow<WeatherState?> =
context.weatherDataStore.data
        .map { preferences ->
            try {
                val jsonStr = preferences[WEATHER_STATE_KEY]
                if (jsonStr != null) {
                    json.decodeFromString(WeatherState.serializer(),
jsonStr)
                } else {
                    null
                }
```

```
            } catch (e: Exception) {
                AppLogger.error("Failed to decode cached weather state",
TAG, e)
                null
            }
        }

    /**
     * Saves weather state to cache.
     *
     * @param state Weather state to cache
     * @return Result indicating success or failure
     */
    suspend fun cacheWeatherState(state: WeatherState): Result<Unit> = try {
        val jsonStr = json.encodeToString(WeatherState.serializer(), state)
        context.weatherDataStore.edit { preferences ->
            preferences[WEATHER_STATE_KEY] = jsonStr
            preferences[LAST_UPDATED_KEY] =
System.currentTimeMillis().toString()
        }
        AppLogger.info("Weather state cached successfully", TAG)
        Result.success(Unit)
    } catch (e: Exception) {
        AppLogger.error("Failed to cache weather state", TAG, e)
        Result.failure(e)
    }

    /**
     * Retrieves cached weather state.
     *
     * @return Result containing cached state or null if not cached
     */
    suspend fun getCachedWeatherState(): Result<WeatherState?> = try {
        val state = context.weatherDataStore.data
            .map { prefs ->
                val jsonStr = prefs[WEATHER_STATE_KEY] ?: return@map null
                json.decodeFromString(WeatherState.serializer(), jsonStr)
            }
            .first()
        Result.success(state)
    } catch (e: Exception) {
        AppLogger.error("Failed to retrieve cached weather state", TAG, e)
        Result.failure(e)
    }

    /**
     * Clears all cached weather data.
     *
     * @return Result indicating success or failure
     */
    suspend fun clearCache(): Result<Unit> = try {
        context.weatherDataStore.edit { preferences ->
            preferences.clear()
        }
        AppLogger.info("Weather cache cleared", TAG)
        Result.success(Unit)
    } catch (e: Exception) {
        AppLogger.error("Failed to clear cache", TAG, e)
        Result.failure(e)
    }

    /**
     * Checks if cached data is still valid.
     * Cache is valid if it exists and is less than CACHE_EXPIRATION_MS old.
     *
     * @return true if cache exists and is fresh, false otherwise
     */
    suspend fun isCacheValid(): Boolean = try {
```

```
        val prefs = context.weatherDataStore.data.first()
        val lastUpdated = prefs[LAST_UPDATED_KEY]?.toLongOrNull() ?: return
false
        val ageMs = System.currentTimeMillis() - lastUpdated
        ageMs < WeatherApiConfig.CACHE_EXPIRATION_MS &&
prefs[WEATHER_STATE_KEY] != null
    } catch (e: Exception) {
        AppLogger.error("Failed to check cache validity", TAG, e)
        false
    }
}
```

**File: exception/WeatherException.kt**

```
package com.example.aiweathermonitor.exception

/**
 * Base exception class for all weather-related errors.
 * Enables specific exception handling and logging.
 */
sealed class WeatherException(
    message: String,
    cause: Throwable? = null
) : Exception(message, cause)

/**
 * Exception thrown when network operations fail.
 * Examples: No internet, DNS resolution failure, timeout
 */
class NetworkException(
    message: String = "Network operation failed",
    cause: Throwable? = null
) : WeatherException(message, cause)

/**
 * Exception thrown for HTTP-level errors (4xx, 5xx).
 */
class HttpException(
    val code: Int,
    val statusMessage: String,
    cause: Throwable? = null
) : WeatherException("HTTP $code: $statusMessage", cause)

/**
 * Exception thrown when API response data cannot be parsed.
 * Examples: Invalid JSON, missing required fields
 */
class DataParsingException(
    message: String = "Failed to parse response data",
    cause: Throwable? = null
) : WeatherException(message, cause)

/**
 * Exception thrown when search operations fail.
 */
class SearchException(
    message: String = "Search operation failed",
    cause: Throwable? = null
) : WeatherException(message, cause)

/**
 * Exception thrown when location operations fail.
 * Examples: Permission denied, location provider unavailable
 */
class LocationException(
    message: String = "Location operation failed",
    cause: Throwable? = null
) : WeatherException(message, cause)

/**
 * Exception thrown when API configuration is invalid.
 * Examples: Missing API key, invalid configuration
 */
class ConfigurationException(
    message: String = "Invalid configuration",
    cause: Throwable? = null
) : WeatherException(message, cause)

/**
 * Extension function to convert generic exceptions to weather exceptions.
```

```
 */
fun Throwable.toWeatherException(): WeatherException = when (this) {
    is WeatherException -> this
    is java.net.UnknownHostException -> NetworkException("No internet
connection", this)
    is java.net.SocketTimeoutException -> NetworkException("Request
timeout", this)
    is java.io.IOException -> NetworkException("Network error", this)
    else -> NetworkException(this.message ?: "Unknown error", this)
}
```

**File: theme/Color.kt**

```
package com.example.aiweathermonitor.theme

// Color tokens intentionally empty — the app uses the plain Material 3
color
// scheme (see Theme.kt). Add semantic colors here as the new design takes
shape.
```

**File: theme/Dimens.kt**

```
package com.example.aiweathermonitor.theme

import androidx.compose.ui.unit.dp

// ── SOFT DAYLIGHT SPACING & SHAPE SYSTEM
──────────────────────────────────────
// One source of truth for rhythm and roundness so every surface feels
related.

object Spacing {
    val screenHorizontal = 16.dp   // left/right page margin
    val sectionGap = 16.dp         // vertical gap between cards/sections
    val cardPaddingH = 18.dp       // inner card padding (horizontal)
    val cardPaddingV = 16.dp       // inner card padding (vertical)
    val itemGap = 12.dp            // gap between side-by-side tiles /
inline rows
    val small = 8.dp
    val tiny = 4.dp
}

object Radii {
    val card = 24.dp   // primary cards
    val pill = 20.dp   // buttons / tab row / search field
    val chip = 14.dp   // small chips / list rows
    val bar = 8.dp     // progress bars / tracks
}

object Elevation {
    val card = 2.dp        // resting soft lift
    val raised = 6.dp      // dropdowns / dialogs (floating above content)
}
```

**File: theme/Theme.kt**

```
package com.example.aiweathermonitor.theme

import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color

// ── SOFT DAYLIGHT
─────────────────────────────────────────────────────────
// A calm, soft, minimal personal-weather palette.
//   · warm off-white canvas, pure-white cards
//   · one soft cornflower-blue accent
//   · muted slate-gray for secondary text
//   · surfaceTint matched to surface so cards stay clean white (shadow
gives depth)

private val LightScheme = lightColorScheme(
    primary              = Color(0xFF5B7FE0),   // soft cornflower blue —
the one accent
    onPrimary            = Color(0xFFFFFFFF),
    primaryContainer     = Color(0xFFDEE7FF),   // pale blue tint
    onPrimaryContainer   = Color(0xFF0E2A66),
    secondary            = Color(0xFF6E7390),   // muted slate
    onSecondary          = Color(0xFFFFFFFF),
    secondaryContainer   = Color(0xFFE6E8F2),
    onSecondaryContainer = Color(0xFF2A2E45),
    tertiary             = Color(0xFF4FA899),   // soft teal — calm/clear
accent
    onTertiary           = Color(0xFFFFFFFF),
    tertiaryContainer    = Color(0xFFD3F0E9),
    onTertiaryContainer  = Color(0xFF0A2F29),
    background           = Color(0xFFF5F6FA),   // soft off-white canvas
    onBackground         = Color(0xFF1A1B22),   // soft near-black ink
    surface              = Color(0xFFFFFFFF),   // pure white — cards
    onSurface            = Color(0xFF1A1B22),
    surfaceVariant       = Color(0xFFECEEF5),   // soft gray — chips /
tracks
    onSurfaceVariant     = Color(0xFF6E7390),   // muted secondary text
    surfaceTint          = Color(0xFFFFFFFF),   // keep elevated cards pure
white
    outline              = Color(0xFFD7DAE5),   // whisper-soft separator
    outlineVariant       = Color(0xFFE8EAF1),
    error                = Color(0xFFDE5A57),   // soft red
    onError              = Color(0xFFFFFFFF),
    errorContainer       = Color(0xFFFFE5E3),
    onErrorContainer     = Color(0xFF5A1512),
    inverseSurface       = Color(0xFF2E2F38),
    inverseOnSurface     = Color(0xFFF2F3F8),
    inversePrimary       = Color(0xFFAFC3FF)
)

private val DarkScheme = darkColorScheme(
    primary              = Color(0xFFD97757),   // Claude coral / clay — the
one accent
    onPrimary            = Color(0xFF3A1505),
    primaryContainer     = Color(0xFF5C2E1A),
    onPrimaryContainer   = Color(0xFFFFDBCB),
    secondary            = Color(0xFFB7B2A7),   // warm taupe-gray
    onSecondary          = Color(0xFF2C2823),
    secondaryContainer   = Color(0xFF3B3731),
    onSecondaryContainer = Color(0xFFEBE7DD),
    tertiary             = Color(0xFFD9A77C),   // warm sand
    onTertiary           = Color(0xFF3C2410),
    tertiaryContainer    = Color(0xFF553820),
```

```
    onTertiaryContainer  = Color(0xFFFBE0C6),
    background           = Color(0xFF1A1916),   // warm near-black canvas
    onBackground         = Color(0xFFF0EEE6),   // cream ink
    surface              = Color(0xFF242320),   // lifted warm card
    onSurface            = Color(0xFFF0EEE6),
    surfaceVariant       = Color(0xFF302E2A),   // chips / tracks
    onSurfaceVariant     = Color(0xFFB7B2A7),   // muted warm gray
    surfaceTint          = Color(0xFF242320),   // keep cards warm, not
blue-tinted
    outline              = Color(0xFF4A4740),   // soft separator
    outlineVariant       = Color(0xFF35332E),
    error                = Color(0xFFFFB4AB),
    onError              = Color(0xFF690005),
    errorContainer       = Color(0xFF93000A),
    onErrorContainer     = Color(0xFFFFDAD6),
    inverseSurface       = Color(0xFFF0EEE6),
    inverseOnSurface     = Color(0xFF2E2D2A),
    inversePrimary       = Color(0xFFC15F3C)
)

@Composable
fun AIWeatherMonitorTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) DarkScheme else LightScheme
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

**File: theme/Type.kt**

```
package com.example.aiweathermonitor.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

// ── SOFT DAYLIGHT TYPE SCALE ──────────────────────────────────────
// Light, airy display weights for the hero; calm, legible body. System sans
so it
// feels native and soft. Roles not listed here fall back to the M3
defaults.

private val Sans = FontFamily.SansSerif

val Typography = Typography(
    // Hero temperature — large but soft (W200, not fragile ultra-thin)
    displayLarge = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W200,
        fontSize = 82.sp, lineHeight = 86.sp, letterSpacing = (-2).sp
    ),
    // City name
    displaySmall = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W400,
        fontSize = 30.sp, lineHeight = 36.sp, letterSpacing = (-0.4).sp
    ),
    // Condition line / prominent secondary text
    titleMedium = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W400,
        fontSize = 18.sp, lineHeight = 24.sp, letterSpacing = 0.sp
    ),
    // Metric values
    titleLarge = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W500,
        fontSize = 26.sp, lineHeight = 32.sp, letterSpacing = 0.sp
    ),
    // Card section eyebrow (UPPERCASE)
    titleSmall = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W700,
        fontSize = 11.sp, lineHeight = 14.sp, letterSpacing = 1.sp
    ),
    // Body
    bodyLarge = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W400,
        fontSize = 16.sp, lineHeight = 24.sp, letterSpacing = 0.2.sp
    ),
    // Body (secondary)
    bodyMedium = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W400,
        fontSize = 14.sp, lineHeight = 20.sp, letterSpacing = 0.2.sp
    ),
    // Buttons / chips
    labelLarge = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W500,
        fontSize = 13.sp, lineHeight = 18.sp, letterSpacing = 0.4.sp
    ),
    // Captions / fine print
    labelSmall = TextStyle(
        fontFamily = Sans, fontWeight = FontWeight.W500,
        fontSize = 11.sp, lineHeight = 14.sp, letterSpacing = 0.4.sp
    )
)
```

**File: ui/DashboardScreen.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.*
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.slideInVertically
import androidx.compose.animation.slideOutVertically
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.foundation.Canvas
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.drawscope.Stroke
import kotlin.math.sin
import kotlin.math.cos
import androidx.compose.ui.platform.LocalContext
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.FlowRow
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.gestures.detectHorizontalDragGestures
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Place
import androidx.compose.material.icons.filled.Refresh
import androidx.compose.material.icons.filled.Search
import androidx.compose.material.icons.filled.Settings
import androidx.compose.material.icons.automirrored.filled.List
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Star
import androidx.compose.material.icons.filled.KeyboardArrowDown
import androidx.compose.material.icons.filled.KeyboardArrowUp
import androidx.compose.material.icons.filled.Share
import androidx.compose.material.icons.outlined.WbSunny
import androidx.compose.material.icons.outlined.WbTwilight
import androidx.compose.foundation.layout.offset
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.nativeCanvas
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.unit.sp
import com.example.aiweathermonitor.getWeatherCodeDescription
import com.example.aiweathermonitor.DayForecast
import com.example.aiweathermonitor.GeocodingResult
import com.example.aiweathermonitor.HourForecast
import com.example.aiweathermonitor.WeatherState
import com.example.aiweathermonitor.data.models.NewsArticle
import com.example.aiweathermonitor.getAqiDescription
import com.example.aiweathermonitor.getUvDescription
import com.example.aiweathermonitor.ui.AppSettings
import com.example.aiweathermonitor.ui.WindUnit
import com.example.aiweathermonitor.ui.PressureUnit
import com.example.aiweathermonitor.ui.VisibilityUnit
import com.example.aiweathermonitor.ui.LiquidGlassTextField
import com.example.aiweathermonitor.util.formatTemp
import com.example.aiweathermonitor.util.formatWind
import com.example.aiweathermonitor.util.formatPressure
import com.example.aiweathermonitor.util.formatVisibility
import com.example.aiweathermonitor.theme.Spacing
import kotlin.math.roundToInt

import androidx.compose.ui.graphics.graphicsLayer

data class DashboardActions(
    val onQueryChange: (String) -> Unit,
    val onSelectCity: (GeocodingResult) -> Unit,
    val onRefresh: () -> Unit,
    val onLocationClick: () -> Unit,
    val onSettingsClick: () -> Unit,
    val onNavigateToSettings: () -> Unit = {},
    val onSaveWeatherApiKey: (String) -> Unit,
    val onDismissSettings: () -> Unit,
    val onSaveCity: () -> Unit,
    val onRemoveCity: (GeocodingResult) -> Unit,
    val onToggleCelsius: () -> Unit,
    val onToggleSavedCitiesScreen: (Boolean) -> Unit,
    val onShareWeather: () -> Unit = {},
    val onOpenArticle: (String) -> Unit = {},
    val onRefreshNews: () -> Unit = {}
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DashboardScreen(
    state: WeatherState,
    settings: AppSettings,
    actions: DashboardActions,
    modifier: Modifier = Modifier
) {
    val scrollState = rememberScrollState()

    val isDarkTheme = androidx.compose.foundation.isSystemInDarkTheme()
    val mainTextColor = MaterialTheme.colorScheme.onBackground
    val subTextColor = MaterialTheme.colorScheme.onSurfaceVariant

    // Day vs night for icon selection — current time within
sunrise..sunset.
    val nowMins = run {
        val cal = java.util.Calendar.getInstance()
        cal.get(java.util.Calendar.HOUR_OF_DAY) * 60 +
cal.get(java.util.Calendar.MINUTE)
    }
    val isDay = nowMins in state.sunriseMinutes..state.sunsetMinutes

    CompositionLocalProvider(LocalWeatherCode provides state.weatherCode) {
        Box(
        modifier = modifier
            .fillMaxSize()
```

```
            .background(MaterialTheme.colorScheme.background)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .statusBarsPadding()
                .navigationBarsPadding()
                .padding(horizontal = Spacing.screenHorizontal)
                .verticalScroll(scrollState),
            verticalArrangement = Arrangement.spacedBy(Spacing.sectionGap)
        ) {
            Spacer(modifier = Modifier.height(Spacing.small))

            // 1. Offline Banner — liquid glass
            LiquidGlassOfflineBanner(isOnline = state.isOnline,
lastRefreshedTime = state.lastRefreshedTime, lastRefreshedEpoch =
state.lastRefreshedEpoch)

            // 2. Search Bar — liquid glass
            LiquidGlassSearchBar(
                searchQuery = state.searchQuery,
                onQueryChange = actions.onQueryChange,
                leadingIcon = {
                    Icon(
                        Icons.Default.Search,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.onSurfaceVariant,
                        modifier = Modifier.size(18.dp)
                    )
                }
            )

            // 3. Action Toolbar — liquid glass buttons
            GlassActionToolbar(
                onLocationClick = actions.onLocationClick,
                onToggleSavedCitiesScreen =
actions.onToggleSavedCitiesScreen,
                onSettingsClick = actions.onSettingsClick,
                onNavigateToSettings = actions.onNavigateToSettings,
                onRefresh = actions.onRefresh,
                onShareWeather = actions.onShareWeather
            )

            // 4. Search Results — glass dropdown
            LiquidGlassSearchResults(
                searchResults = state.searchResults,
                onSelectCity = actions.onSelectCity
            )

            if (state.isLoading) {
                LinearProgressIndicator(
                    modifier =
Modifier.fillMaxWidth().clip(RoundedCornerShape(2.dp))
                )
            }

            state.errorMessage?.let { error ->
                LiquidGlassAlertCard(
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
                        text = error,
                        color = MaterialTheme.colorScheme.onErrorContainer,
                        fontSize = 12.sp,
                        fontWeight = FontWeight.W600,
                        modifier = Modifier.padding(4.dp),
                        textAlign = TextAlign.Center
                    )
```

```
                }
            }

            // 5. Weather Header — ultra-thin glass typography
            WeatherHeader(
                selectedCity = state.selectedCity,
                savedCities = state.savedCities,
                weatherCode = state.weatherCode,
                temperature = state.temperature,
                isCelsius = state.isCelsius,
                dailyForecast = state.dailyForecast,
                onRemoveCity = actions.onRemoveCity,
                onSaveCity = actions.onSaveCity,
                onToggleCelsius = actions.onToggleCelsius,
                mainTextColor = mainTextColor,
                subTextColor = subTextColor,
                isDay = isDay
            )

            // 5b. Smart Summary Card
            if (state.smartSummary.isNotBlank()) {
                SmartSummaryCard(summary = state.smartSummary, mainTextColor
= mainTextColor)
            }

            // 6. Severe Weather Alert — glass alert card
            SevereWeatherAlertCard(
                alertEvent = state.alertEvent,
                alertHeadline = state.alertHeadline,
                alertDesc = state.alertDescription
            )

            // ── Forecast & all weather details — one continuous, calm
scroll ──
            run {
                        if (state.hourlyForecast.isNotEmpty()) {
                            LiquidGlassCard(
                                title = "HOURLY FORECAST",
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                LazyRow(
                                    modifier = Modifier.fillMaxWidth(),
                                    horizontalArrangement =
Arrangement.spacedBy(16.dp),
                                    contentPadding =
PaddingValues(horizontal = 4.dp, vertical = 6.dp)
                                ) {
                                    itemsIndexed(state.hourlyForecast)
{ index, hour ->
                                        // Hourly slots are consecutive from
"Now"; derive day/night per slot.
                                        val slotMins = (nowMins + index *
60) % 1440
                                        HourlyItem(
                                            hour = hour.hour,
                                            temp =
formatTemp(hour.temperature, state.isCelsius),
                                            weatherCode = hour.weatherCode,
                                            precipChance =
hour.precipChance,
                                            isDay = slotMins in
state.sunriseMinutes..state.sunsetMinutes
                                        )
                                    }
                                }
                            }
```

```
                            HourlyTemperatureChartCard(state.hourlyForecast,
state.isCelsius)
                        }

                        if (state.dailyForecast.isNotEmpty()) {
                            LiquidGlassCard(
                                title = "7-DAY FORECAST",
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Column(
                                    verticalArrangement =
Arrangement.spacedBy(12.dp),
                                    modifier = Modifier.padding(vertical =
4.dp)
                                ) {
                                    val weekMinC = state.dailyForecast.minOf
{ it.tempMin }
                                    val weekMaxC = state.dailyForecast.maxOf
{ it.tempMax }
                                    state.dailyForecast.forEach { day ->
                                        DailyItem(
                                            day = day.day,
                                            weatherCode = day.weatherCode,
                                            tempMin =
formatTemp(day.tempMin, state.isCelsius),
                                            tempMax =
formatTemp(day.tempMax, state.isCelsius),
                                            tempMinC = day.tempMin,
                                            tempMaxC = day.tempMax,
                                            weekMinC = weekMinC,
                                            weekMaxC = weekMaxC,
                                            uvIndex = day.uvIndex,
                                            precipChance = day.precipChance
                                        )
                                    }
                                }
                            }
                        }
                    }

                    run {
                        Text(
                            text = "Weather Details",
                            style = MaterialTheme.typography.titleMedium,
                            fontWeight = FontWeight.W300,
                            color =
MaterialTheme.colorScheme.onSurfaceVariant,
                            modifier = Modifier.padding(top = 4.dp)
                        )

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement =
Arrangement.spacedBy(12.dp)
                        ) {
                            val dpC = state.temperature - (100f -
state.humidity) / 5f
                            val dpDesc = "The dew point is approx $
{formatTemp(dpC, state.isCelsius)} right now."

                            LiquidGlassMetricCard(
                                title = "Humidity",
                                value = "${state.humidity.roundToInt()}%",
                                desc = dpDesc,
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                Box(
```

```
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .height(6.dp)
                                        .clip(RoundedCornerShape(3.dp))
                                        .background(MaterialTheme.colorSchem
e.surfaceVariant)
                                ) {
                                    Box(
                                        modifier = Modifier
                                            .fillMaxHeight()
                                            .fillMaxWidth(state.humidity /
100f)
                                            .background(MaterialTheme.colorS
cheme.primary)
                                    )
                                }
                            }

                            LiquidGlassMetricCard(
                                title = "Wind Speed",
                                value = formatWind(state.windSpeed,
settings.windUnit) + if (state.windDir.isNotBlank()) "  ${state.windDir}"
else "",
                                desc = if (state.windDir.isNotBlank()) {
                                    val windDesc = when {
                                        state.windSpeed < 20f -> "Light
breeze."
                                        state.windSpeed < 40f -> "Moderate
winds."
                                        else -> "Strong winds — take care
outdoors."
                                    }
                                    "Direction: ${state.windDir}. $windDesc"
                                } else "Wind speed from current source.",
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                // Static speed bar (0–120 km/h scale)
                                Box(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .height(5.dp)
                                        .clip(RoundedCornerShape(3.dp))
                                        .background(MaterialTheme.colorSchem
e.outline.copy(alpha = 0.15f))
                                ) {
                                    Box(
                                        modifier = Modifier
                                            .fillMaxHeight()
                                            .fillMaxWidth((state.windSpeed /
120f).coerceIn(0f, 1f))
                                            .background(MaterialTheme.colorS
cheme.primary)
                                    )
                                }
                            }
                        }

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement =
Arrangement.spacedBy(12.dp)
                        ) {
                            LiquidGlassMetricCard(
                                title = "Pressure",
                                value = formatPressure(state.pressure,
settings.pressureUnit),
```

```
                                desc = if (state.pressure < 1009) "Low
pressure center. Rain showers possible." else "High pressure system. Clear
and dry.",
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                val pTrack =
MaterialTheme.colorScheme.surfaceVariant
                                val pTick =
MaterialTheme.colorScheme.outline
                                val pAccent =
MaterialTheme.colorScheme.primary
                                Canvas(modifier =
Modifier.fillMaxWidth().height(16.dp)) {
                                    val w = size.width
                                    val h = size.height
                                    val barHeight = 4.dp.toPx()
                                    val y = (h - barHeight) / 2f
                                    drawRoundRect(
                                        color = pTrack,
                                        topLeft = Offset(0f, y),
                                        size = Size(w, barHeight),
                                        cornerRadius =
CornerRadius(2.dp.toPx(), 2.dp.toPx())
                                    )
                                    val normX = w / 2f
                                    drawLine(
                                        color = pTick,
                                        start = Offset(normX, y -
2.dp.toPx()),
                                        end = Offset(normX, y + barHeight +
2.dp.toPx()),
                                        strokeWidth = 1.dp.toPx()
                                    )
                                    val fraction = ((state.pressure -
980f) / 60f).coerceIn(0f, 1f)
                                    val cx = fraction * w
                                    drawCircle(
                                        color = pAccent,
                                        radius = 4.dp.toPx(),
                                        center = Offset(cx, h / 2f)
                                    )
                                }
                            }

                            LiquidGlassMetricCard(
                                title = "Air Quality (US-AQI)",
                                value = if (state.aqi > 0) "${state.aqi} · $
{state.aqi.getAqiDescription()}" else "N/A",
                                desc = when (state.aqi) {
                                    1 -> "Good air quality. Perfect for
outdoor activities!"
                                    2 -> "Moderate. Acceptable for most
people."
                                    3 -> "Unhealthy for sensitive groups.
Limit strenuous outdoor activity."
                                    4 -> "Unhealthy. Avoid prolonged outdoor
exertion."
                                    5 -> "Very Unhealthy. Avoid outdoor
activity."
                                    6 -> "Hazardous. Stay indoors."
                                    else -> "AQI unavailable. Enable
WeatherAPI key for air quality data."
                                },
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
```

```
                                Canvas(modifier =
Modifier.fillMaxWidth().height(16.dp)) {
                                    val w = size.width
                                    val h = size.height
                                    val barHeight = 6.dp.toPx()
                                    val y = (h - barHeight) / 2f
                                    drawRoundRect(
                                        brush = Brush.horizontalGradient(
                                            colors = listOf(
                                                Color(0xFF10B981),
                                                Color(0xFFFBBF24),
                                                Color(0xFFF97316),
                                                Color(0xFFEF4444),
                                                Color(0xFF8B5CF6)
                                            )
                                        ),
                                        topLeft = Offset(0f, y),
                                        size = Size(w, barHeight),
                                        cornerRadius =
CornerRadius(3.dp.toPx(), 3.dp.toPx())
                                    )
                                    val fraction = (state.aqi /
200f).coerceIn(0f, 1f)
                                    val cx = fraction * w
                                    drawCircle(
                                        color = Color.White,
                                        radius = 5.dp.toPx(),
                                        center = Offset(cx, h / 2f)
                                    )
                                    drawCircle(
                                        color = Color(0xFF0F172A),
                                        radius = 2.dp.toPx(),
                                        center = Offset(cx, h / 2f)
                                    )
                                }
                            }
                        }

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement =
Arrangement.spacedBy(12.dp)
                        ) {
                            LiquidGlassMetricCard(
                                title = "UV Index",
                                value = "${if (state.uvIndex % 1 == 0f)
state.uvIndex.toInt().toString() else state.uvIndex.toString()} · $
{state.uvIndex.getUvDescription()}",
                                desc = when {
                                    state.uvIndex >= 11f -> "Extreme — stay
indoors between 10 AM–4 PM."
                                    state.uvIndex >= 8f  -> "Very High — SPF
30+ & protective clothing required."
                                    state.uvIndex >= 6f  -> "High — seek
shade, wear a hat."
                                    state.uvIndex >= 3f  -> "Moderate —
sunscreen recommended."
                                    else                 -> "Low UV
radiation. Safe for outdoor activity."
                                },
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                Canvas(modifier =
Modifier.fillMaxWidth().height(16.dp)) {
                                    val w = size.width
                                    val h = size.height
                                    val barHeight = 6.dp.toPx()
```

```
                                    val y = (h - barHeight) / 2f
                                    drawRoundRect(
                                        brush = Brush.horizontalGradient(
                                            colors = listOf(
                                                Color(0xFF10B981),
                                                Color(0xFFFBBF24),
                                                Color(0xFFF97316),
                                                Color(0xFFEF4444),
                                                Color(0xFF8B5CF6)
                                            )
                                        ),
                                        topLeft = Offset(0f, y),
                                        size = Size(w, barHeight),
                                        cornerRadius =
CornerRadius(3.dp.toPx(), 3.dp.toPx())
                                    )
                                    val fraction = (state.uvIndex /
11f).coerceIn(0f, 1f)
                                    val cx = fraction * w
                                    drawCircle(
                                        color = Color.White,
                                        radius = 5.dp.toPx(),
                                        center = Offset(cx, h / 2f)
                                    )
                                    drawCircle(
                                        color = Color(0xFF0F172A),
                                        radius = 2.dp.toPx(),
                                        center = Offset(cx, h / 2f)
                                    )
                                }
                            }

                            Spacer(modifier = Modifier.weight(1f))
                        }
                    }

                    run {
                        SunriseSunsetArcCard(state)

                        val pulseAlpha = 0.85f  // static — removed infinite
transition

                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement =
Arrangement.spacedBy(12.dp)
                        ) {
                            LiquidGlassMetricCard(
                                title = "Feels Like",
                                value = formatTemp(state.feelsLike,
state.isCelsius),
                                desc = when {
                                    state.feelsLike < state.temperature - 3
-> "Feels colder due to wind chill."
                                    state.feelsLike > state.temperature + 3
-> "Feels hotter due to humidity."
                                    else -> "Feels close to the actual
temperature."
                                },
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                Box(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .height(6.dp)
                                        .clip(RoundedCornerShape(3.dp))
```

```
                                        .background(MaterialTheme.colorSchem
e.surfaceVariant)
                                ) {
                                    Box(
                                        modifier = Modifier
                                            .fillMaxHeight()
                                            .fillMaxWidth(((state.feelsLike
+ 20f) / 60f).coerceIn(0f, 1f))
                                            .background(MaterialTheme.colorS
cheme.primary)
                                    )
                                }
                            }

                            LiquidGlassMetricCard(
                                title = "Visibility",
                                value = formatVisibility(state.visibility,
settings.visibilityUnit),
                                desc = when {
                                    state.visibility >= 10f -> "Clear
visibility. Great conditions."
                                    state.visibility >= 5f  -> "Good
visibility with minor haze."
                                    state.visibility >= 2f  -> "Moderate fog
or haze present."
                                    else                    -> "Poor
visibility. Drive with caution."
                                },
                                isDark = isDarkTheme,
                                modifier = Modifier.weight(1f)
                            ) {
                                Box(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .height(6.dp)
                                        .clip(RoundedCornerShape(3.dp))
                                        .background(MaterialTheme.colorSchem
e.surfaceVariant)
                                ) {
                                    Box(
                                        modifier = Modifier
                                            .fillMaxHeight()
                                            .fillMaxWidth((state.visibility
/ 20f).coerceIn(0f, 1f))
                                            .background(MaterialTheme.colorS
cheme.primary)
                                    )
                                }
                            }
                        }

                                                // Pollen card — shown when
data is available
                        val pollen = state.pollenData
                        if (pollen != null && pollen.peak > 0f) {
                            PollenCard(
                                pollen = pollen,
                                isDark = isDarkTheme,
                                modifier = Modifier.fillMaxWidth()
                            )
                        }

                                                LiquidGlassMetricCard(
                            title = "Climatic Anomalies",
                            value = "Normal",
                            desc = "Current telemetry values align with 30-
year seasonal models.",
                            isDark = isDarkTheme,
```

```
                            modifier = Modifier.fillMaxWidth()
                        ) {
                            Row(
                                verticalAlignment =
Alignment.CenterVertically,
                                horizontalArrangement =
Arrangement.spacedBy(6.dp),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                val statusDot =
MaterialTheme.colorScheme.primary
                                Canvas(modifier = Modifier.size(8.dp)) {
                                    drawCircle(
                                        color = statusDot.copy(alpha =
pulseAlpha),
                                        radius = 4.dp.toPx()
                                    )
                                }
                                Text(
                                    text = "Telemetry Aligned",
                                    fontSize = 11.sp,
                                    fontWeight = FontWeight.W700,
                                    color =
MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            }
                        }


            }

            // ── Location-aware news (RSS): weather + local
──────────────────
            NewsCard(
                weatherArticles = state.newsArticles,
                localArticles = state.localNewsArticles,
                isLoading = state.isNewsLoading,
                errorMessage = state.newsError,
                onOpenArticle = actions.onOpenArticle,
                onRefreshNews = actions.onRefreshNews,
                modifier = Modifier.fillMaxWidth()
            )

            Spacer(modifier = Modifier.height(24.dp))
        }

        // Pinned compact header — appears once the hero scrolls out of
view.
        val showCompactHeader = scrollState.value >
with(LocalDensity.current) { 330.dp.toPx() }
        AnimatedVisibility(
            visible = showCompactHeader,
            enter = fadeIn() + slideInVertically { -it },
            exit = fadeOut() + slideOutVertically { -it },
            modifier = Modifier.align(Alignment.TopCenter)
        ) {
            CompactWeatherHeader(
                city = state.selectedCity,
                temp = formatTemp(state.temperature, state.isCelsius),
                condition = state.weatherCode.getWeatherCodeDescription(),
                weatherCode = state.weatherCode,
                isDay = isDay
            )
        }

        // Settings Dialog — liquid glass
        if (state.showKeyDialog) {
            SettingsDialog(
```

```
                state = state,
                onDismissSettings = actions.onDismissSettings,
                onSaveWeatherApiKey = actions.onSaveWeatherApiKey,
                onToggleCelsius = actions.onToggleCelsius
            )
        }

        // Saved Cities Dialog — liquid glass
        if (state.showSavedCitiesScreen) {
            SavedCitiesDialog(
                state = state,
                onToggleSavedCitiesScreen =
actions.onToggleSavedCitiesScreen,
                onSelectCity = actions.onSelectCity,
                onRemoveCity = actions.onRemoveCity
            )
        }
    }
    }
}

// ─── COMPACT (PINNED) HEADER ──────────────────────────────────────

@Composable
private fun CompactWeatherHeader(
    city: String,
    temp: String,
    condition: String,
    weatherCode: Int,
    isDay: Boolean
) {
    Surface(
        color = MaterialTheme.colorScheme.surface,
        shadowElevation = 4.dp,
        modifier = Modifier.fillMaxWidth()
    ) {
        Row(
            modifier = Modifier
                .statusBarsPadding()
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = city,
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.W600,
                    color = MaterialTheme.colorScheme.onSurface,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis
                )
                Text(
                    text = condition,
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis
                )
            }
            Spacer(modifier = Modifier.width(10.dp))
            Text(
                text = temp,
                style = MaterialTheme.typography.titleLarge,
                color = MaterialTheme.colorScheme.onSurface
            )
            Spacer(modifier = Modifier.width(8.dp))
```

```
            WeatherIcon(weatherCode, modifier = Modifier.size(30.dp), isDay
= isDay)
        }
    }
}

// ─── NEWS CARD (RSS headlines) ──────────────────────────────────

@Composable
fun NewsCard(
    weatherArticles: List<NewsArticle>,
    localArticles: List<NewsArticle>,
    isLoading: Boolean,
    errorMessage: String?,
    onOpenArticle: (String) -> Unit,
    onRefreshNews: () -> Unit,
    modifier: Modifier = Modifier
) {
    var tab by rememberSaveable { mutableStateOf(0) }   // 0 = Weather, 1 =
Local
    val articles = if (tab == 0) weatherArticles else localArticles

    LiquidGlassCard(modifier = modifier) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Row(
                horizontalArrangement = Arrangement.spacedBy(18.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                NewsTabLabel("Weather", selected = tab == 0) { tab = 0 }
                NewsTabLabel("Local", selected = tab == 1) { tab = 1 }
            }
            IconButton(onClick = onRefreshNews, modifier =
Modifier.size(32.dp)) {
                Icon(
                    imageVector = Icons.Default.Refresh,
                    contentDescription = "Refresh news",
                    tint = MaterialTheme.colorScheme.onSurfaceVariant,
                    modifier = Modifier.size(18.dp)
                )
            }
        }

        Spacer(modifier = Modifier.height(Spacing.small))

        when {
            isLoading && articles.isEmpty() -> {
                Row(
                    modifier = Modifier.fillMaxWidth().padding(vertical =
12.dp),
                    horizontalArrangement = Arrangement.Center,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    CircularProgressIndicator(
                        modifier = Modifier.size(20.dp),
                        strokeWidth = 2.dp
                    )
                    Spacer(modifier = Modifier.width(10.dp))
                    Text(
                        text = "Loading headlines…",
                        fontSize = 13.sp,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
```

```
            articles.isEmpty() && errorMessage != null -> {
                Text(
                    text = errorMessage,
                    fontSize = 13.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    modifier = Modifier.padding(vertical = 8.dp)
                )
            }

            articles.isEmpty() -> {
                Text(
                    text = if (tab == 0) "No weather news right now." else
"No local news right now.",
                    fontSize = 13.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    modifier = Modifier.padding(vertical = 8.dp)
                )
            }

            else -> {
                Column(verticalArrangement = Arrangement.spacedBy(2.dp)) {
                    articles.forEachIndexed { index, article ->
                        NewsItem(article = article, onClick =
{ onOpenArticle(article.link) })
                        if (index != articles.lastIndex) {
                            HorizontalDivider(
                                thickness = 0.5.dp,
                                color =
MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.4f)
                            )
                        }
                    }
                }
            }
        }
    }
}

@Composable
private fun NewsTabLabel(text: String, selected: Boolean, onClick: () ->
Unit) {
    Text(
        text = text.uppercase(),
        style = MaterialTheme.typography.titleSmall,
        color = if (selected) MaterialTheme.colorScheme.primary
                else MaterialTheme.colorScheme.onSurfaceVariant,
        modifier = Modifier.clickable(onClick = onClick)
    )
}

@Composable
private fun NewsItem(
    article: NewsArticle,
    onClick: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(vertical = 10.dp),
        verticalArrangement = Arrangement.spacedBy(3.dp)
    ) {
        Text(
            text = article.title,
            style = MaterialTheme.typography.bodyMedium,
            fontWeight = FontWeight.W600,
            color = MaterialTheme.colorScheme.onSurface,
```

```
            maxLines = 3,
            overflow = TextOverflow.Ellipsis
        )
        val meta = listOf(article.source, article.pubDate)
            .filter { it.isNotBlank() }
            .joinToString("  •  ")
        if (meta.isNotBlank()) {
            Text(
                text = meta,
                fontSize = 11.sp,
                color =
MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.75f),
                maxLines = 1,
                overflow = TextOverflow.Ellipsis
            )
        }
    }
}

// ─── BACKGROUND MESH (static — no animation overhead) ──────────────

@Composable
fun LiquidMeshOverlay(modifier: Modifier = Modifier) {
    // Intentionally empty: removed heavy infinite-transition Canvas blobs.
    // Background gradient is handled by the screen's Box background.
}

// ─── GLASS ACTION TOOLBAR ──────────────────────────────────────────

@Composable
fun GlassActionToolbar(
    onLocationClick: () -> Unit,
    onToggleSavedCitiesScreen: (Boolean) -> Unit,
    onSettingsClick: () -> Unit,
    onNavigateToSettings: () -> Unit = {},
    onRefresh: () -> Unit,
    onShareWeather: () -> Unit = {},
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
            LiquidGlassToolbarButton(onClick = onLocationClick) {
                Icon(Icons.Default.Place, contentDescription = "Use Current
Location", modifier = Modifier.size(20.dp))
            }

            LiquidGlassToolbarButton(onClick =
{ onToggleSavedCitiesScreen(true) }) {
                Icon(Icons.AutoMirrored.Filled.List, contentDescription =
"Favorites", modifier = Modifier.size(20.dp))
            }
        }

        Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
            LiquidGlassToolbarButton(onClick = onRefresh) {
                Icon(Icons.Default.Refresh, contentDescription = "Refresh
Weather", modifier = Modifier.size(20.dp))
            }
            LiquidGlassToolbarButton(onClick = onShareWeather) {
                Icon(Icons.Default.Share, contentDescription = "Share
weather", modifier = Modifier.size(20.dp))
            }
            LiquidGlassToolbarButton(onClick = onNavigateToSettings) {
```

```
                Icon(Icons.Default.Settings, contentDescription =
"Settings", modifier = Modifier.size(20.dp))
            }
        }
    }
}

// ─── WEATHER HEADER ──────────────────────────────────────────────

@Composable
fun WeatherHeader(
    selectedCity: String,
    savedCities: List<GeocodingResult>,
    weatherCode: Int,
    temperature: Float,
    isCelsius: Boolean,
    dailyForecast: List<DayForecast>,
    onRemoveCity: (GeocodingResult) -> Unit,
    onSaveCity: () -> Unit,
    onToggleCelsius: () -> Unit,
    mainTextColor: Color,
    subTextColor: Color,
    isDay: Boolean = true,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.fillMaxWidth()
        ) {
            // Left spacer balances the favourite button so the title stays
centered.
            Spacer(modifier = Modifier.size(36.dp))
            Text(
                text = selectedCity,
                style = MaterialTheme.typography.displaySmall,
                color = mainTextColor,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis,
                textAlign = TextAlign.Center,
                modifier = Modifier
                    .weight(1f)
                    .padding(horizontal = 4.dp)
            )

            val isFavorite = savedCities.any { it.name.equals(selectedCity,
ignoreCase = true) }
            IconButton(
                onClick = {
                    if (isFavorite) {
                        val matched = savedCities.firstOrNull
{ it.name.equals(selectedCity, ignoreCase = true) }
                        if (matched != null) onRemoveCity(matched)
                    } else {
                        onSaveCity()
                    }
                },
                modifier = Modifier.size(36.dp)
            ) {
                Icon(
                    imageVector = Icons.Default.Star,
                    contentDescription = "Toggle Favorite",
                    tint = if (isFavorite) MaterialTheme.colorScheme.primary
else MaterialTheme.colorScheme.onSurfaceVariant,
                    modifier = Modifier.size(20.dp)
```

```
                )
            }
        }

        Spacer(modifier = Modifier.height(4.dp))

        // Temperature gently counts up to its value on load / change.
        val animatedTemp by animateFloatAsState(
            targetValue = temperature,
            animationSpec = tween(durationMillis = 900, easing =
FastOutSlowInEasing),
            label = "tempCountUp"
        )
        Row(
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.Center
        ) {
            Text(
                text = formatTemp(animatedTemp, isCelsius),
                style = MaterialTheme.typography.displayLarge,
                color = mainTextColor,
                modifier = Modifier.clickable { onToggleCelsius() }
            )
            Spacer(modifier = Modifier.width(12.dp))
            WeatherIcon(
                code = weatherCode,
                modifier = Modifier.size(76.dp),
                isDay = isDay
            )
        }

        Spacer(modifier = Modifier.height(2.dp))

        Text(
            text = weatherCode.getWeatherCodeDescription(),
            style = MaterialTheme.typography.titleMedium,
            color = subTextColor
        )

        val firstDay = dailyForecast.firstOrNull()
        if (firstDay != null) {
            Spacer(modifier = Modifier.height(2.dp))
            Text(
                text = "H:${formatTemp(firstDay.tempMax, isCelsius)}   L:$
{formatTemp(firstDay.tempMin, isCelsius)}",
                style = MaterialTheme.typography.bodyMedium,
                color = subTextColor
            )
        }
    }
}

// ─── POLLEN CARD
──────────────────────────────────────────────────────────

@Composable
fun PollenCard(
    pollen: com.example.aiweathermonitor.data.models.PollenData,
    isDark: Boolean,
    modifier: Modifier = Modifier
) {
    val riskColor = when (pollen.riskLevel) {
        "Low"       -> Color(0xFF22C55E)
        "Moderate"  -> Color(0xFFEAB308)
        "High"      -> Color(0xFFF97316)
        else         -> Color(0xFFEF4444)  // Very High
    }
```

```
    LiquidGlassMetricCard(
        title = "Pollen",
        value = pollen.riskLevel,
        desc = if (pollen.activeTypes.isNotEmpty())
            "Active: ${pollen.activeTypes.joinToString(", ")}"
        else
            "No significant pollen detected today",
        isDark = isDark,
        modifier = modifier
    ) {
        Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
            // Risk bar
            val fraction = (pollen.peak / 100f).coerceIn(0f, 1f)
            Box(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(4.dp)
                    .clip(androidx.compose.foundation.shape.RoundedCornerSha
pe(2.dp))
                    .background(MaterialTheme.colorScheme.surfaceVariant)
            ) {
                Box(
                    modifier = Modifier
                        .fillMaxWidth(fraction)
                        .fillMaxHeight()
                        .clip(androidx.compose.foundation.shape.RoundedCorne
rShape(2.dp))
                        .background(riskColor)
                )
            }
            // Per-type breakdown (non-zero only)
            val types = listOf(
                "Grass" to pollen.grass,
                "Birch" to pollen.birch,
                "Ragweed" to pollen.ragweed,
                "Alder" to pollen.alder,
                "Mugwort" to pollen.mugwort,
                "Olive" to pollen.olive
            ).filter { it.second > 1f }
            if (types.isNotEmpty()) {
                Row(
                    horizontalArrangement = Arrangement.spacedBy(8.dp),
                    modifier =
Modifier.horizontalScroll(rememberScrollState())
                ) {
                    types.forEach { (name, count) ->
                        Surface(
                            color = riskColor.copy(alpha = 0.15f),
                            shape =
androidx.compose.foundation.shape.RoundedCornerShape(20.dp)
                        ) {
                            Text(
                                text = " ${count.toInt()}",
                                fontSize = 11.sp,
                                color = riskColor,
                                modifier = Modifier.padding(horizontal =
8.dp, vertical = 3.dp)
                            )
                        }
                    }
                }
            }
        }
    }
}

// ——— SMART SUMMARY CARD
————————————————————————————————————————
```

```
@Composable
fun SmartSummaryCard(
    summary: String,
    mainTextColor: Color,
    modifier: Modifier = Modifier
) {
    LiquidGlassCard(modifier = modifier.fillMaxWidth()) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(10.dp),
            modifier = Modifier.padding(horizontal = 16.dp, vertical =
12.dp)
        ) {
            Text(
                text = "✦",
                fontSize = 14.sp,
                color = mainTextColor.copy(alpha = 0.70f)
            )
            Text(
                text = summary,
                fontSize = 14.sp,
                color = mainTextColor.copy(alpha = 0.90f),
                fontWeight = FontWeight.W400,
                lineHeight = 20.sp,
                modifier = Modifier.weight(1f)
            )
        }
    }
}

// ─── SEVERE WEATHER ALERT ──────────────────────────────────────

@Composable
fun SevereWeatherAlertCard(
    alertEvent: String?,
    alertHeadline: String?,
    alertDesc: String?,
    modifier: Modifier = Modifier
) {
    if (!alertEvent.isNullOrBlank()) {
        LiquidGlassAlertCard(
            modifier = modifier.fillMaxWidth()
        ) {
            Text(
                text = "⚠ ${alertEvent.uppercase()}",
                color = MaterialTheme.colorScheme.onErrorContainer,
                fontSize = 13.sp,
                fontWeight = FontWeight.W700
            )
            if (!alertHeadline.isNullOrBlank()) {
                Text(
                    text = alertHeadline,
                    color = MaterialTheme.colorScheme.onErrorContainer,
                    fontSize = 14.sp,
                    fontWeight = FontWeight.W600
                )
            }
            if (!alertDesc.isNullOrBlank()) {
                Text(
                    text = alertDesc,
                    color =
MaterialTheme.colorScheme.onErrorContainer.copy(alpha = 0.80f),
                    fontSize = 12.sp,
                    lineHeight = 16.sp
                )
            }
        }
```

```
    }
}

// ─── SETTINGS DIALOG ──────────────────────────────────────────

@Composable
fun SettingsDialog(
    state: WeatherState,
    onDismissSettings: () -> Unit,
    onSaveWeatherApiKey: (String) -> Unit,
    onToggleCelsius: () -> Unit
) {
    var tempWeatherApiKeyText by remember
{ mutableStateOf(state.weatherApiKey) }

    LiquidGlassDialog(
        onDismissRequest = onDismissSettings,
        title = {
            Text(
                "Settings",
                fontSize = 18.sp,
                fontWeight = FontWeight.W600,
                color = MaterialTheme.colorScheme.onSurface
            )
        },
        text = {
            Column(
                verticalArrangement = Arrangement.spacedBy(12.dp),
                modifier = Modifier.fillMaxWidth()
            ) {
                Text(
                    text = "Temperature Units",
                    fontWeight = FontWeight.W700,
                    fontSize = 14.sp,
                    color = MaterialTheme.colorScheme.onSurface
                )
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = "Use Fahrenheit (°F)",
                        fontSize = 13.sp,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Switch(
                        checked = !state.isCelsius,
                        onCheckedChange = { onToggleCelsius() }
                    )
                }

                HorizontalDivider()

                Text(
                    text = "WeatherAPI.com Key (Optional)",
                    fontWeight = FontWeight.W700,
                    fontSize = 14.sp,
                    color = MaterialTheme.colorScheme.onSurface
                )
                LiquidGlassTextField(
                    value = tempWeatherApiKeyText,
                    onValueChange = { tempWeatherApiKeyText = it },
                    placeholder = { Text("Enter WeatherAPI.com Key",
fontSize = 12.sp, color = MaterialTheme.colorScheme.onSurfaceVariant) },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth()
                )
```

```
                Text(
                    text = "Optional: unlocks UV index, AQI, weather alerts
& 3-day forecast from weatherapi.com. Free tier available.",
                    fontSize = 11.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )

                HorizontalDivider()

                Text(
                    text = "Open-Meteo (No Key Required)",
                    fontWeight = FontWeight.W700,
                    fontSize = 14.sp,
                    color = MaterialTheme.colorScheme.primary
                )
                Text(
                    text = "Always active as primary or fallback source.
Provides 14-day forecast with zero configuration.",
                    fontSize = 11.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        },
        confirmButton = {
            Button(
                onClick = {
                    onSaveWeatherApiKey(tempWeatherApiKeyText.trim())
                }
            ) {
                Text("Save")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismissSettings) {
                Text("Cancel")
            }
        }
    )
}

// ─── SAVED CITIES DIALOG ──────────────────────────────────────

@Composable
fun SavedCitiesDialog(
    state: WeatherState,
    onToggleSavedCitiesScreen: (Boolean) -> Unit,
    onSelectCity: (GeocodingResult) -> Unit,
    onRemoveCity: (GeocodingResult) -> Unit
) {
    LiquidGlassDialog(
        onDismissRequest = { onToggleSavedCitiesScreen(false) },
        title = {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    "Saved Favorites",
                    fontSize = 18.sp,
                    fontWeight = FontWeight.W600,
                    color = MaterialTheme.colorScheme.onSurface
                )
                TextButton(onClick = { onToggleSavedCitiesScreen(false) }) {
                    Text("Close")
                }
            }
        },
```

```
        text = {
            Column(
                modifier = Modifier.fillMaxWidth(),
                verticalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                if (state.savedCities.isEmpty()) {
                    Text(
                        text = "No favorite cities saved yet. Tap the star
button next to any city name to bookmark it!",
                        fontSize = 13.sp,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        textAlign = TextAlign.Center,
                        modifier = Modifier.padding(vertical = 12.dp)
                    )
                } else {
                    androidx.compose.foundation.lazy.LazyColumn(
                        verticalArrangement = Arrangement.spacedBy(8.dp),
                        modifier = Modifier.fillMaxWidth().heightIn(max =
300.dp)
                    ) {
                        items(state.savedCities) { city ->
                            val cardShape = RoundedCornerShape(16.dp)
                            Card(
                                shape = cardShape,
                                colors =
CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant),
                                modifier = Modifier
                                    .fillMaxWidth()
                            ) {
                                Row(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .clickable {
                                            onSelectCity(city)
                                            onToggleSavedCitiesScreen(false)
                                        }
                                        .padding(horizontal = 14.dp,
vertical = 12.dp),
                                    horizontalArrangement =
Arrangement.SpaceBetween,
                                    verticalAlignment =
Alignment.CenterVertically
                                ) {
                                    Column {
                                        Text(text = city.name, fontWeight =
FontWeight.W600, fontSize = 14.sp, color =
MaterialTheme.colorScheme.onSurface)
                                        if (city.country != null) {
                                            Text(text = city.country, color
= MaterialTheme.colorScheme.onSurfaceVariant, fontSize = 11.sp)
                                        }
                                    }

                                    IconButton(
                                        onClick = { onRemoveCity(city) },
                                        modifier = Modifier.size(32.dp)
                                    ) {
                                        Icon(
                                            imageVector =
Icons.Default.Delete,
                                            contentDescription = "Remove
Favorite",
                                            tint =
MaterialTheme.colorScheme.error,
                                            modifier = Modifier.size(18.dp)
                                        )
                                    }
```

```
                                }
                            }
                        }
                    }
                }
            }
        },
        confirmButton = {}
    )
}

// ─── SUNRISE/SUNSET ARC CARD ──────────────────────────────────────

private fun calculateSunPosition(
    timeInMins: Int,
    sunriseMins: Int,
    sunsetMins: Int,
    width: Float,
    height: Float
): Offset? {
    if (timeInMins in sunriseMins..sunsetMins) {
        val progress = if (sunsetMins > sunriseMins) {
            (timeInMins - sunriseMins).toFloat() / (sunsetMins -
sunriseMins).toFloat()
        } else 0.5f
        val t = progress
        val u = 1f - t
        val x = u * u * 0f + 2f * u * t * (width / 2f) + t * t * width
        val y = u * u * height + 2f * u * t * (-height * 0.4f) + t * t *
height
        return Offset(x, y)
    }
    return null
}

private fun calculateChartPoints(
    temps: List<Float>,
    minTemp: Float,
    tempRange: Float,
    width: Float,
    topY: Float,        // top of the plotting band (leaves room for temp
labels)
    bottomY: Float,     // bottom of the plotting band (leaves room for hour
labels)
    xInset: Float       // horizontal inset so edge dots/labels don't clip
): List<Offset> {
    val n = temps.size
    val usableWidth = (width - 2f * xInset).coerceAtLeast(1f)
    val stepX = if (n > 1) usableWidth / (n - 1) else 0f
    val bandHeight = (bottomY - topY).coerceAtLeast(1f)
    val points = ArrayList<Offset>(n)
    for (i in temps.indices) {
        val x = xInset + i * stepX
        val normalized = (temps[i] - minTemp) / tempRange   // 0 = coldest,
1 = hottest
        val y = bottomY - normalized * bandHeight            // hottest sits
at topY
        points.add(Offset(x, y))
    }
    return points
}

@Composable
fun SunriseSunsetArcCard(
    state: WeatherState,
    modifier: Modifier = Modifier
) {
    val sunriseMins = state.sunriseMinutes
```

```
    val sunsetMins = state.sunsetMinutes
    val diffMins = (sunsetMins - sunriseMins).coerceAtLeast(0)
    val diffHours = diffMins / 60
    val diffMinsRemain = diffMins % 60
    val daylightStr = if (diffMinsRemain > 0) {
        "$diffHours h $diffMinsRemain m of daylight"
    } else {
        "$diffHours hours of daylight"
    }

    LiquidGlassCard(
        title = "SUNRISE & SUNSET",
        modifier = modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier.fillMaxWidth(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            val arcColor = MaterialTheme.colorScheme.outline
            val baseLineColor = MaterialTheme.colorScheme.outlineVariant
            val sunColor = MaterialTheme.colorScheme.primary
            val offColor = MaterialTheme.colorScheme.onSurfaceVariant

            val nowMins = run {
                val cal = java.util.Calendar.getInstance()
                cal.get(java.util.Calendar.HOUR_OF_DAY) * 60 +
cal.get(java.util.Calendar.MINUTE)
            }
            val total = (sunsetMins - sunriseMins).coerceAtLeast(1)
            val progress = ((nowMins - sunriseMins).toFloat() /
total).coerceIn(0f, 1f)
            val isUp = nowMins in sunriseMins..sunsetMins

            BoxWithConstraints(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(80.dp)
                    .padding(horizontal = 16.dp)
            ) {
                val wDp = maxWidth
                val hDp = maxHeight
                Canvas(modifier = Modifier.fillMaxSize()) {
                val width = size.width
                val height = size.height
                val arcPath = androidx.compose.ui.graphics.Path().apply {
                    moveTo(0f, height)
                    quadraticTo(width / 2f, -height * 0.4f, width, height)
                }

                drawPath(
                    path = arcPath,
                    color = arcColor,
                    style = androidx.compose.ui.graphics.drawscope.Stroke(
                        width = 2.dp.toPx(),
                        pathEffect =
androidx.compose.ui.graphics.PathEffect.dashPathEffect(floatArrayOf(15f,
15f), 0f)
                    )
                )

                drawLine(
                    color = baseLineColor,
                    start = androidx.compose.ui.geometry.Offset(0f, height),
                    end = androidx.compose.ui.geometry.Offset(width,
height),
                    strokeWidth = 1.dp.toPx()
                )
```

```
            }

                // Sun icon riding the arc at the current sun position.
                val iconSize = 26.dp
                if (isUp) {
                    val t = progress
                    val u = 1f - t
                    val yFrac = u * u + t * t - 0.8f * u * t
                    Icon(
                        imageVector = Icons.Outlined.WbSunny,
                        contentDescription = "Sun position",
                        tint = sunColor,
                        modifier = Modifier
                            .size(iconSize)
                            .offset(x = wDp * progress - iconSize / 2, y =
hDp * yFrac - iconSize / 2)
                    )
                } else {
                    // Below the horizon — dimmed at the appropriate edge.
                    Icon(
                        imageVector = Icons.Outlined.WbSunny,
                        contentDescription = "Sun below horizon",
                        tint = offColor.copy(alpha = 0.45f),
                        modifier = Modifier
                            .size(20.dp)
                            .offset(
                                x = if (nowMins < sunriseMins) (-4).dp else
wDp - 16.dp,
                                y = hDp - 14.dp
                            )
                    )
                }
            }

            Spacer(modifier = Modifier.height(8.dp))

            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Column(horizontalAlignment = Alignment.Start) {
                    Row(
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.spacedBy(4.dp)
                    ) {
                        Icon(
                            imageVector = Icons.Outlined.WbSunny,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.primary,
                            modifier = Modifier.size(14.dp)
                        )
                        Text("Sunrise", fontSize = 10.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant, fontWeight = FontWeight.W700)
                    }
                    Text(state.sunrise, fontSize = 13.sp, color =
MaterialTheme.colorScheme.onSurface, fontWeight = FontWeight.W600)
                }
                Text(
                    text = daylightStr,
                    fontSize = 11.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    fontWeight = FontWeight.W500
                )
                Column(horizontalAlignment = Alignment.End) {
                    Row(
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.spacedBy(4.dp)
```

```
                    ) {
                        Icon(
                            imageVector = Icons.Outlined.WbTwilight,
                            contentDescription = null,
                            tint = MaterialTheme.colorScheme.tertiary,
                            modifier = Modifier.size(14.dp)
                        )
                        Text("Sunset", fontSize = 10.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant, fontWeight = FontWeight.W700)
                    }
                    Text(state.sunset, fontSize = 13.sp, color =
MaterialTheme.colorScheme.onSurface, fontWeight = FontWeight.W600)
                }
            }
        }
    }
}

// ─── HOURLY TEMPERATURE CHART ──────────────────────────────

@Composable
fun HourlyTemperatureChartCard(
    hourlyForecast: List<HourForecast>,
    isCelsius: Boolean,
    modifier: Modifier = Modifier
) {
    LiquidGlassCard(
        title = "TEMPERATURE TREND",
        modifier = modifier.fillMaxWidth()
    ) {
        if (hourlyForecast.isEmpty()) {
            Text("No forecast data available", fontSize = 12.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant)
        } else {
            Column(modifier = Modifier.fillMaxWidth()) {
                val chartLine = MaterialTheme.colorScheme.primary
                val surfaceColor = MaterialTheme.colorScheme.surface
                val tempLabelArgb =
MaterialTheme.colorScheme.onSurface.toArgb()
                val hourLabelArgb =
MaterialTheme.colorScheme.onSurfaceVariant.toArgb()
                val readoutArgb =
MaterialTheme.colorScheme.onPrimary.toArgb()
                val n = hourlyForecast.size
                var selectedIndex by remember(hourlyForecast)
{ mutableStateOf(-1) }
                Canvas(
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(140.dp)
                        .pointerInput(hourlyForecast) {
                            val xi = 18.dp.toPx()
                            fun idxFor(x: Float): Int {
                                val usable = (size.width - 2f *
xi).coerceAtLeast(1f)
                                val step = if (n > 1) usable / (n - 1) else
0f
                                return if (step <= 0f) 0 else ((x - xi) /
step).roundToInt().coerceIn(0, n - 1)
                            }
                            detectTapGestures { o -> selectedIndex =
idxFor(o.x) }
                        }
                        .pointerInput(hourlyForecast) {
                            val xi = 18.dp.toPx()
                            fun idxFor(x: Float): Int {
                                val usable = (size.width - 2f *
xi).coerceAtLeast(1f)
```

```
                                val step = if (n > 1) usable / (n - 1) else
0f
                                return if (step <= 0f) 0 else ((x - xi) /
step).roundToInt().coerceIn(0, n - 1)
                            }
                            detectHorizontalDragGestures(
                                onDragStart = { o -> selectedIndex =
idxFor(o.x) },
                                onHorizontalDrag = { change, _ ->
selectedIndex = idxFor(change.position.x) }
                            )
                        }
                ) {
                    val width = size.width
                    val height = size.height

                    val temps = hourlyForecast.map {
                        if (isCelsius) it.temperature else (it.temperature *
9f / 5f + 32f)
                    }
                    val minTemp = temps.minOrNull() ?: 0f
                    val maxTemp = temps.maxOrNull() ?: 100f
                    val tempRange = (maxTemp - minTemp).coerceAtLeast(1f)

                    // Insets: room above for temp labels, below for hour
labels, sides for edge dots.
                    val xInset = 18.dp.toPx()
                    val topY = 30.dp.toPx()
                    val bottomY = height - 22.dp.toPx()
                    val points = calculateChartPoints(temps, minTemp,
tempRange, width, topY, bottomY, xInset)

                    // Gradient fill under the line (down to the baseline)
                    val fillPath = androidx.compose.ui.graphics.Path().apply
{
                        if (points.isNotEmpty()) {
                            moveTo(points.first().x, bottomY)
                            for (p in points) lineTo(p.x, p.y)
                            lineTo(points.last().x, bottomY)
                            close()
                        }
                    }
                    drawPath(
                        path = fillPath,
                        brush =
androidx.compose.ui.graphics.Brush.verticalGradient(
                            colors = listOf(
                                chartLine.copy(alpha = 0.18f),
                                chartLine.copy(alpha = 0.04f),
                                Color.Transparent
                            ),
                            startY = topY,
                            endY = bottomY
                        )
                    )

                    val linePath = androidx.compose.ui.graphics.Path().apply
{
                        if (points.isNotEmpty()) {
                            moveTo(points[0].x, points[0].y)
                            for (i in 1 until points.size)
lineTo(points[i].x, points[i].y)
                        }
                    }
                    drawPath(
                        path = linePath,
                        color = chartLine,
```

```
                        style =
androidx.compose.ui.graphics.drawscope.Stroke(
                            width = 2.5.dp.toPx(),
                            cap =
androidx.compose.ui.graphics.StrokeCap.Round
                        )
                    )

                    val tempPaint = android.graphics.Paint().apply {
                        color = tempLabelArgb
                        textSize = 10.dp.toPx()
                        textAlign = android.graphics.Paint.Align.CENTER
                        typeface = android.graphics.Typeface.DEFAULT_BOLD
                        isAntiAlias = true
                    }
                    val hourPaint = android.graphics.Paint().apply {
                        color = hourLabelArgb
                        textSize = 9.dp.toPx()
                        textAlign = android.graphics.Paint.Align.CENTER
                        typeface = android.graphics.Typeface.DEFAULT_BOLD
                        isAntiAlias = true
                    }

                    // Thin out labels so they never crowd, regardless of
point count.
                    val n = points.size
                    val tempStep = when { n > 16 -> 3; n > 10 -> 2; else ->
1 }
                    val hourStep = ((n + 5) / 6).coerceAtLeast(1)
                    for (i in points.indices) {
                        drawCircle(color = chartLine.copy(alpha = 0.15f),
radius = 5.dp.toPx(), center = points[i])
                        drawCircle(color = chartLine, radius =
2.5.dp.toPx(), center = points[i])

                        if (i % tempStep == 0 || i == n - 1) {
                            drawContext.canvas.nativeCanvas.drawText(
                                temps[i].roundToInt().toString() + "°",
                                points[i].x,
                                points[i].y - 9.dp.toPx(),
                                tempPaint
                            )
                        }
                        if (i % hourStep == 0 || i == n - 1) {
                            drawContext.canvas.nativeCanvas.drawText(
                                hourlyForecast[i].hour,
                                points[i].x,
                                height - 5.dp.toPx(),
                                hourPaint
                            )
                        }
                    }

                    // ── Scrubber: guide line, highlight + readout for the
touched point ──
                    if (selectedIndex in points.indices) {
                        val sp = points[selectedIndex]
                        drawLine(
                            color = chartLine.copy(alpha = 0.45f),
                            start =
androidx.compose.ui.geometry.Offset(sp.x, topY),
                            end = androidx.compose.ui.geometry.Offset(sp.x,
bottomY),
                            strokeWidth = 1.5.dp.toPx(),
                            pathEffect =
androidx.compose.ui.graphics.PathEffect.dashPathEffect(floatArrayOf(8f, 8f),
0f)
                        )
```

```
                        drawCircle(color = chartLine.copy(alpha = 0.20f),
radius = 9.dp.toPx(), center = sp)
                        drawCircle(color = surfaceColor, radius =
5.dp.toPx(), center = sp)
                        drawCircle(color = chartLine, radius = 4.dp.toPx(),
center = sp)

                        val readoutPaint = android.graphics.Paint().apply {
                            color = readoutArgb
                            textSize = 10.dp.toPx()
                            textAlign = android.graphics.Paint.Align.CENTER
                            typeface =
android.graphics.Typeface.DEFAULT_BOLD
                            isAntiAlias = true
                        }
                        val label = hourlyForecast[selectedIndex].hour + "
" + temps[selectedIndex].roundToInt() + "°"
                        val padH = 10.dp.toPx()
                        val pillW = readoutPaint.measureText(label) + 2f *
padH
                        val pillH = 20.dp.toPx()
                        val cx = sp.x.coerceIn(pillW / 2f, width - pillW /
2f)
                        val pillTop = 1.dp.toPx()
                        drawRoundRect(
                            color = chartLine,
                            topLeft = androidx.compose.ui.geometry.Offset(cx
- pillW / 2f, pillTop),
                            size = androidx.compose.ui.geometry.Size(pillW,
pillH),
                            cornerRadius =
androidx.compose.ui.geometry.CornerRadius(pillH / 2f, pillH / 2f)
                        )
                        drawContext.canvas.nativeCanvas.drawText(
                            label, cx, pillTop + pillH / 2f +
readoutPaint.textSize / 3f, readoutPaint
                        )
                    }
                }
            }
        }
    }
}

// ─── HOURLY & DAILY ITEMS ──────────────────────────────────────

@Composable
fun HourlyItem(
    hour: String,
    temp: String,
    weatherCode: Int,
    precipChance: Int = 0,
    isDay: Boolean = true
) {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(6.dp),
        modifier = Modifier.padding(horizontal = 4.dp)
    ) {
        Text(hour, fontSize = 13.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant, fontWeight = FontWeight.W600)
        WeatherIcon(weatherCode, modifier = Modifier.size(24.dp), isDay =
isDay)
        Text(temp, fontSize = 14.sp, color =
MaterialTheme.colorScheme.onSurface, fontWeight = FontWeight.W600)
        if (precipChance > 0) {
            Text(
                text = "$precipChance%",
```

```
                fontSize = 10.sp,
                color = MaterialTheme.colorScheme.primary,
                fontWeight = FontWeight.W700
            )
        }
    }
}

/** Maps a Celsius temperature to a cool→warm colour (icy blue → hot red).
*/
private fun tempColor(c: Float): Color = when {
    c <= 0f  -> Color(0xFF93C5FD)   // icy light blue
    c <= 8f  -> Color(0xFF60A5FA)   // cool blue
    c <= 16f -> Color(0xFF38BDF8)   // sky
    c <= 22f -> Color(0xFF34D399)   // mild green
    c <= 28f -> Color(0xFFFBBF24)   // warm amber
    c <= 34f -> Color(0xFFFB923C)   // hot orange
    else     -> Color(0xFFEF4444)   // very hot red
}

@Composable
fun DailyItem(
    day: String,
    weatherCode: Int,
    tempMin: String,
    tempMax: String,
    tempMinC: Float,
    tempMaxC: Float,
    weekMinC: Float,
    weekMaxC: Float,
    uvIndex: Float = 0f,
    precipChance: Int = 0
) {
    Column(modifier = Modifier.fillMaxWidth()) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(
                text = day,
                fontSize = 14.sp,
                fontWeight = FontWeight.W600,
                color = MaterialTheme.colorScheme.onSurface,
                modifier = Modifier.width(60.dp)
            )

            Box(modifier = Modifier.width(30.dp), contentAlignment =
Alignment.Center) {
                WeatherIcon(weatherCode, modifier = Modifier.size(22.dp))
            }

            // Precip chance
            if (precipChance > 0) {
                Text(
                    text = "$precipChance%",
                    fontSize = 11.sp,
                    color = MaterialTheme.colorScheme.primary,
                    fontWeight = FontWeight.W700,
                    modifier = Modifier.width(34.dp),
                    textAlign = TextAlign.Center
                )
            } else {
                Spacer(modifier = Modifier.width(34.dp))
            }

            Text(
                text = tempMin,
```

```
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                fontWeight = FontWeight.W600,
                modifier = Modifier.width(32.dp),
                textAlign = TextAlign.End
            )

            // Realistic range bar: positioned within the week's min..max,
            // filled with a cool→warm gradient (light/cool on the low side,
            // bright/warm on the high side).
            val weekRange = (weekMaxC - weekMinC).coerceAtLeast(1f)
            val startFrac = ((tempMinC - weekMinC) / weekRange).coerceIn(0f,
1f)
            val endFrac = ((tempMaxC - weekMinC) / weekRange).coerceIn(0f,
1f)
            BoxWithConstraints(
                modifier = Modifier
                    .width(80.dp)
                    .height(6.dp)
                    .clip(RoundedCornerShape(3.dp))
                    .background(MaterialTheme.colorScheme.surfaceVariant)
            ) {
                val trackW = maxWidth
                Box(
                    modifier = Modifier
                        .offset(x = trackW * startFrac)
                        .width((trackW * (endFrac -
startFrac)).coerceAtLeast(8.dp))
                        .fillMaxHeight()
                        .clip(RoundedCornerShape(3.dp))
                        .background(
                            Brush.horizontalGradient(
                                listOf(tempColor(tempMinC),
tempColor(tempMaxC))
                            )
                        )
                )
            }

            Text(
                text = tempMax,
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurface,
                fontWeight = FontWeight.W600,
                modifier = Modifier.width(32.dp),
                textAlign = TextAlign.End
            )
        }

        // UV label below if noteworthy
        if (uvIndex >= 3f) {
            Text(
                text = "UV ${if (uvIndex % 1 == 0f)
uvIndex.toInt().toString() else "%.1f".format(uvIndex)} · $
{uvIndex.getUvDescription()}",
                fontSize = 10.sp,
                color = when {
                    uvIndex >= 8f -> Color(0xFFF97316).copy(alpha = 0.90f)
                    uvIndex >= 6f -> Color(0xFFFBBF24).copy(alpha = 0.80f)
                    else          ->
MaterialTheme.colorScheme.onSurfaceVariant
                },
                fontWeight = FontWeight.W600,
                modifier = Modifier.padding(start = 68.dp, top = 2.dp)
            )
        }
    }
}
```

```
// ─── WEATHER BACKGROUND EFFECTS ──────────────────────────────────

// ─── WEATHER BACKGROUND (static tint — no animation overhead) ────────
@Composable
fun WeatherBackgroundEffect(weatherCode: Int, modifier: Modifier = Modifier)
{
    // Static per-condition tint instead of animated particles.
    // This eliminates the always-running infinite transition that was
    // re-drawing every frame even when the screen was idle.
    val tint = when (weatherCode) {
        0, 1            -> Color(0xFFFFD700).copy(alpha = 0.04f)   //
clear / sunny
        2, 3            -> Color(0xFFB0C4DE).copy(alpha = 0.04f)   // cloudy
        45, 48          -> Color(0xFF9E9E9E).copy(alpha = 0.04f)   // fog
        51, 53, 55,
        61, 63, 65,
        80, 81, 82      -> Color(0xFF4FC3F7).copy(alpha = 0.05f)   // rain
        71, 73, 75,
        77, 85, 86      -> Color(0xFFE1F5FE).copy(alpha = 0.06f)   // snow
        95, 96, 99      -> Color(0xFFF57F17).copy(alpha = 0.05f)   // storm
        else            -> Color.Transparent
    }
    androidx.compose.foundation.layout.Box(
        modifier = modifier.background(tint)
    )
}
```

**File: ui/LiquidGlassComponents.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.aiweathermonitor.GeocodingResult
import com.example.aiweathermonitor.theme.Elevation
import com.example.aiweathermonitor.theme.Radii
import com.example.aiweathermonitor.theme.Spacing

// ─── COMPOSITION LOCAL
──────────────────────────────────────────────────────

val LocalWeatherCode = compositionLocalOf { 0 }

// ─── WEATHER MODE (kept for source compatibility)
──────────────────────────

enum class OrganicWeatherMode { SUNNY, RAINY, SNOWY, STORMY, NEUTRAL }

fun getWeatherMode(code: Int): OrganicWeatherMode = when (code) {
    0, 1, 2, 3 -> OrganicWeatherMode.SUNNY
    45, 48, 51, 53, 55, 56, 57, 61, 63, 65, 66, 67, 80, 81, 82 ->
OrganicWeatherMode.RAINY
    71, 73, 75, 77, 85, 86 -> OrganicWeatherMode.SNOWY
    95, 96, 99 -> OrganicWeatherMode.STORMY
    else -> OrganicWeatherMode.NEUTRAL
}

/** No-op — removes all per-card infinite-transition overhead. */
@Composable
fun Modifier.organicMotion(mode: OrganicWeatherMode): Modifier = this

@Composable
fun glassRefractionBorder(): Brush = Brush.linearGradient(
    colors = listOf(
        MaterialTheme.colorScheme.outline.copy(alpha = 0.4f),
        MaterialTheme.colorScheme.outline.copy(alpha = 0.12f)
    )
)

fun glassSubtleBorder(): Brush = Brush.linearGradient(
    colors = listOf(Color.White.copy(alpha = 0.18f), Color.White.copy(alpha
= 0.06f))
)

// ─── LIQUID GLASS CARD
──────────────────────────────────────────────────────

@Composable
fun LiquidGlassCard(
```

```
    title: String = "",
    modifier: Modifier = Modifier,
    cornerRadius: Dp = Radii.card,
    glassAlpha: Float = 0.08f,      // kept for API compat
    showShimmer: Boolean = true,     // kept for API compat
    content: @Composable () -> Unit
) {
    ElevatedCard(
        shape = RoundedCornerShape(cornerRadius),
        colors = CardDefaults.elevatedCardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        elevation = CardDefaults.elevatedCardElevation(defaultElevation =
Elevation.card),
        modifier = modifier.fillMaxWidth()
    ) {
        Column(modifier = Modifier.padding(horizontal =
Spacing.cardPaddingH, vertical = Spacing.cardPaddingV)) {
            if (title.isNotBlank()) {
                Text(
                    text = title.uppercase(),
                    style = MaterialTheme.typography.titleSmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
                Spacer(modifier = Modifier.height(Spacing.small))
            }
            content()
        }
    }
}

// ——— LIQUID GLASS METRIC CARD
————————————————————————————————————

@Composable
fun LiquidGlassMetricCard(
    title: String,
    value: String,
    desc: String,
    modifier: Modifier = Modifier,
    isDark: Boolean = false,         // kept for API compat
    content: @Composable (ColumnScope.() -> Unit)? = null
) {
    var showDetail by remember { mutableStateOf(false) }

    ElevatedCard(
        onClick = { showDetail = true },
        shape = RoundedCornerShape(Radii.card),
        colors = CardDefaults.elevatedCardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        elevation = CardDefaults.elevatedCardElevation(defaultElevation =
Elevation.card),
        modifier = modifier
            .fillMaxWidth()
            .heightIn(min = 140.dp)
    ) {
        Column(
            modifier = Modifier
                .padding(horizontal = Spacing.cardPaddingH, vertical =
Spacing.cardPaddingV)
                .fillMaxWidth(),
            verticalArrangement = Arrangement.spacedBy(Spacing.tiny)
        ) {
            Text(
                text = title.uppercase(),
                style = MaterialTheme.typography.titleSmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
```

```
                maxLines = 1,
                overflow = TextOverflow.Ellipsis
            )
            // Capped to one line so a long value can't blow up the card
height —
            // the full text is available in the tap-to-open detail dialog.
            Text(
                text = value,
                style = MaterialTheme.typography.titleLarge,
                color = MaterialTheme.colorScheme.onSurface,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis
            )
            if (content != null) {
                Column(modifier = Modifier.fillMaxWidth()) { content() }
            }
            Text(
                text = desc,
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                maxLines = 2,
                overflow = TextOverflow.Ellipsis
            )
        }
    }

    if (showDetail) {
        AlertDialog(
            onDismissRequest = { showDetail = false },
            title = {
                Text(
                    text = title,
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.W600,
                    color = MaterialTheme.colorScheme.onSurface
                )
            },
            text = {
                Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text(
                        text = value,
                        style = MaterialTheme.typography.headlineSmall,
                        color = MaterialTheme.colorScheme.onSurface
                    )
                    if (desc.isNotBlank()) {
                        Text(
                            text = desc,
                            style = MaterialTheme.typography.bodyMedium,
                            color =
MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            },
            confirmButton = {
                TextButton(onClick = { showDetail = false }) { Text("Close")
}
            },
            shape = RoundedCornerShape(Radii.card),
            containerColor = MaterialTheme.colorScheme.surface
        )
    }
}

// ——— TOOLBAR BUTTON
————————————————————————————————————————————

@Composable
```

```
fun LiquidGlassToolbarButton(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    FilledIconButton(
        onClick = onClick,
        colors = IconButtonDefaults.filledIconButtonColors(
            containerColor =
MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.90f),
            contentColor = MaterialTheme.colorScheme.onSurfaceVariant
        ),
        modifier = modifier.size(42.dp)
    ) { content() }
}

// ——— BUTTON
————————————————————————————————————————————————————————

@Composable
fun LiquidGlassButton(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    content: @Composable RowScope.() -> Unit
) {
    FilledTonalButton(
        onClick = onClick,
        shape = RoundedCornerShape(Radii.pill),
        modifier = modifier.fillMaxWidth(),
        content = content
    )
}

// ——— TAB ROW
————————————————————————————————————————————————————————

@Composable
fun LiquidGlassTabRow(
    selectedTabIndex: Int,
    tabTitles: List<String>,
    onTabSelected: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    SecondaryTabRow(
        selectedTabIndex = selectedTabIndex,
        containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha
= 0.85f),
        contentColor = MaterialTheme.colorScheme.primary,
        modifier = modifier
            .fillMaxWidth()
            .clip(RoundedCornerShape(Radii.pill))
    ) {
        tabTitles.forEachIndexed { index, title ->
            Tab(
                selected = selectedTabIndex == index,
                onClick = { onTabSelected(index) },
                text = {
                    Text(
                        text = title,
                        fontSize = 13.sp,
                        fontWeight = if (selectedTabIndex == index)
FontWeight.W700 else FontWeight.W500
                    )
                }
            )
        }
    }
}
```

```
// ——— TEXT FIELD ——————————————————————————————————————————

@Composable
fun LiquidGlassTextField(
    value: String,
    onValueChange: (String) -> Unit,
    modifier: Modifier = Modifier,
    placeholder: @Composable (() -> Unit)? = null,
    label: @Composable (() -> Unit)? = null,
    singleLine: Boolean = true,
    visualTransformation: VisualTransformation = VisualTransformation.None,
    trailingIcon: @Composable (() -> Unit)? = null,
    leadingIcon: @Composable (() -> Unit)? = null,
    textStyle: androidx.compose.ui.text.TextStyle =
androidx.compose.ui.text.TextStyle.Default
) {
    OutlinedTextField(
        value = value,
        onValueChange = onValueChange,
        placeholder = placeholder,
        label = label,
        singleLine = singleLine,
        visualTransformation = visualTransformation,
        trailingIcon = trailingIcon,
        leadingIcon = leadingIcon,
        shape = RoundedCornerShape(Radii.pill),
        textStyle = textStyle,
        colors = OutlinedTextFieldDefaults.colors(
            focusedBorderColor = MaterialTheme.colorScheme.primary,
            unfocusedBorderColor =
MaterialTheme.colorScheme.outline.copy(alpha = 0.45f)
        ),
        modifier = modifier.fillMaxWidth()
    )
}

// ——— SEARCH BAR ——————————————————————————————————————————

@Composable
fun LiquidGlassSearchBar(
    searchQuery: String,
    onQueryChange: (String) -> Unit,
    modifier: Modifier = Modifier,
    leadingIcon: @Composable (() -> Unit)? = null
) {
    LiquidGlassTextField(
        value = searchQuery,
        onValueChange = onQueryChange,
        placeholder = {
            Text(
                text = "Search city (e.g. Paris, Tokyo)",
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurface.copy(alpha =
0.45f)
            )
        },
        leadingIcon = leadingIcon,
        modifier = modifier.fillMaxWidth()
    )
}

// ——— SEARCH RESULTS DROPDOWN ————————————————————————————

@Composable
```

```
fun LiquidGlassSearchResults(
    searchResults: List<GeocodingResult>,
    onSelectCity: (GeocodingResult) -> Unit,
    modifier: Modifier = Modifier
) {
    if (searchResults.isEmpty()) return

    ElevatedCard(
        shape = RoundedCornerShape(Radii.pill),
        elevation = CardDefaults.elevatedCardElevation(defaultElevation =
Elevation.raised),
        colors = CardDefaults.elevatedCardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        modifier = modifier.fillMaxWidth()
    ) {
        LazyColumn(modifier = Modifier.heightIn(max = 240.dp)) {
            items(searchResults) { result ->
                ListItem(
                    headlineContent = {
                        Text(result.name, fontSize = 14.sp, fontWeight =
FontWeight.W500)
                    },
                    supportingContent = {
                        val parts = listOfNotNull(result.admin1,
result.country)
                        if (parts.isNotEmpty()) {
                            Text(
                                text = parts.joinToString(", "),
                                fontSize = 12.sp,
                                color =
MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.65f)
                            )
                        }
                    },
                    modifier = Modifier.clickable { onSelectCity(result) }
                )
                if (result != searchResults.last()) {
                    HorizontalDivider(
                        thickness = 0.5.dp,
                        color =
MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.4f)
                    )
                }
            }
        }
    }
}

// ——— ALERT CARD
————————————————————————————————————————————

@Composable
fun LiquidGlassAlertCard(
    modifier: Modifier = Modifier,
    content: @Composable ColumnScope.() -> Unit
) {
    Card(
        shape = RoundedCornerShape(Radii.card),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.errorContainer
        ),
        modifier = modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier.padding(horizontal = Spacing.cardPaddingH,
vertical = Spacing.cardPaddingV),
            verticalArrangement = Arrangement.spacedBy(Spacing.tiny),
```

```
            content = content
        )
    }
}

// ——— DIALOG ——————————————————————————————————————————————————————————

@Composable
fun LiquidGlassDialog(
    onDismissRequest: () -> Unit,
    title: @Composable (() -> Unit)? = null,
    text: @Composable (() -> Unit)? = null,
    confirmButton: @Composable (() -> Unit)? = null,
    dismissButton: @Composable (() -> Unit)? = null
) {
    AlertDialog(
        onDismissRequest = onDismissRequest,
        title = title,
        text = text,
        confirmButton = { confirmButton?.invoke() },
        dismissButton = { dismissButton?.invoke() },
        shape = RoundedCornerShape(24.dp),
        containerColor = MaterialTheme.colorScheme.surface,
        tonalElevation = 6.dp
    )
}
```

**File: ui/OfflineBanner.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.animation.*
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.WifiOff
import androidx.compose.material.icons.outlined.AccessTime
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

/**
 * Shows a banner when:
 *  - device is offline (always shown), OR
 *  - data is stale (lastRefreshedEpoch > 0 and age > 1 hour)
 *
 * Disappears when online and data is fresh.
 */
@Composable
fun LiquidGlassOfflineBanner(
    isOnline: Boolean,
    lastRefreshedTime: String,
    lastRefreshedEpoch: Long = 0L,
    modifier: Modifier = Modifier
) {
    val staleThresholdMs = 60 * 60 * 1000L  // 1 hour
    val isStale = lastRefreshedEpoch > 0L &&
            (System.currentTimeMillis() - lastRefreshedEpoch) >
staleThresholdMs

    val shouldShow = !isOnline || isStale

    AnimatedVisibility(
        visible = shouldShow,
        enter = expandVertically() + fadeIn(),
        exit = shrinkVertically() + fadeOut(),
        modifier = modifier
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(8.dp),
            modifier = Modifier
                .fillMaxWidth()
                .clip(RoundedCornerShape(12.dp))
                .background(
                    if (!isOnline) MaterialTheme.colorScheme.errorContainer
                    else MaterialTheme.colorScheme.secondaryContainer
                )
                .padding(horizontal = 14.dp, vertical = 10.dp)
        ) {
            val onBanner = if (!isOnline)
MaterialTheme.colorScheme.onErrorContainer
                           else
MaterialTheme.colorScheme.onSecondaryContainer
            Icon(
                imageVector = if (!isOnline) Icons.Outlined.WifiOff else
Icons.Outlined.AccessTime,
```

```
                contentDescription = null,
                tint = onBanner,
                modifier = Modifier.size(16.dp)
            )
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = if (!isOnline) "No internet connection" else
"Weather data may be outdated",
                    fontSize = 13.sp,
                    fontWeight = FontWeight.W500,
                    color = onBanner
                )
                if (lastRefreshedTime.isNotBlank()) {
                    Text(
                        text = "Last updated $lastRefreshedTime",
                        fontSize = 11.sp,
                        color = onBanner
                    )
                }
            }
        }
    }
}
```

**File: ui/OnboardingScreen.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.animation.*
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

// ── Pages ──────────────────────────────────────────────────────────────

private sealed class OnboardingPage {
    data object Welcome  : OnboardingPage()
    data object Features : OnboardingPage()
    data object Location : OnboardingPage()
}

private val pages = listOf(
    OnboardingPage.Welcome,
    OnboardingPage.Features,
    OnboardingPage.Location
)

// ── Entry composable ──────────────────────────────────────────────────

@Composable
fun OnboardingScreen(
    onRequestLocation: () -> Unit,
    onSkipToSearch: () -> Unit,
    modifier: Modifier = Modifier
) {
    var currentPage by remember { mutableIntStateOf(0) }

    Surface(modifier = modifier.fillMaxSize(), color =
MaterialTheme.colorScheme.background) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .systemBarsPadding()
                .padding(horizontal = 28.dp)
        ) {
            // Step dots
            Spacer(Modifier.height(24.dp))
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically
            ) {
                pages.forEachIndexed { idx, _ ->
                    StepDot(active = idx == currentPage)
                    if (idx < pages.lastIndex) Spacer(Modifier.width(6.dp))
                }
            }
```

```
            Spacer(Modifier.height(8.dp))

            // Page content
            AnimatedContent(
                targetState = currentPage,
                transitionSpec = {
                    if (targetState > initialState)
                        slideInHorizontally { it } + fadeIn() togetherWith
                        slideOutHorizontally { -it } + fadeOut()
                    else
                        slideInHorizontally { -it } + fadeIn() togetherWith
                        slideOutHorizontally { it } + fadeOut()
                },
                modifier = Modifier.weight(1f),
                label = "onboarding_page"
            ) { page ->
                when (pages[page]) {
                    OnboardingPage.Welcome  -> WelcomePage()
                    OnboardingPage.Features -> FeaturesPage()
                    OnboardingPage.Location -> LocationPage()
                }
            }

            // Actions
            when (currentPage) {
                0 -> {
                    PrimaryButton("Get started") { currentPage = 1 }
                    Spacer(Modifier.height(12.dp))
                    Text(
                        "Powered by Open-Meteo · WeatherAPI.com",
                        fontSize = 12.sp,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        modifier = Modifier.fillMaxWidth(),
                        textAlign = TextAlign.Center
                    )
                }
                1 -> PrimaryButton("Continue") { currentPage = 2 }
                2 -> {
                    PrimaryButton("Allow location") { onRequestLocation() }
                    Spacer(Modifier.height(10.dp))
                    OutlinedButton(
                        onClick = onSkipToSearch,
                        modifier = Modifier.fillMaxWidth(),
                        shape = RoundedCornerShape(14.dp)
                    ) { Text("Search city manually", fontSize = 14.sp) }
                    Spacer(Modifier.height(12.dp))
                    Text(
                        "You can change this later in Settings",
                        fontSize = 12.sp,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        modifier = Modifier.fillMaxWidth(),
                        textAlign = TextAlign.Center
                    )
                }
            }
            Spacer(Modifier.height(32.dp))
        }
    }
}

// ── Page composables ──────────────────────────────────────────

@Composable
private fun WelcomePage() {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
```

```
            verticalArrangement = Arrangement.Center
        ) {
            HeroIconBox(Icons.Outlined.WbCloudy)
            Spacer(Modifier.height(28.dp))
            Text(
                "Your weather,\nbeautifully clear",
                fontSize = 26.sp,
                fontWeight = FontWeight.Medium,
                color = MaterialTheme.colorScheme.onBackground,
                textAlign = TextAlign.Center,
                lineHeight = 32.sp
            )
            Spacer(Modifier.height(12.dp))
            Text(
                "Real-time conditions, hourly and 14-day forecasts, air quality,
UV index, and alerts — all in one place.",
                fontSize = 15.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                textAlign = TextAlign.Center,
                lineHeight = 22.sp
            )
        }
    }

    @Composable
    private fun FeaturesPage() {
        Column(modifier = Modifier.fillMaxSize()) {
            Spacer(Modifier.height(12.dp))
            Text(
                "Everything you need to know",
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = MaterialTheme.colorScheme.onBackground
            )
            Text(
                "All conditions, no subscriptions.",
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.padding(top = 6.dp, bottom = 20.dp)
            )
            val features = listOf(
                Triple(Icons.Outlined.LightMode,   "UV & AQI",           "Hourly
UV index and EPA air quality with health guidance"),
                Triple(Icons.Outlined.CalendarMonth,"14-day forecast",    "Daily
highs, lows, precip chance, and UV per day"),
                Triple(Icons.Outlined.Warning,      "Severe alerts",      "Push
notifications for storms, heat advisories, and heavy rain"),
                Triple(Icons.Outlined.Refresh,      "Background refresh",
"Conditions update every 30 min, even when the app is closed"),
            )
            features.forEach { (icon, title, sub) ->
                FeatureRow(icon = icon, title = title, subtitle = sub)
                Spacer(Modifier.height(14.dp))
            }
        }
    }

    @Composable
    private fun LocationPage() {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Spacer(Modifier.height(12.dp))
            HeroIconBox(Icons.Outlined.LocationOn)
            Spacer(Modifier.height(20.dp))
            Text(
                "Allow location access",
```

```
            fontSize = 24.sp,
            fontWeight = FontWeight.Medium,
            color = MaterialTheme.colorScheme.onBackground,
            textAlign = TextAlign.Center
        )
        Spacer(Modifier.height(10.dp))
        Text(
            "To show your local weather automatically.\nYou can also search
any city manually.",
            fontSize = 15.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            textAlign = TextAlign.Center,
            lineHeight = 22.sp
        )
        Spacer(Modifier.height(20.dp))
        // Permission card
        Surface(
            shape = RoundedCornerShape(20.dp),
            color = MaterialTheme.colorScheme.surfaceVariant,
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(Modifier.padding(18.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(14.dp)) {
                    Surface(
                        shape = RoundedCornerShape(14.dp),
                        color = MaterialTheme.colorScheme.surface,
                        border =
androidx.compose.foundation.BorderStroke(0.5.dp,
MaterialTheme.colorScheme.outlineVariant),
                        modifier = Modifier.size(48.dp)
                    ) {
                        Box(contentAlignment = Alignment.Center) {
                            Icon(Icons.Outlined.LocationOn,
contentDescription = null, modifier = Modifier.size(24.dp))
                        }
                    }
                    Column {
                        Text("Location", fontSize = 15.sp, fontWeight =
FontWeight.Medium)
                        Text("While using the app", fontSize = 13.sp, color
= MaterialTheme.colorScheme.onSurfaceVariant)
                    }
                }
                Spacer(Modifier.height(14.dp))
                PermissionBullet(icon = Icons.Outlined.Check, positive =
true,  text = "Automatic weather for your current city")
                Spacer(Modifier.height(8.dp))
                PermissionBullet(icon = Icons.Outlined.Check, positive =
true,  text = "Never shared or stored on any server")
                Spacer(Modifier.height(8.dp))
                PermissionBullet(icon = Icons.Outlined.Close, positive =
false, text = "Not used in the background for tracking")
            }
        }
        Spacer(Modifier.weight(1f))
    }
}

// ── Small components
─────────────────────────────────────────────────────────


@Composable
private fun StepDot(active: Boolean) {
    Box(
        modifier = Modifier
            .height(6.dp)
            .clip(CircleShape)
```

```
            .background(
                if (active) MaterialTheme.colorScheme.onBackground
                else MaterialTheme.colorScheme.outlineVariant
            )
            .animateContentSize()
            .width(if (active) 20.dp else 6.dp)
    )
}

@Composable
private fun HeroIconBox(icon: ImageVector) {
    Surface(
        shape = RoundedCornerShape(28.dp),
        color = MaterialTheme.colorScheme.surfaceVariant,
        modifier = Modifier.size(88.dp)
    ) {
        Box(contentAlignment = Alignment.Center) {
            Icon(icon, contentDescription = null, modifier =
Modifier.size(44.dp))
        }
    }
}

@Composable
private fun FeatureRow(icon: ImageVector, title: String, subtitle: String) {
    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        Surface(
            shape = RoundedCornerShape(10.dp),
            color = MaterialTheme.colorScheme.surfaceVariant,
            modifier = Modifier.size(36.dp)
        ) {
            Box(contentAlignment = Alignment.Center) {
                Icon(icon, contentDescription = null, modifier =
Modifier.size(20.dp))
            }
        }
        Column {
            Text(title, fontSize = 14.sp, fontWeight = FontWeight.Medium)
            Text(subtitle, fontSize = 13.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant, lineHeight = 18.sp)
        }
    }
}

@Composable
private fun PermissionBullet(icon: ImageVector, positive: Boolean, text:
String) {
    Row(horizontalArrangement = Arrangement.spacedBy(8.dp),
verticalAlignment = Alignment.CenterVertically) {
        Icon(
            icon,
            contentDescription = null,
            modifier = Modifier.size(14.dp),
            tint = if (positive) MaterialTheme.colorScheme.primary else
MaterialTheme.colorScheme.onSurfaceVariant
        )
        Text(text, fontSize = 13.sp, color =
MaterialTheme.colorScheme.onSurfaceVariant)
    }
}

@Composable
private fun PrimaryButton(text: String, onClick: () -> Unit) {
    Button(
        onClick = onClick,
        modifier = Modifier.fillMaxWidth().height(52.dp),
        shape = RoundedCornerShape(14.dp)
    ) {
```

```
        Text(text, fontSize = 15.sp, fontWeight = FontWeight.Medium)
    }
}
```

**File: ui/SettingsScreen.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.aiweathermonitor.WeatherState
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.*
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.*
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.asComposeRenderEffect
import androidx.compose.ui.text.font.FontWeight

// ── Enums ──────────────────────────────────────────────────────────────

enum class WindUnit(val label: String) { KMH("km/h"), MPH("mph"),
MS("m/s") }
enum class PressureUnit(val label: String) { HPA("hPa"), INHG("inHg") }
enum class VisibilityUnit(val label: String) { KM("km"), MI("mi") }
enum class AppTheme(val label: String) { SYSTEM("System"), LIGHT("Light"),
DARK("Dark") }

// ── Settings state ─────────────────────────────────────────────────────

data class AppSettings(
    val windUnit: WindUnit = WindUnit.KMH,
    val pressureUnit: PressureUnit = PressureUnit.HPA,
    val visibilityUnit: VisibilityUnit = VisibilityUnit.KM,
    val theme: AppTheme = AppTheme.SYSTEM,
    val notifySevere: Boolean = true,
    val notifyRain: Boolean = true,
    val notifyDailyBriefing: Boolean = false,
    val notifyHighUv: Boolean = false,
    val backgroundRefresh: Boolean = true,
    val soundEnabled: Boolean = false
)

// ── Main composable ────────────────────────────────────────────────────

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SettingsScreen(
    state: WeatherState,
    settings: AppSettings,
```

```
    onSettingsChange: (AppSettings) -> Unit,
    onSaveWeatherApiKey: (String) -> Unit,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val contentColor = MaterialTheme.colorScheme.onSurface
    val dividerColor = MaterialTheme.colorScheme.outline.copy(alpha = 0.15f)

    var showPrivacy by remember { mutableStateOf(false) }
    var showLicenses by remember { mutableStateOf(false) }

    Box(
        modifier = modifier
            .fillMaxSize()
            .background(MaterialTheme.colorScheme.background)
    ) {
        Scaffold(
            topBar = {
                TopAppBar(
                    title = { Text("Settings", fontSize = 17.sp, color =
contentColor, fontWeight = FontWeight.Bold) },
                    navigationIcon = {
                        IconButton(onClick = onNavigateBack) {
                            Icon(
                                imageVector = settingsBackIcon(),
                                contentDescription = "Back",
                                tint = contentColor
                            )
                        }
                    },
                    colors = TopAppBarDefaults.topAppBarColors(
                        containerColor =
MaterialTheme.colorScheme.background,
                        scrolledContainerColor =
MaterialTheme.colorScheme.background,
                        titleContentColor = contentColor,
                        navigationIconContentColor = contentColor
                    )
                )
            },
            containerColor = Color.Transparent,
            modifier = Modifier.fillMaxSize()
        ) { innerPadding ->
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .verticalScroll(rememberScrollState())
                    .padding(innerPadding)
                    .padding(horizontal = 16.dp),
                verticalArrangement = Arrangement.spacedBy(4.dp)
            ) {
            Spacer(Modifier.height(8.dp))

            // ── Units
──────────────────────────────────────────────
            SettingsGroupLabel("Units")
            SettingsCard {
                SegmentedRow(
                    icon = icons.temperature,
                    iconTint = MaterialTheme.colorScheme.tertiary,
                    label = "Temperature",
                    options = listOf("°C", "°F"),
                    selected = if (state.isCelsius) 0 else 1,
                    onSelect = { /* isCelsius toggled via WeatherState — see
caller */ }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                SegmentedRow(
```

```
                    icon = icons.wind,
                    iconTint = MaterialTheme.colorScheme.primary,
                    label = "Wind speed",
                    options = WindUnit.entries.map { it.label },
                    selected = settings.windUnit.ordinal,
                    onSelect = { onSettingsChange(settings.copy(windUnit =
WindUnit.entries[it])) }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                SegmentedRow(
                    icon = icons.pressure,
                    iconTint = MaterialTheme.colorScheme.secondary,
                    label = "Pressure",
                    options = PressureUnit.entries.map { it.label },
                    selected = settings.pressureUnit.ordinal,
                    onSelect = { onSettingsChange(settings.copy(pressureUnit
= PressureUnit.entries[it])) }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                SegmentedRow(
                    icon = icons.visibility,
                    iconTint = MaterialTheme.colorScheme.secondary,
                    label = "Visibility",
                    options = VisibilityUnit.entries.map { it.label },
                    selected = settings.visibilityUnit.ordinal,
                    onSelect =
{ onSettingsChange(settings.copy(visibilityUnit =
VisibilityUnit.entries[it])) }
                )
            }

            Spacer(Modifier.height(8.dp))

            // ── Appearance
────────────────────────────────────────────
            SettingsGroupLabel("Appearance")
            SettingsCard {
                SegmentedRow(
                    icon = icons.theme,
                    iconTint = MaterialTheme.colorScheme.secondary,
                    label = "Theme",
                    options = AppTheme.entries.map { it.label },
                    selected = settings.theme.ordinal,
                    onSelect = { onSettingsChange(settings.copy(theme =
AppTheme.entries[it])) }
                )
            }

            Spacer(Modifier.height(8.dp))

            // ── Sound
──────────────────────────────────────────────────
            SettingsGroupLabel("Sound")
            SettingsCard {
                ToggleRow(
                    icon = icons.sound,
                    iconBgColor =
MaterialTheme.colorScheme.primaryContainer,
                    iconTint = MaterialTheme.colorScheme.primary,
                    label = "Weather ambience",
                    subtitle = "Play background sound matching the weather",
                    checked = settings.soundEnabled,
                    onCheckedChange =
{ onSettingsChange(settings.copy(soundEnabled = it)) }
                )
            }

            Spacer(Modifier.height(8.dp))
```

```
            // ── Notifications
────────────────────────────────────────────────
            SettingsGroupLabel("Notifications")
            SettingsCard {
                ToggleRow(
                    icon = icons.alert,
                    iconBgColor = MaterialTheme.colorScheme.errorContainer,
                    iconTint = MaterialTheme.colorScheme.error,
                    label = "Severe weather alerts",
                    subtitle = "Storms, heat advisories, floods",
                    checked = settings.notifySevere,
                    onCheckedChange =
{ onSettingsChange(settings.copy(notifySevere = it)) }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                ToggleRow(
                    icon = icons.rain,
                    iconBgColor =
MaterialTheme.colorScheme.primaryContainer,
                    iconTint = MaterialTheme.colorScheme.primary,
                    label = "Rain alerts",
                    subtitle = "Notify 30 min before rain",
                    checked = settings.notifyRain,
                    onCheckedChange =
{ onSettingsChange(settings.copy(notifyRain = it)) }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                ToggleRow(
                    icon = icons.sun,
                    iconBgColor =
MaterialTheme.colorScheme.secondaryContainer,
                    iconTint = MaterialTheme.colorScheme.secondary,
                    label = "Daily briefing",
                    subtitle = "Morning summary at 7:00 AM",
                    checked = settings.notifyDailyBriefing,
                    onCheckedChange =
{ onSettingsChange(settings.copy(notifyDailyBriefing = it)) }
                )
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                ToggleRow(
                    icon = icons.uv,
                    iconBgColor =
MaterialTheme.colorScheme.tertiaryContainer,
                    iconTint = MaterialTheme.colorScheme.tertiary,
                    label = "High UV alert",
                    subtitle = "When UV index ≥ 8",
                    checked = settings.notifyHighUv,
                    onCheckedChange =
{ onSettingsChange(settings.copy(notifyHighUv = it)) }
                )
            }

            Spacer(Modifier.height(8.dp))

            // ── Data sources
──────────────────────────────────────────────
            SettingsGroupLabel("Data sources")
            SettingsCard {
                // Open-Meteo row (always active, no toggle)
                Row(
                    modifier = Modifier.padding(14.dp),
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.spacedBy(12.dp)
                ) {
                    RowIconBox(icon = icons.globe, iconTint = contentColor,
bgColor = MaterialTheme.colorScheme.secondary)
                    Column(Modifier.weight(1f)) {
```

```
                        Text("Open-Meteo", fontSize = 14.sp, color =
contentColor.copy(alpha = 0.90f))
                        Text("Always active · no key required", fontSize =
12.sp, color = contentColor.copy(alpha = 0.60f))
                    }
                    Surface(
                        color = MaterialTheme.colorScheme.primaryContainer,
                        shape = RoundedCornerShape(20.dp)
                    ) {
                        Text("Active", fontSize = 11.sp, color =
MaterialTheme.colorScheme.onPrimaryContainer, modifier =
Modifier.padding(horizontal = 10.dp, vertical = 3.dp))
                    }
                }
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                ToggleRow(
                    icon = icons.refresh,
                    iconBgColor = MaterialTheme.colorScheme.surfaceVariant,
                    iconTint = MaterialTheme.colorScheme.onSurfaceVariant,
                    label = "Background refresh",
                    subtitle = "Every 30 minutes",
                    checked = settings.backgroundRefresh,
                    onCheckedChange =
{ onSettingsChange(settings.copy(backgroundRefresh = it)) }
                )
            }

            Spacer(Modifier.height(8.dp))

            // ── About
──────────────────────────────────────────────────────
            SettingsGroupLabel("About")
            SettingsCard {
                NavRow(icon = icons.info, label = "Version", trailingText =
"1.0.0", onClick = {})
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                NavRow(icon = icons.privacy, label = "Privacy policy",
onClick = { showPrivacy = true })
                HorizontalDivider(thickness = 0.5.dp, color = dividerColor)
                NavRow(icon = icons.licenses, label = "Open source
licenses", onClick = { showLicenses = true })
            }

            Spacer(Modifier.height(8.dp))
            Text(
                "Weather Monitor · Open-Meteo + WeatherAPI.com",
                fontSize = 12.sp,
                color = contentColor.copy(alpha = 0.50f),
                modifier = Modifier.align(Alignment.CenterHorizontally)
            )
            Spacer(Modifier.height(32.dp))
        }
    }

        PrivacyPolicyDialog(visible = showPrivacy, onDismiss = { showPrivacy
= false })
        LicensesDialog(visible = showLicenses, onDismiss = { showLicenses =
false })
    }
}

// ── About dialogs
──────────────────────────────────────────────────────────

@Composable
private fun InfoDialog(
    visible: Boolean,
    title: String,
```

```
    onDismiss: () -> Unit,
    body: @Composable ColumnScope.() -> Unit
) {
    if (!visible) return
    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Text(
                text = title,
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.W600,
                color = MaterialTheme.colorScheme.onSurface
            )
        },
        text = {
            Column(
                modifier = Modifier
                    .heightIn(max = 440.dp)
                    .verticalScroll(rememberScrollState()),
                verticalArrangement = Arrangement.spacedBy(8.dp),
                content = body
            )
        },
        confirmButton = { TextButton(onClick = onDismiss)
{ Text("Close") } },
        shape = RoundedCornerShape(20.dp),
        containerColor = MaterialTheme.colorScheme.surface
    )
}

@Composable
private fun PolicyHeading(text: String) =
    Text(
        text = text,
        fontSize = 13.sp,
        fontWeight = FontWeight.W700,
        color = MaterialTheme.colorScheme.onSurface,
        modifier = Modifier.padding(top = 4.dp)
    )

@Composable
private fun PolicyBody(text: String) =
    Text(
        text = text,
        fontSize = 13.sp,
        lineHeight = 19.sp,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )

@Composable
private fun PrivacyPolicyDialog(visible: Boolean, onDismiss: () -> Unit) {
    InfoDialog(visible = visible, title = "Privacy Policy", onDismiss =
onDismiss) {
        PolicyBody("Last updated: June 2026")
        PolicyBody(
            "This app is built privacy-first. We do not collect, store, or
share any " +
                "personal information, and there are no accounts or sign-
ins."
        )
        PolicyHeading("Location")
        PolicyBody(
            "If you grant location permission, your coordinates are used
only on your " +
                "device to fetch local weather and are sent solely to the
weather provider " +
                "to return a forecast. Your location is never stored on a
server or shared " +
```

```
                "with third parties. You can also use the app by searching
for cities manually."
        )
        PolicyHeading("Data sources")
        PolicyBody(
            "Weather comes from Open-Meteo and (optionally) WeatherAPI.com.
News headlines " +
                "come from the Google News RSS feed. Requests contain only
the city or " +
                "coordinates needed to return results."
        )
        PolicyHeading("On-device storage")
        PolicyBody(
            "Your selected city, saved favourites, units, and last fetched
weather are cached " +
                "locally so the app works offline. This data never leaves
your device, and you " +
                "can clear it any time from the system app settings."
        )
        PolicyHeading("No tracking or ads")
        PolicyBody("The app contains no analytics, advertising, or third-
party trackers.")
        PolicyHeading("Permissions")
        PolicyBody(
            "Location (optional) — current-location weather.\n" +
                "Notifications (optional) — severe weather, rain, and UV
alerts."
        )
    }
}

@Composable
private fun LicensesDialog(visible: Boolean, onDismiss: () -> Unit) {
    InfoDialog(visible = visible, title = "Open-Source Licenses", onDismiss
= onDismiss) {
        PolicyBody("This app is built with these open-source projects:")
        val libraries = listOf(
            "Jetpack Compose & AndroidX" to "Apache License 2.0",
            "Material 3 & Material Icons" to "Apache License 2.0",
            "Kotlin & Coroutines" to "Apache License 2.0",
            "kotlinx.serialization" to "Apache License 2.0",
            "OkHttp" to "Apache License 2.0",
            "Koin (dependency injection)" to "Apache License 2.0",
            "Lottie for Android (Airbnb)" to "Apache License 2.0",
            "DataStore & WorkManager" to "Apache License 2.0",
            "Meteocons (Bas Milius)" to "MIT License"
        )
        libraries.forEach { (name, license) ->
            Column {
                Text(
                    text = name,
                    fontSize = 13.sp,
                    fontWeight = FontWeight.W600,
                    color = MaterialTheme.colorScheme.onSurface
                )
                Text(
                    text = license,
                    fontSize = 12.sp,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        }
        PolicyHeading("Data")
        PolicyBody(
            "Weather: Open-Meteo (CC BY 4.0) and WeatherAPI.com.\n" +
                "News: Google News RSS.\n" +
                "All trademarks belong to their respective owners."
        )
```

```
    }
}

// ── Sub-composables ──────────────────────────────────────────────

@Composable
private fun SettingsGroupLabel(text: String) {
    Text(
        text = text.uppercase(),
        fontSize = 11.sp,
        fontWeight = FontWeight.Bold,
        letterSpacing = 0.8.sp,
        color = MaterialTheme.colorScheme.onSurface.copy(alpha = 0.60f),
        modifier = Modifier.padding(start = 4.dp, top = 12.dp, bottom =
4.dp)
    )
}

@Composable
private fun SettingsCard(content: @Composable ColumnScope.() -> Unit) {
    ElevatedCard(
        shape =
RoundedCornerShape(com.example.aiweathermonitor.theme.Radii.card),
        colors = CardDefaults.elevatedCardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        elevation = CardDefaults.elevatedCardElevation(
            defaultElevation =
com.example.aiweathermonitor.theme.Elevation.card
        ),
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(content = content)
    }
}

@Composable
private fun RowIconBox(icon: ImageVector, iconTint:
androidx.compose.ui.graphics.Color, bgColor:
androidx.compose.ui.graphics.Color) {
    Box(
        modifier = Modifier
            .size(32.dp)
            .clip(RoundedCornerShape(8.dp))
            .background(bgColor.copy(alpha = 0.25f)),
        contentAlignment = Alignment.Center
    ) {
        Icon(icon, contentDescription = null, tint = iconTint, modifier =
Modifier.size(18.dp))
    }
}

@Composable
private fun SegmentedRow(
    icon: ImageVector,
    iconTint: androidx.compose.ui.graphics.Color,
    label: String,
    options: List<String>,
    selected: Int,
    onSelect: (Int) -> Unit
) {
    val contentColor = MaterialTheme.colorScheme.onSurface
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 14.dp, vertical = 10.dp),
        verticalAlignment = Alignment.CenterVertically,
```

```
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Icon(icon, contentDescription = null, tint = iconTint.copy(alpha =
0.85f), modifier = Modifier.size(18.dp))
        Text(label, fontSize = 14.sp, color = contentColor.copy(alpha =
0.90f), modifier = Modifier.weight(1f))
        Row(
            modifier = Modifier
                .clip(RoundedCornerShape(8.dp))
                .background(contentColor.copy(alpha = 0.06f))
                .padding(2.dp),
            horizontalArrangement = Arrangement.spacedBy(2.dp)
        ) {
            options.forEachIndexed { idx, opt ->
                val isActive = idx == selected
                Box(
                    modifier = Modifier
                        .clip(RoundedCornerShape(6.dp))
                        .background(
                            if (isActive) contentColor.copy(alpha = 0.18f)
                            else Color.Transparent
                        )
                        .clickable { onSelect(idx) }
                        .padding(horizontal = 10.dp, vertical = 5.dp),
                    contentAlignment = Alignment.Center
                ) {
                    Text(
                        opt,
                        fontSize = 12.sp,
                        color = if (isActive) contentColor else
contentColor.copy(alpha = 0.45f),
                        fontWeight = if (isActive) FontWeight.Bold else
FontWeight.Medium
                    )
                }
            }
        }
    }
}

@Composable
private fun ToggleRow(
    icon: ImageVector,
    iconBgColor: androidx.compose.ui.graphics.Color,
    iconTint: androidx.compose.ui.graphics.Color,
    label: String,
    subtitle: String,
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit
) {
    val contentColor = MaterialTheme.colorScheme.onSurface
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable { onCheckedChange(!checked) }
            .padding(horizontal = 14.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        RowIconBox(icon = icon, iconTint = iconTint, bgColor = iconBgColor)
        Column(Modifier.weight(1f)) {
            Text(label, fontSize = 14.sp, color = contentColor.copy(alpha =
0.90f))
            Text(subtitle, fontSize = 12.sp, color = contentColor.copy(alpha
= 0.60f))
        }
        Switch(
            checked = checked,
```

```
            onCheckedChange = onCheckedChange,
            colors = SwitchDefaults.colors(
                checkedThumbColor = MaterialTheme.colorScheme.onPrimary,
                checkedTrackColor = MaterialTheme.colorScheme.primary,
                uncheckedThumbColor = contentColor.copy(alpha = 0.6f),
                uncheckedTrackColor = contentColor.copy(alpha = 0.15f)
            )
        )
    }
}

@Composable
private fun NavRow(
    icon: ImageVector,
    label: String,
    trailingText: String? = null,
    onClick: () -> Unit
) {
    val contentColor = MaterialTheme.colorScheme.onSurface
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(horizontal = 14.dp, vertical = 13.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        RowIconBox(
            icon = icon,
            iconTint = MaterialTheme.colorScheme.primary,
            bgColor = MaterialTheme.colorScheme.primaryContainer
        )
        Text(
            text = label,
            fontSize = 14.sp,
            color = contentColor.copy(alpha = 0.90f),
            modifier = Modifier.weight(1f)
        )
        if (trailingText != null) {
            Text(
                text = trailingText,
                fontSize = 13.sp,
                color = contentColor.copy(alpha = 0.55f)
            )
        }
    }
}

// ── Icon set (outlined Material symbols) ─────────────────────

private object icons {
    val temperature = Icons.Outlined.Thermostat
    val wind        = Icons.Outlined.Air
    val pressure    = Icons.Outlined.Speed
    val visibility  = Icons.Outlined.Visibility
    val theme       = Icons.Outlined.DarkMode
    val alert       = Icons.Outlined.Warning
    val rain        = Icons.Outlined.WaterDrop
    val uv          = Icons.Outlined.WbSunny
    val sun         = Icons.Outlined.WbSunny
    val refresh     = Icons.Outlined.Refresh
    val sound       = Icons.Outlined.MusicNote
    val globe       = Icons.Outlined.Public
    val key         = Icons.Outlined.Key
    val info        = Icons.Outlined.Info
    val licenses    = Icons.Outlined.Description
    val privacy     = Icons.Outlined.Shield
```

```
}

private fun settingsBackIcon(): ImageVector = Icons.Outlined.ArrowBackIosNew
```

**File: ui/WeatherIcon.kt**

```
package com.example.aiweathermonitor.ui

import androidx.compose.animation.core.*
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.size
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.unit.dp
import kotlin.math.cos
import kotlin.math.sin
import androidx.annotation.RawRes
import com.airbnb.lottie.compose.LottieAnimation
import com.airbnb.lottie.compose.LottieCompositionSpec
import com.airbnb.lottie.compose.LottieConstants
import com.airbnb.lottie.compose.rememberLottieComposition
import com.example.aiweathermonitor.R

/**
 * Maps a WMO weather code to a bundled meteocon Lottie animation (res/raw).
 * Clear and partly-cloudy conditions use night variants when [isDay] is
false.
 */
@RawRes
fun getWeatherLottieRes(code: Int, isDay: Boolean = true): Int = when (code)
{
    0 -> if (isDay) R.raw.clear_day else R.raw.clear_night
    1, 2, 3 -> if (isDay) R.raw.partly_cloudy_day else
R.raw.partly_cloudy_night
    45, 48 -> R.raw.fog
    51, 53, 55, 61, 63, 65, 80, 81, 82 -> R.raw.rain
    71, 73, 75 -> R.raw.snow
    95, 96, 99 -> R.raw.thunderstorms
    else -> R.raw.cloudy
}

@Composable
fun WeatherIcon(code: Int, modifier: Modifier = Modifier, isDay: Boolean =
true) {
    val compositionResult = rememberLottieComposition(
        spec = LottieCompositionSpec.RawRes(getWeatherLottieRes(code,
isDay))
    )
    val composition = compositionResult.value

    if (composition != null && !compositionResult.isFailure) {
        LottieAnimation(
            composition = composition,
            iterations = LottieConstants.IterateForever,
            modifier = modifier
        )
    } else {
        val infiniteTransition = rememberInfiniteTransition(label =
"WeatherIconAnimation")

        // Sun Rotation Animation
        val sunRotationAngle by infiniteTransition.animateFloat(
            initialValue = 0f,
            targetValue = 360f,
```

```
            animationSpec = infiniteRepeatable(
                animation = tween(12000, easing = LinearEasing),
                repeatMode = RepeatMode.Restart
            ),
            label = "SunRotation"
        )

        // Cloud Hovering Animation
        val cloudOffset by infiniteTransition.animateFloat(
            initialValue = -2f,
            targetValue = 2f,
            animationSpec = infiniteRepeatable(
                animation = tween(2500, easing = FastOutSlowInEasing),
                repeatMode = RepeatMode.Reverse
            ),
            label = "CloudOffset"
        )

        // Rain Drop Fall Animation
        val rainDropY by infiniteTransition.animateFloat(
            initialValue = 0f,
            targetValue = 24f,
            animationSpec = infiniteRepeatable(
                animation = tween(1000, easing = LinearEasing),
                repeatMode = RepeatMode.Restart
            ),
            label = "RainDropY"
        )

        // Lightning Flash Animation
        val lightningAlpha by infiniteTransition.animateFloat(
            initialValue = 0.3f,
            targetValue = 1f,
            animationSpec = infiniteRepeatable(
                animation = tween(600, easing = FastOutLinearInEasing),
                repeatMode = RepeatMode.Reverse
            ),
            label = "LightningAlpha"
        )

        CanvasFallback(
            code = code,
            modifier = modifier,
            sunRotationAngle = sunRotationAngle,
            cloudOffset = cloudOffset,
            rainDropY = rainDropY,
            lightningAlpha = lightningAlpha
        )
    }
}

@Composable
fun CanvasFallback(
    code: Int,
    modifier: Modifier = Modifier,
    sunRotationAngle: Float,
    cloudOffset: Float,
    rainDropY: Float,
    lightningAlpha: Float
) {
    Canvas(modifier = modifier.size(48.dp)) {
        val w = size.width
        val h = size.height
        val center = Offset(w / 2f, h / 2f)

        when (code) {
            // 0: Clear Sky (Slowly rotating golden sun with soft outer
glow)
```

```
            0 -> {
                // Outer warm glow
                drawCircle(
                    brush = Brush.radialGradient(
                        colors = listOf(Color(0xFFFBBF24).copy(alpha =
0.35f), Color.Transparent),
                        center = center,
                        radius = w * 0.45f
                    ),
                    radius = w * 0.45f
                )
                // Main sun body
                drawCircle(
                    brush = Brush.radialGradient(
                        colors = listOf(Color(0xFFFDE047),
Color(0xFFF59E0B)),
                        center = center,
                        radius = w * 0.25f
                    ),
                    radius = w * 0.25f
                )
                // Sun rays
                val numRays = 8
                val rayLength = w * 0.12f
                val innerRadius = w * 0.29f
                for (i in 0 until numRays) {
                    val angleRad = Math.toRadians((sunRotationAngle + (i *
(360f / numRays))).toDouble())
                    val startX = center.x + innerRadius *
cos(angleRad).toFloat()
                    val startY = center.y + innerRadius *
sin(angleRad).toFloat()
                    val endX = center.x + (innerRadius + rayLength) *
cos(angleRad).toFloat()
                    val endY = center.y + (innerRadius + rayLength) *
sin(angleRad).toFloat()
                    drawLine(
                        color = Color(0xFFF59E0B),
                        start = Offset(startX, startY),
                        end = Offset(endX, endY),
                        strokeWidth = 3.dp.toPx(),
                        cap = androidx.compose.ui.graphics.StrokeCap.Round
                    )
                }
            }

            // 1, 2, 3: Partly Cloudy (Sun peeking behind cloud)
            1, 2, 3 -> {
                // Sun in top-right background
                val sunCenter = Offset(w * 0.65f, h * 0.35f)
                drawCircle(
                    brush = Brush.radialGradient(
                        colors = listOf(Color(0xFFFDE047),
Color(0xFFF59E0B)),
                        center = sunCenter,
                        radius = w * 0.18f
                    ),
                    center = sunCenter,
                    radius = w * 0.18f
                )

                // Overlapping Cloud
                val cloudPath = Path().apply {
                    val baseLine = h * 0.72f + cloudOffset
                    moveTo(w * 0.2f, baseLine)
                    // Bottom curve
                    lineTo(w * 0.8f, baseLine)
                    // Right curve
```

```
                    cubicTo(w * 0.95f, baseLine, w * 0.95f, baseLine - h *
0.25f, w * 0.78f, baseLine - h * 0.22f)
                    // Top-right curve
                    cubicTo(w * 0.7f, baseLine - h * 0.48f, w * 0.45f,
baseLine - h * 0.48f, w * 0.4f, baseLine - h * 0.28f)
                    // Left curve
                    cubicTo(w * 0.28f, baseLine - h * 0.32f, w * 0.1f,
baseLine - h * 0.22f, w * 0.2f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color.White, Color(0xFFF1F5F9))
                    )
                )
            }

            // 45, 48: Fog / Rime Fog (Soft overlapping slate clouds with
horizontal mist lines)
            45, 48 -> {
                // Cloud outline
                val cloudPath = Path().apply {
                    val baseLine = h * 0.62f + cloudOffset
                    moveTo(w * 0.25f, baseLine)
                    lineTo(w * 0.75f, baseLine)
                    cubicTo(w * 0.9f, baseLine, w * 0.9f, baseLine - h *
0.22f, w * 0.75f, baseLine - h * 0.2f)
                    cubicTo(w * 0.68f, baseLine - h * 0.42f, w * 0.45f,
baseLine - h * 0.42f, w * 0.42f, baseLine - h * 0.25f)
                    cubicTo(w * 0.3f, baseLine - h * 0.28f, w * 0.15f,
baseLine - h * 0.2f, w * 0.25f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color(0xFFE2E8F0),
Color(0xFF94A3B8))
                    )
                )

                // Fog lines at the bottom
                val fogLines = listOf(
                    Pair(w * 0.2f, w * 0.8f) to h * 0.76f,
                    Pair(w * 0.3f, w * 0.7f) to h * 0.84f,
                    Pair(w * 0.4f, w * 0.6f) to h * 0.92f
                )
                fogLines.forEach { (range, yVal) ->
                    val (startX, endX) = range
                    drawLine(
                        color = Color.White.copy(alpha = 0.8f),
                        start = Offset(startX, yVal),
                        end = Offset(endX, yVal),
                        strokeWidth = 3.dp.toPx(),
                        cap = androidx.compose.ui.graphics.StrokeCap.Round
                    )
                }
            }

            // 51 to 65, 80 to 82: Rainy / Drizzle / Rain Showers (Soft blue
cloud with falling animated raindrops)
            51, 53, 55, 61, 63, 65, 80, 81, 82 -> {
                // Cloud
                val cloudPath = Path().apply {
                    val baseLine = h * 0.58f + cloudOffset
                    moveTo(w * 0.25f, baseLine)
                    lineTo(w * 0.75f, baseLine)
```

```
                    cubicTo(w * 0.9f, baseLine, w * 0.9f, baseLine - h *
0.22f, w * 0.75f, baseLine - h * 0.2f)
                    cubicTo(w * 0.68f, baseLine - h * 0.42f, w * 0.45f,
baseLine - h * 0.42f, w * 0.42f, baseLine - h * 0.25f)
                    cubicTo(w * 0.3f, baseLine - h * 0.28f, w * 0.15f,
baseLine - h * 0.2f, w * 0.25f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color(0xFFE2E8F0),
Color(0xFF475569))
                    )
                )

                // 3 Falling Rain Drops
                val dropOffsetPx = rainDropY.dp.toPx()
                val dropWidth = 2.dp.toPx()
                val dropLength = 6.dp.toPx()
                val dropCols = listOf(w * 0.35f, w * 0.5f, w * 0.65f)
                dropCols.forEachIndexed { index, x ->
                    val individualY = h * 0.68f + ((dropOffsetPx + (index *
(h * 0.12f))) % (h * 0.28f))
                    if (individualY < h * 0.95f) {
                        drawLine(
                            color = Color(0xFF60A5FA),
                            start = Offset(x, individualY),
                            end = Offset(x - 2.dp.toPx(), individualY +
dropLength),
                            strokeWidth = dropWidth,
                            cap =
androidx.compose.ui.graphics.StrokeCap.Round
                        )
                    }
                }
            }

            // 71, 73, 75: Snowy (Light frosty blue cloud with falling
snowflakes)
            71, 73, 75 -> {
                // Cloud
                val cloudPath = Path().apply {
                    val baseLine = h * 0.58f + cloudOffset
                    moveTo(w * 0.25f, baseLine)
                    lineTo(w * 0.75f, baseLine)
                    cubicTo(w * 0.9f, baseLine, w * 0.9f, baseLine - h *
0.22f, w * 0.75f, baseLine - h * 0.2f)
                    cubicTo(w * 0.68f, baseLine - h * 0.42f, w * 0.45f,
baseLine - h * 0.42f, w * 0.42f, baseLine - h * 0.25f)
                    cubicTo(w * 0.3f, baseLine - h * 0.28f, w * 0.15f,
baseLine - h * 0.2f, w * 0.25f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color(0xFFF1F5F9),
Color(0xFF94A3B8))
                    )
                )

                // Snow flakes
                val snowOffset = rainDropY.dp.toPx() * 0.7f
                val snowCols = listOf(w * 0.35f, w * 0.5f, w * 0.65f)
                snowCols.forEachIndexed { index, x ->
                    val y = h * 0.68f + ((snowOffset + (index * (h *
0.12f))) % (h * 0.28f))
```

```
                    if (y < h * 0.95f) {
                        drawCircle(
                            color = Color(0xFFE0F2FE),
                            radius = 2.5.dp.toPx(),
                            center = Offset(x, y)
                        )
                    }
                }
            }

            // 95, 96, 99: Thunderstorm (Dark stormy cloud with flashing
bright yellow lightning)
            95, 96, 99 -> {
                // Cloud
                val cloudPath = Path().apply {
                    val baseLine = h * 0.56f + cloudOffset
                    moveTo(w * 0.25f, baseLine)
                    lineTo(w * 0.75f, baseLine)
                    cubicTo(w * 0.9f, baseLine, w * 0.9f, baseLine - h *
0.22f, w * 0.75f, baseLine - h * 0.2f)
                    cubicTo(w * 0.68f, baseLine - h * 0.42f, w * 0.45f,
baseLine - h * 0.42f, w * 0.42f, baseLine - h * 0.25f)
                    cubicTo(w * 0.3f, baseLine - h * 0.28f, w * 0.15f,
baseLine - h * 0.2f, w * 0.25f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color(0xFF334155),
Color(0xFF0F172A))
                    )
                )

                // Lightning bolt
                val boltPath = Path().apply {
                    moveTo(w * 0.52f, h * 0.58f)
                    lineTo(w * 0.43f, h * 0.76f)
                    lineTo(w * 0.53f, h * 0.76f)
                    lineTo(w * 0.45f, h * 0.96f)
                    lineTo(w * 0.59f, h * 0.72f)
                    lineTo(w * 0.49f, h * 0.72f)
                    close()
                }
                drawPath(
                    path = boltPath,
                    color = Color(0xFFFBBF24).copy(alpha = lightningAlpha)
                )
            }

            // Fallback (Partly cloudy layout)
            else -> {
                val cloudPath = Path().apply {
                    val baseLine = h * 0.65f + cloudOffset
                    moveTo(w * 0.25f, baseLine)
                    lineTo(w * 0.75f, baseLine)
                    cubicTo(w * 0.9f, baseLine, w * 0.9f, baseLine - h *
0.22f, w * 0.75f, baseLine - h * 0.2f)
                    cubicTo(w * 0.68f, baseLine - h * 0.42f, w * 0.45f,
baseLine - h * 0.42f, w * 0.42f, baseLine - h * 0.25f)
                    cubicTo(w * 0.3f, baseLine - h * 0.28f, w * 0.15f,
baseLine - h * 0.2f, w * 0.25f, baseLine)
                    close()
                }
                drawPath(
                    path = cloudPath,
                    brush = Brush.verticalGradient(
                        colors = listOf(Color.White, Color(0xFFE2E8F0))
```

```
                    )
                )
            }
        }
    }
}
```

**File: ui/main/MainScreen.kt**

```
package com.example.aiweathermonitor.ui.main

import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.Send
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.compose.ui.platform.LocalContext
import androidx.core.content.ContextCompat
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.navigation3.runtime.NavKey
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleEventObserver
import androidx.lifecycle.compose.LocalLifecycleOwner
import com.example.aiweathermonitor.audio.WeatherSoundPlayer
import com.example.aiweathermonitor.ui.DashboardScreen
import com.example.aiweathermonitor.ui.DashboardActions
import com.example.aiweathermonitor.ui.AppSettings
import org.koin.androidx.compose.koinViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MainScreen(
    modifier: Modifier = Modifier,
    settings: AppSettings = AppSettings(),
    onNavigateToSettings: () -> Unit = {},
    viewModel: MainScreenViewModel = koinViewModel()
) {
    val state by viewModel.weatherState.collectAsStateWithLifecycle()

    val context = LocalContext.current

    // ── Weather ambience: layered background sound matching the weather
code ──
    val soundPlayer = remember
{ WeatherSoundPlayer(context.applicationContext) }
    DisposableEffect(Unit) {
        onDispose { soundPlayer.release() }
    }
    // Start/stop & switch soundscape when the toggle or weather condition
changes
    LaunchedEffect(settings.soundEnabled, state.weatherCode,
state.isLoading) {
        if (settings.soundEnabled && !state.isLoading) {
            soundPlayer.play(state.weatherCode)
        } else if (!settings.soundEnabled) {
            soundPlayer.stop()
        }
    }
    // Pause when the app is backgrounded, resume when it returns
    val lifecycleOwner = LocalLifecycleOwner.current
    DisposableEffect(lifecycleOwner, settings.soundEnabled) {
        val observer = LifecycleEventObserver { _, event ->
            when (event) {
                Lifecycle.Event.ON_STOP -> soundPlayer.pause()
                Lifecycle.Event.ON_START -> if (settings.soundEnabled)
soundPlayer.resume()
```

```
                else -> {}
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
    }

    // Launcher for POST_NOTIFICATIONS permission (Android 13+)
    val notificationPermissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { _ ->
        // Handled silently
    }

    LaunchedEffect(Unit) {
        viewModel.initializeCache(context)
        if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.TIRAMISU) {
            val hasPermission = ContextCompat.checkSelfPermission(
                context, android.Manifest.permission.POST_NOTIFICATIONS
            ) == PackageManager.PERMISSION_GRANTED
            if (!hasPermission) {

notificationPermissionLauncher.launch(android.Manifest.permission.POST_NOTIF
ICATIONS)
            }
        }
    }

    val locationPermissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestMultiplePermissions()
    ) { permissions ->
        val fineGranted =
permissions[android.Manifest.permission.ACCESS_FINE_LOCATION] ?: false
        val coarseGranted =
permissions[android.Manifest.permission.ACCESS_COARSE_LOCATION] ?: false
        if (fineGranted || coarseGranted) {
            viewModel.fetchWeatherForCurrentLocation(context)
        } else {
            viewModel.updateWeatherState(
                state.copy(errorMessage = "Location permission denied.
Cannot fetch current location weather.")
            )
        }
    }

    Surface(
        modifier = modifier,
        color = MaterialTheme.colorScheme.background
    ) {
        val actions = remember(viewModel, context, state) {
            DashboardActions(
                onQueryChange = { viewModel.updateSearchQuery(it) },
                onSelectCity = { viewModel.selectCity(it, context) },
                onRefresh = { viewModel.refreshWeather(context) },
                onLocationClick = {
                    val hasFine = ContextCompat.checkSelfPermission(
                        context,
android.Manifest.permission.ACCESS_FINE_LOCATION
                    ) == PackageManager.PERMISSION_GRANTED
                    val hasCoarse = ContextCompat.checkSelfPermission(
                        context,
android.Manifest.permission.ACCESS_COARSE_LOCATION
                    ) == PackageManager.PERMISSION_GRANTED
                    if (hasFine || hasCoarse) {
                        viewModel.fetchWeatherForCurrentLocation(context)
                    } else {
                        locationPermissionLauncher.launch(
```

```
                            arrayOf(
android.Manifest.permission.ACCESS_FINE_LOCATION,
android.Manifest.permission.ACCESS_COARSE_LOCATION
                            )
                        )
                    }
                },
                onSettingsClick =
{ viewModel.updateWeatherState(state.copy(showKeyDialog = true)) },
                onNavigateToSettings = onNavigateToSettings,
                onSaveWeatherApiKey = { key ->
viewModel.updateWeatherState(state.copy(weatherApiKey = key, showKeyDialog =
false)) },
                onDismissSettings =
{ viewModel.updateWeatherState(state.copy(showKeyDialog = false)) },
                onSaveCity = { viewModel.saveCurrentCity() },
                onRemoveCity = { viewModel.removeCityFromFavorites(it) },
                onToggleCelsius = { viewModel.toggleCelsius() },
                onToggleSavedCitiesScreen =
{ viewModel.updateWeatherState(state.copy(showSavedCitiesScreen = it)) },
                onShareWeather =
{ com.example.aiweathermonitor.util.ShareWeatherCard.share(context,
state) },
                onOpenArticle = { url ->
                    if (url.isNotBlank()) {
                        runCatching {
                            val intent = android.content.Intent(
                                android.content.Intent.ACTION_VIEW,
                                android.net.Uri.parse(url)
                            ).addFlags(android.content.Intent.FLAG_ACTIVITY_
NEW_TASK)
                            context.startActivity(intent)
                        }
                    }
                },
                onRefreshNews = { viewModel.fetchNews() }
            )
        }

        DashboardScreen(
            state = state,
            settings = settings,
            actions = actions,
            modifier = Modifier.fillMaxSize()
        )
    }
}
```

**File: ui/main/MainScreenViewModel.kt**

```
package com.example.aiweathermonitor.ui.main

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.aiweathermonitor.*
import com.example.aiweathermonitor.data.models.PollenData
import com.example.aiweathermonitor.data.models.PollenResponse
import com.example.aiweathermonitor.BuildConfig
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.async
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.debounce
import kotlinx.coroutines.flow.distinctUntilChanged
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import kotlinx.serialization.json.Json
import okhttp3.OkHttpClient
import okhttp3.Request
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.concurrent.TimeUnit
import java.util.concurrent.atomic.AtomicBoolean
import androidx.core.content.edit
import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.data.WeatherRepository
import com.example.aiweathermonitor.data.DefaultWeatherRepository
import com.example.aiweathermonitor.util.HttpClientFactory
import com.example.aiweathermonitor.util.AppLogger
import com.example.aiweathermonitor.util.ErrorHandler
import com.example.aiweathermonitor.util.UrlBuilder
import com.example.aiweathermonitor.data.models.*
import com.example.aiweathermonitor.ui.AppSettings
import com.example.aiweathermonitor.ui.WindUnit
import com.example.aiweathermonitor.ui.PressureUnit
import com.example.aiweathermonitor.ui.VisibilityUnit
import com.example.aiweathermonitor.ui.AppTheme
import com.example.aiweathermonitor.data.NewsRepository
import com.example.aiweathermonitor.data.DefaultNewsRepository

private const val TAG = "MainScreenViewModel"
private const val TIME_DEFAULT_SUNRISE = "6:00 AM"
private const val TIME_DEFAULT_SUNSET = "6:00 PM"
private const val WEATHER_ALERT_NOTIFICATION_ID = 1001

class MainScreenViewModel(
    private val repository: WeatherRepository = DefaultWeatherRepository(
        HttpClientFactory.create(),
        Json { ignoreUnknownKeys = true }
    ),
    private val newsRepository: NewsRepository = DefaultNewsRepository(
        HttpClientFactory.create()
    ),
    private val jsonParser: Json = Json { ignoreUnknownKeys = true },
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO
) : ViewModel() {

    private val _weatherState = MutableStateFlow(WeatherState())
    val weatherState: StateFlow<WeatherState> = _weatherState.asStateFlow()

    private val _appSettings = MutableStateFlow(AppSettings())
```

```
    val appSettings: StateFlow<AppSettings> = _appSettings.asStateFlow()

    private var searchJob: Job? = null
    private val isCacheInitialized = AtomicBoolean(false)

    private val effectiveWeatherApiKey: String
        get() = _weatherState.value.weatherApiKey.ifBlank
{ BuildConfig.WEATHER_API_KEY }

    // ── Initialization
──────────────────────────────────────────────────


    fun initializeCache(context: android.content.Context) {
        if (!isCacheInitialized.compareAndSet(false, true)) return

        val connectivityObserver =
com.example.aiweathermonitor.NetworkConnectivityObserver(context)
        viewModelScope.launch {
            connectivityObserver.isConnected.collect { online ->
                _weatherState.value = _weatherState.value.copy(isOnline =
online)
            }
        }

        _appSettings.value = loadSettingsFromPrefs(context)

        // Kick off a news fetch alongside weather on launch.
        fetchNews()

        val cached = loadStateFromPrefs(context)
        if (cached != null) {
            _weatherState.value = cached
            refreshWeather(context)
        } else {
            fetchWeatherForCoordinates(
                cityName = WeatherApiConfig.Defaults.DEFAULT_CITY,
                latitude =
WeatherApiConfig.Defaults.DEFAULT_LATITUDE.toFloat(),
                longitude =
WeatherApiConfig.Defaults.DEFAULT_LONGITUDE.toFloat(),
                context = context
            )
        }

        viewModelScope.launch {
            // Debounce + filter: only persist when real weather data
changes.
            // Skips transient UI state (search query, loading flag, etc.)
            // so we don't hit SharedPreferences on every keystroke.
            _weatherState
                .debounce(500L)
                .distinctUntilChanged { a, b ->
                    a.temperature    == b.temperature    &&
                    a.weatherCode    == b.weatherCode    &&
                    a.selectedCity   == b.selectedCity   &&
                    a.hourlyForecast == b.hourlyForecast &&
                    a.dailyForecast  == b.dailyForecast  &&
                    a.uvIndex        == b.uvIndex        &&
                    a.aqi            == b.aqi            &&
                    a.weatherApiKey  == b.weatherApiKey  &&
                    a.isCelsius      == b.isCelsius      &&
                    a.savedCities    == b.savedCities
                }
                .collect { state ->
                    withContext(ioDispatcher) { saveStateToPrefs(context,
state) }
                }
        }
```

```
        viewModelScope.launch {
            _appSettings.collect { settings ->
                withContext(ioDispatcher) { saveSettingsToPrefs(context,
settings) }
                if (settings.backgroundRefresh) {
                    WeatherRefreshWorker.schedule(context)
                } else {
                    WeatherRefreshWorker.cancel(context)
                }
            }
        }
    }

    private fun saveSettingsToPrefs(context: android.content.Context,
settings: AppSettings) {
        try {
            val prefs = context.getSharedPreferences(
                WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
                android.content.Context.MODE_PRIVATE
            )
            prefs.edit(commit = false) {
                putString("wind_unit", settings.windUnit.name)
                putString("pressure_unit", settings.pressureUnit.name)
                putString("visibility_unit", settings.visibilityUnit.name)
                putString("app_theme", settings.theme.name)
                putBoolean("notif_severe", settings.notifySevere)
                putBoolean("notif_rain", settings.notifyRain)
                putBoolean("notif_briefing", settings.notifyDailyBriefing)
                putBoolean("notif_uv", settings.notifyHighUv)
                putBoolean("bg_refresh", settings.backgroundRefresh)
                putBoolean("sound_enabled", settings.soundEnabled)
            }
        } catch (e: Exception) {
            AppLogger.error("Failed to save settings: ${e.message}", TAG, e)
        }
    }

    private fun loadSettingsFromPrefs(context: android.content.Context):
AppSettings {
        return try {
            val prefs = context.getSharedPreferences(
                WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
                android.content.Context.MODE_PRIVATE
            )
            AppSettings(
                windUnit = WindUnit.valueOf(prefs.getString("wind_unit",
WindUnit.KMH.name) ?: WindUnit.KMH.name),
                pressureUnit =
PressureUnit.valueOf(prefs.getString("pressure_unit", PressureUnit.HPA.name)
?: PressureUnit.HPA.name),
                visibilityUnit =
VisibilityUnit.valueOf(prefs.getString("visibility_unit",
VisibilityUnit.KM.name) ?: VisibilityUnit.KM.name),
                theme = AppTheme.valueOf(prefs.getString("app_theme",
AppTheme.SYSTEM.name) ?: AppTheme.SYSTEM.name),
                notifySevere = prefs.getBoolean("notif_severe", true),
                notifyRain = prefs.getBoolean("notif_rain", true),
                notifyDailyBriefing = prefs.getBoolean("notif_briefing",
false),
                notifyHighUv = prefs.getBoolean("notif_uv", false),
                backgroundRefresh = prefs.getBoolean("bg_refresh", true),
                soundEnabled = prefs.getBoolean("sound_enabled", false)
            )
        } catch (e: Exception) {
            AppSettings()
        }
    }
```

```
    private fun saveStateToPrefs(context: android.content.Context, state:
WeatherState) {
        try {
            val prefs = context.getSharedPreferences(
                WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
                android.content.Context.MODE_PRIVATE
            )
            val json = jsonParser.encodeToString(WeatherState.serializer(),
state)
            prefs.edit(commit = false) {

putString(WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY, json)
            }
            // Only update widgets when they are actually present — skips
IPC overhead otherwise
            val widgetIds =
android.appwidget.AppWidgetManager.getInstance(context).getAppWidgetIds(
                android.content.ComponentName(context,
WeatherWidgetProvider::class.java)
            )
            if (widgetIds.isNotEmpty()) {
                val intent = android.content.Intent(context,
WeatherWidgetProvider::class.java).apply {
                    action =
android.appwidget.AppWidgetManager.ACTION_APPWIDGET_UPDATE

putExtra(android.appwidget.AppWidgetManager.EXTRA_APPWIDGET_IDS, widgetIds)
                    setPackage(context.packageName)
                }
                context.sendBroadcast(intent) // NOSONAR
            }
        } catch (e: java.io.IOException) {
            AppLogger.error("IO error saving state: ${e.message}", TAG, e)
        } catch (e: kotlinx.serialization.SerializationException) {
            AppLogger.error("Serialization error saving state: $
{e.message}", TAG, e)
        } catch (e: Exception) {
            AppLogger.error("Failed to save state: ${e.message}", TAG, e)
        }
    }

    private fun loadStateFromPrefs(context: android.content.Context):
WeatherState? {
        return try {
            val prefs = context.getSharedPreferences(
                WeatherApiConfig.SharedPrefsKeys.PREFS_NAME,
                android.content.Context.MODE_PRIVATE
            )
            val jsonStr =
prefs.getString(WeatherApiConfig.SharedPrefsKeys.WEATHER_STATE_KEY, null)
            if (!jsonStr.isNullOrBlank()) {
                jsonParser.decodeFromString(WeatherState.serializer(),
jsonStr).copy(
                    isLoading = false,
                    isSearching = false,
                    searchQuery = "",
                    searchResults = emptyList(),
                    showKeyDialog = false,
                    showSavedCitiesScreen = false,
                    errorMessage = null
                )
            } else null
        } catch (e: kotlinx.serialization.SerializationException) {
            AppLogger.debug("Cached state corrupted, clearing", TAG)
            null
        } catch (e: Exception) {
            AppLogger.warning("Failed to load cached state", TAG, e)
            null
```

```
        }
    }

    // ── Public state updaters ──────────────────────────────────

    fun updateWeatherState(newState: WeatherState) { _weatherState.value =
newState }

    fun updateSettings(settings: AppSettings) { _appSettings.value =
settings }

    fun updateSearchQuery(query: String) {
        _weatherState.value = _weatherState.value.copy(searchQuery = query)
        searchJob?.cancel()
        if (query.trim().length >= WeatherApiConfig.SEARCH_MIN_CHARS) {
            searchJob = viewModelScope.launch {
                delay(WeatherApiConfig.SEARCH_DEBOUNCE_MS.toLong())
                searchCities(query.trim())
            }
        } else {
            _weatherState.value = _weatherState.value.copy(searchResults =
emptyList())
        }
    }

    fun selectCity(result: GeocodingResult, context:
android.content.Context? = null) {
        _weatherState.value = _weatherState.value.copy(
            searchQuery = "",
            searchResults = emptyList(),
            selectedCity = result.name,
            latitude = result.latitude,
            longitude = result.longitude
        )
        fetchWeatherForCoordinates(result.name, result.latitude.toFloat(),
result.longitude.toFloat(), context)
    }

    fun refreshWeather(context: android.content.Context? = null) {
        val s = _weatherState.value
        fetchWeatherForCoordinates(s.selectedCity, s.latitude.toFloat(),
s.longitude.toFloat(), context)
    }

    // ── News (delegated to NewsRepository) ─────────────────────

    /**
     * Fetches two location-aware feeds for the current city — weather news
and
     * general/local news — in parallel, into [WeatherState.newsArticles]
and
     * [WeatherState.localNewsArticles].
     */
    fun fetchNews() {
        val city = _weatherState.value.selectedCity.trim()
        val weatherQuery = if (city.isNotBlank()) "$city weather" else
"weather"
        val localQuery = if (city.isNotBlank()) city else "top stories"

        viewModelScope.launch {
            _weatherState.value = _weatherState.value.copy(isNewsLoading =
true, newsError = null)

            val weatherDeferred = async
{ newsRepository.fetchNews(UrlBuilder.googleNewsSearch(weatherQuery)) }
```

```
            val localDeferred = async
{ newsRepository.fetchNews(UrlBuilder.googleNewsSearch(localQuery)) }
            val weatherRes = weatherDeferred.await()
            val localRes = localDeferred.await()

            val bothFailed = weatherRes.isFailure && localRes.isFailure
            if (bothFailed) {
                val e = weatherRes.exceptionOrNull()
                AppLogger.error("News fetch failed", TAG, e)
            }
            _weatherState.value = _weatherState.value.copy(
                newsArticles = weatherRes.getOrNull() ?: emptyList(),
                localNewsArticles = localRes.getOrNull() ?: emptyList(),
                isNewsLoading = false,
                newsError = if (bothFailed) {
                    val e = weatherRes.exceptionOrNull()
                    if (e != null && ErrorHandler.isNetworkError(e))
                        WeatherApiConfig.ErrorMessages.ERROR_NO_INTERNET
                    else "Couldn't load news right now."
                } else null
            )
        }
    }

    fun saveCurrentCity() {
        val s = _weatherState.value
        if (s.savedCities.none { it.name.equals(s.selectedCity, ignoreCase =
true) }) {
            _weatherState.value = s.copy(
                savedCities = s.savedCities + GeocodingResult(
                    name = s.selectedCity,
                    latitude = s.latitude,
                    longitude = s.longitude
                )
            )
        }
    }

    fun removeCityFromFavorites(city: GeocodingResult) {
        val s = _weatherState.value
        _weatherState.value = s.copy(
            savedCities = s.savedCities.filterNot
{ it.name.equals(city.name, ignoreCase = true) }
        )
    }

    fun toggleCelsius() {
        _weatherState.value = _weatherState.value.copy(isCelsius = !
_weatherState.value.isCelsius)
    }

    // ── City search (delegated to WeatherRepository)
──────────────────────

    private suspend fun searchCities(query: String) {
        _weatherState.value = _weatherState.value.copy(isSearching = true)
        repository.searchCities(query, effectiveWeatherApiKey)
            .onSuccess { results ->
                _weatherState.value = _weatherState.value.copy(
                    searchResults = results,
                    isSearching = false,
                    errorMessage = null
                )
            }
            .onFailure { e ->
                val msg = if (ErrorHandler.isNetworkError(e))
                    WeatherApiConfig.ErrorMessages.ERROR_NO_INTERNET
                else
```

```
                    ErrorHandler.getFriendlyErrorMessage(e)
                AppLogger.error("City search failed", TAG, e)
                _weatherState.value = _weatherState.value.copy(isSearching =
false, errorMessage = msg)
            }
    }

    // ── Weather fetch (delegated to WeatherRepository)
────────────────────────

    fun fetchWeatherForCoordinates(
        cityName: String,
        latitude: Float,
        longitude: Float,
        context: android.content.Context? = null
    ) {
        viewModelScope.launch {
            _weatherState.value = _weatherState.value.copy(isLoading = true)
            repository.getWeather(
                base = _weatherState.value,
                cityName = cityName,
                latitude = latitude,
                longitude = longitude,
                apiKey = effectiveWeatherApiKey
            ).onSuccess { newState ->
                val cityChanged = newState.selectedCity !=
_weatherState.value.selectedCity
                _weatherState.value = newState
                if (context != null) maybeNotify(context, newState)
                // Refresh location-aware news whenever the city changes.
                if (cityChanged) fetchNews()
            }.onFailure { e ->
                AppLogger.error("Failed to fetch weather", TAG, e)
                _weatherState.value = _weatherState.value.copy(
                    isLoading = false,
                    errorMessage = ErrorHandler.getFriendlyErrorMessage(e)
                )
            }
        }
    }

    /** Fires a local alert notification for extreme UV or thunderstorm
conditions. */
    private fun maybeNotify(context: android.content.Context, s:
WeatherState) {
        val alertMsg = when {
            s.uvIndex >= 8 -> "▫ Extreme UV (${s.uvIndex}) in $
{s.selectedCity} — protect yourself!"
            s.weatherCode in listOf(95, 96, 99) -> " Thunderstorm warning
active in ${s.selectedCity}!"
            else -> null
        }
        if (alertMsg != null) triggerLocalNotification(context, "Sunnyside
Alert: ${s.selectedCity}", alertMsg)
    }

    // ── Location
──────────────────────────────────────────────────────────

    fun fetchWeatherForCurrentLocation(context: android.content.Context) {
        viewModelScope.launch {
            _weatherState.value = _weatherState.value.copy(isLoading = true,
errorMessage = null)
            try {
                // Permission check
                val hasFine   =
androidx.core.content.ContextCompat.checkSelfPermission(
```

```
                    context,
android.Manifest.permission.ACCESS_FINE_LOCATION
                ) == android.content.pm.PackageManager.PERMISSION_GRANTED
                val hasCoarse =
androidx.core.content.ContextCompat.checkSelfPermission(
                    context,
android.Manifest.permission.ACCESS_COARSE_LOCATION
                ) == android.content.pm.PackageManager.PERMISSION_GRANTED

                if (!hasFine && !hasCoarse) {
                    _weatherState.value = _weatherState.value.copy(
                        isLoading = false, errorMessage = "Location
permission not granted."
                    )
                    return@launch
                }

                // Provider check
                val lm =
context.getSystemService(android.content.Context.LOCATION_SERVICE)
                        as android.location.LocationManager
                if (!
lm.isProviderEnabled(android.location.LocationManager.GPS_PROVIDER) &&
                    !
lm.isProviderEnabled(android.location.LocationManager.NETWORK_PROVIDER)) {
                    _weatherState.value = _weatherState.value.copy(
                        isLoading = false,
                        errorMessage = "Location services disabled. Enable
GPS and try again."
                    )
                    return@launch
                }

                // Suspend coroutine wrapper — avoids nested launch-in-
callback anti-pattern.
                // Strategy: try lastLocation (instant) → balanced accuracy
(fast, cell/WiFi) →
                //           high accuracy (GPS, slower) — each with an 8-
second timeout.
                val loc: android.location.Location? =
getLocationSuspending(context)

                if (loc == null) {
                    _weatherState.value = _weatherState.value.copy(
                        isLoading = false,
                        errorMessage = "Could not get location. Open GPS,
wait a moment outdoors, then tap the location button again."
                    )
                    return@launch
                }

                val city = geocodeCoordinates(loc.latitude.toFloat(),
loc.longitude.toFloat(), context)
                    ?: "Current Location"
                _weatherState.value = _weatherState.value.copy(
                    selectedCity = city, latitude = loc.latitude, longitude
= loc.longitude
                )
                fetchWeatherForCoordinates(city, loc.latitude.toFloat(),
loc.longitude.toFloat(), context)

            } catch (e: SecurityException) {
                _weatherState.value = _weatherState.value.copy(
                    isLoading = false, errorMessage = "Permission error: $
{e.message}"
                )
            } catch (e: Exception) {
                _weatherState.value = _weatherState.value.copy(
```

```
                    isLoading = false, errorMessage = "Location failed: $
{e.message}"
                )
            }
        }
    }

    /**
     * Suspending location fetch — 4-tier strategy.
     * 1. FusedLocationProvider lastLocation   — cached, instant
     * 2. FusedLocationProvider getCurrentLocation(HIGH_ACCURACY) — 10s
     * 3. FusedLocationProvider requestLocationUpdates — 12s (powers GPS
radio)
     * 4. Raw LocationManager fallback — works even with limited Play
Services
     */
    @Suppress("MissingPermission")
    private suspend fun getLocationSuspending(
        context: android.content.Context
    ): android.location.Location? = withContext(ioDispatcher) {

        // ── Tier 1-3: FusedLocationProvider
────────────────────────────────
        val fusedResult = runCatching {
            val fusedClient =
com.google.android.gms.location.LocationServices
                .getFusedLocationProviderClient(context)

            // Tier 1: cached last location (< 2 min old)
            val lastKnown =
kotlinx.coroutines.suspendCancellableCoroutine<android.location.Location?> {
cont ->
                fusedClient.lastLocation
                    .addOnSuccessListener { loc -> cont.resume(loc, null) }
                    .addOnFailureListener { cont.resume(null, null) }
            }
            if (lastKnown != null && System.currentTimeMillis() -
lastKnown.time < 2 * 60 * 1000L) {
                return@runCatching lastKnown
            }

            // Tier 2: getCurrentLocation HIGH_ACCURACY — 10s
            val freshLoc = kotlinx.coroutines.withTimeoutOrNull(10_000L) {

kotlinx.coroutines.suspendCancellableCoroutine<android.location.Location?> {
cont ->
                    val cts =
com.google.android.gms.tasks.CancellationTokenSource()
                    fusedClient.getCurrentLocation(

com.google.android.gms.location.Priority.PRIORITY_HIGH_ACCURACY,
                        cts.token
                    )
                        .addOnSuccessListener { loc -> cont.resume(loc,
null) }
                        .addOnFailureListener { cont.resume(null, null) }
                    cont.invokeOnCancellation { cts.cancel() }
                }
            }
            if (freshLoc != null) return@runCatching freshLoc

            // Tier 3: requestLocationUpdates — 12s (actually powers GPS
radio)
            kotlinx.coroutines.withTimeoutOrNull(12_000L) {

kotlinx.coroutines.suspendCancellableCoroutine<android.location.Location?> {
cont ->
```

```
                    val req =
com.google.android.gms.location.LocationRequest.Builder(

com.google.android.gms.location.Priority.PRIORITY_HIGH_ACCURACY, 1000L
                    ).setMaxUpdates(1).setMinUpdateIntervalMillis(500L).buil
d()

                    val cb = object :
com.google.android.gms.location.LocationCallback() {
                        override fun onLocationResult(r:
com.google.android.gms.location.LocationResult) {
                            fusedClient.removeLocationUpdates(this)
                            cont.resume(r.lastLocation, null)
                        }
                    }
                    fusedClient.requestLocationUpdates(req, cb,
android.os.Looper.getMainLooper())
                    cont.invokeOnCancellation
{ fusedClient.removeLocationUpdates(cb) }
                }
            }
        }.getOrNull()

        if (fusedResult != null) return@withContext fusedResult

        // ── Tier 4: Raw LocationManager (works without full Play Services)
──
        AppLogger.warning("FusedLocationProvider failed, falling back to
LocationManager", TAG)
        rawLocationManagerFix(context)
    }

    @Suppress("MissingPermission")
    private suspend fun rawLocationManagerFix(
        context: android.content.Context
    ): android.location.Location? = withContext(ioDispatcher) {
        val lm =
context.getSystemService(android.content.Context.LOCATION_SERVICE)
                as android.location.LocationManager

        // Quick: try last known from both providers
        val lastGps =
lm.getLastKnownLocation(android.location.LocationManager.GPS_PROVIDER)
        val lastNet =
lm.getLastKnownLocation(android.location.LocationManager.NETWORK_PROVIDER)
        val freshThreshold = System.currentTimeMillis() - 5 * 60 * 1000L
        val lastKnown = listOfNotNull(lastGps, lastNet)
            .filter { it.time > freshThreshold }
            .maxByOrNull { it.time }
        if (lastKnown != null) return@withContext lastKnown

        // Active fix: try GPS first (15s), then network (8s)
        val gpsFix = if
(lm.isProviderEnabled(android.location.LocationManager.GPS_PROVIDER)) {
            kotlinx.coroutines.withTimeoutOrNull(15_000L) {

kotlinx.coroutines.suspendCancellableCoroutine<android.location.Location?> {
cont ->
                    val listener = object :
android.location.LocationListener {
                        override fun onLocationChanged(loc:
android.location.Location) {
                            lm.removeUpdates(this)
                            cont.resume(loc, null)
                        }
                        @Deprecated("") override fun onStatusChanged(p:
String?, s: Int, e: android.os.Bundle?) {}
                        override fun onProviderEnabled(p: String) {}
```

```
                        override fun onProviderDisabled(p: String)
{ cont.resume(null, null) }
                    }
                    lm.requestLocationUpdates(
                        android.location.LocationManager.GPS_PROVIDER, 0L,
0f, listener,
                        android.os.Looper.getMainLooper()
                    )
                    cont.invokeOnCancellation { lm.removeUpdates(listener) }
                }
            }
        } else null
        if (gpsFix != null) return@withContext gpsFix

        // Network fix (cell/WiFi triangulation) — faster but less accurate
        return@withContext if
(lm.isProviderEnabled(android.location.LocationManager.NETWORK_PROVIDER)) {
            kotlinx.coroutines.withTimeoutOrNull(8_000L) {

kotlinx.coroutines.suspendCancellableCoroutine<android.location.Location?> {
cont ->
                    val listener = object :
android.location.LocationListener {
                        override fun onLocationChanged(loc:
android.location.Location) {
                            lm.removeUpdates(this)
                            cont.resume(loc, null)
                        }
                        @Deprecated("") override fun onStatusChanged(p:
String?, s: Int, e: android.os.Bundle?) {}
                        override fun onProviderEnabled(p: String) {}
                        override fun onProviderDisabled(p: String)
{ cont.resume(null, null) }
                    }
                    lm.requestLocationUpdates(
                        android.location.LocationManager.NETWORK_PROVIDER,
0L, 0f, listener,
                        android.os.Looper.getMainLooper()
                    )
                    cont.invokeOnCancellation { lm.removeUpdates(listener) }
                }
            }
        } else null
    }

    /** Reverse-geocode coordinates to a human-readable city/area name. */
    private suspend fun geocodeCoordinates(
        latitude: Float,
        longitude: Float,
        context: android.content.Context
    ): String? = withContext(ioDispatcher) {
        runCatching {
            val geocoder = android.location.Geocoder(context,
Locale.getDefault())
            @Suppress("DEPRECATION")
            val addresses = geocoder.getFromLocation(latitude.toDouble(),
longitude.toDouble(), 1)
            addresses?.firstOrNull()?.let { addr ->
                addr.locality ?: addr.subAdminArea ?: addr.adminArea ?:
addr.countryName
            }
        }.getOrNull()
    }

    /** Posts a high-priority local notification on the severe-weather
channel. */
    private fun triggerLocalNotification(
        context: android.content.Context,
```

```
        title: String,
        message: String
    ) {
        if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.TIRAMISU &&
            androidx.core.content.ContextCompat.checkSelfPermission(
                context, android.Manifest.permission.POST_NOTIFICATIONS
            ) != android.content.pm.PackageManager.PERMISSION_GRANTED
        ) return
        val notification = androidx.core.app.NotificationCompat.Builder(
            context, NotificationHelper.CHANNEL_SEVERE
        )
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle(title)
            .setContentText(message)
            .setStyle(androidx.core.app.NotificationCompat.BigTextStyle().bi
gText(message))
            .setPriority(androidx.core.app.NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true)
            .build()
        androidx.core.app.NotificationManagerCompat.from(context)
            .notify(WEATHER_ALERT_NOTIFICATION_ID, notification)
    }
}
```

**File: util/AppLogger.kt**

```
package com.example.aiweathermonitor.util

import android.util.Log
import com.example.aiweathermonitor.exception.WeatherException

/**
 * Centralized logging utility for the application.
 * Provides structured logging with proper tags and error tracking.
 */
object AppLogger {
    private const val TAG = "WeatherApp"

    /**
     * Log debug messages (only in debug builds)
     */
    fun debug(message: String, tag: String = TAG) {
        if (Log.isLoggable(tag, Log.DEBUG)) {
            Log.d(tag, message)
        }
    }

    /**
     * Log info messages
     */
    fun info(message: String, tag: String = TAG) {
        Log.i(tag, message)
    }

    /**
     * Log warning messages
     */
    fun warning(message: String, tag: String = TAG, throwable: Throwable? =
null) {
        if (throwable != null) {
            Log.w(tag, message, throwable)
        } else {
            Log.w(tag, message)
        }
    }

    /**
     * Log error messages with exception
     */
    fun error(message: String, tag: String = TAG, throwable: Throwable? =
null) {
        if (throwable != null) {
            Log.e(tag, message, throwable)
        } else {
            Log.e(tag, message)
        }
    }

    /**
     * Log exception with stack trace
     */
    fun exception(tag: String = TAG, throwable: Throwable) {
        Log.e(tag, "Exception occurred: ${throwable.message}", throwable)
    }

    /**
     * Logs a weather-related exception with structured information.
     */
    fun logWeatherException(tag: String, message: String, exception:
WeatherException) {
        val errorDetails = """
            |Error: $message
```

```
            |Type: ${exception.javaClass.simpleName}
            |Cause: ${exception.cause?.javaClass?.simpleName ?: "None"}
            |Message: ${exception.message}
        """.trimMargin()
        Log.e(tag, errorDetails, exception)
    }

    /**
     * Logs network request details for debugging.
     */
    fun logNetworkRequest(tag: String, url: String, method: String = "GET")
{
        Log.d(tag, "→ $method $url")
    }

    /**
     * Logs network response details for debugging.
     */
    fun logNetworkResponse(tag: String, statusCode: Int, responseTime: Long)
{
        Log.d(tag, "← HTTP $statusCode (${responseTime}ms)")
    }

    /**
     * Logs data parsing operations.
     */
    fun logDataParsing(tag: String, dataType: String, successful: Boolean) {
        val status = if (successful) "✓" else "✗"
        Log.d(tag, "$status Parsing $dataType")
    }
}
```

**File: util/ErrorHandler.kt**

```
package com.example.aiweathermonitor.util

import com.example.aiweathermonitor.config.WeatherApiConfig
import com.example.aiweathermonitor.exception.WeatherException

/**
 * Utility for handling and translating exceptions into user-friendly
messages.
 * Consolidates error detection logic to avoid duplication.
 */
object ErrorHandler {

    /**
     * Determines if an exception is network-related.
     */
    fun isNetworkError(exception: Throwable): Boolean {
        return exception is java.net.UnknownHostException ||
                exception.cause is java.net.UnknownHostException ||
                exception is java.net.SocketTimeoutException ||
                exception is java.io.IOException ||
                exception.message?.contains("Unable to resolve host",
ignoreCase = true) == true ||
                exception.message?.contains("Connection refused", ignoreCase
= true) == true ||
                exception.message?.contains("Network unreachable",
ignoreCase = true) == true
    }

    /**
     * Determines if an exception is a data parsing/format error.
     */
    fun isDataError(exception: Throwable): Boolean {
        return exception.javaClass.simpleName.contains("JsonSyntax",
ignoreCase = true) ||
               exception.javaClass.simpleName.contains("JsonEOF", ignoreCase
= true) ||
               (exception is java.io.IOException &&
                exception.message?.contains("JSON", ignoreCase = true) ==
true)
    }

    /**
     * Determines if an exception is an API configuration error.
     */
    fun isConfigurationError(exception: Throwable): Boolean {
        return exception.message?.contains("Invalid API", ignoreCase = true)
== true ||
               exception.message?.contains("Unauthorized", ignoreCase =
true) == true ||
               exception.message?.contains("API key", ignoreCase = true) ==
true
    }

    /**
     * Converts an exception to a user-friendly error message.
     * Returns a message suitable for display in the UI.
     */
    fun getFriendlyErrorMessage(exception: Throwable): String {
        return when {
            isNetworkError(exception) ->
WeatherApiConfig.ErrorMessages.ERROR_NO_INTERNET
            isConfigurationError(exception) ->
WeatherApiConfig.ErrorMessages.ERROR_INVALID_API_KEY
            exception.message?.contains("timeout", ignoreCase = true) ?:
false ->
                WeatherApiConfig.ErrorMessages.ERROR_TIMEOUT
```

```
            (exception.message?.contains("500") ?: false) ||
            (exception.message?.contains("503") ?: false) ->
                WeatherApiConfig.ErrorMessages.ERROR_SERVER
            isDataError(exception) ->
WeatherApiConfig.ErrorMessages.ERROR_INVALID_DATA
            else -> exception.message ?: "An unexpected error occurred"
        }
    }

    /**
     * Converts an exception to a weather exception for consistent handling.
     */
    fun toWeatherException(exception: Throwable): WeatherException {
        return when {
            exception is WeatherException -> exception
            isNetworkError(exception) ->
com.example.aiweathermonitor.exception.NetworkException(
                WeatherApiConfig.ErrorMessages.ERROR_NO_INTERNET,
                exception
            )
            isConfigurationError(exception) ->
com.example.aiweathermonitor.exception.ConfigurationException(
                WeatherApiConfig.ErrorMessages.ERROR_INVALID_API_KEY,
                exception
            )
            isDataError(exception) ->
com.example.aiweathermonitor.exception.DataParsingException(
                WeatherApiConfig.ErrorMessages.ERROR_INVALID_DATA,
                exception
            )
            exception is java.net.SocketTimeoutException ->
                com.example.aiweathermonitor.exception.NetworkException(
                    WeatherApiConfig.ErrorMessages.ERROR_TIMEOUT,
                    exception
                )
            else -> com.example.aiweathermonitor.exception.NetworkException(
                getFriendlyErrorMessage(exception),
                exception
            )
        }
    }

    /**
     * Safely validates coordinate values.
     * Returns true if coordinates are valid, false otherwise.
     */
    fun isValidCoordinate(latitude: Float, longitude: Float): Boolean {
        return latitude in -90f..90f && longitude in -180f..180f
    }

    /**
     * Safely validates temperature value.
     * Valid range: -50°C to 70°C (covers extreme weather conditions)
     */
    fun isValidTemperature(temp: Float): Boolean {
        return temp in -50f..70f
    }

    /**
     * Safely validates percentage-based metrics (humidity, UV index
normalized).
     * Valid range: 0-100
     */
    fun isValidPercentage(value: Float): Boolean {
        return value in 0f..100f
    }

    /**
```

```
     * Safely validates pressure value.
     * Valid range: 870-1050 hPa (normal atmospheric variations)
     */
    fun isValidPressure(pressure: Float): Boolean {
        return pressure in 870f..1050f
    }

    /**
     * Safely validates wind speed.
     * Valid range: 0-150 km/h (covers even extreme hurricane conditions)
     */
    fun isValidWindSpeed(windSpeed: Float): Boolean {
        return windSpeed >= 0f && windSpeed <= 150f
    }
}
```

**File: util/Formatters.kt**

```
package com.example.aiweathermonitor.util

import com.example.aiweathermonitor.ui.PressureUnit
import com.example.aiweathermonitor.ui.VisibilityUnit
import com.example.aiweathermonitor.ui.WindUnit
import kotlin.math.roundToInt

/**
 * Shared unit-formatting helpers used by both the main dashboard UI and the
 * home-screen widget. Centralising these keeps conversion/rounding rules in
one
 * place so the app and the widget can never drift apart.
 */

/** Full temperature with unit suffix, e.g. "21°C" / "70°F". */
fun formatTemp(tempC: Float, isCelsius: Boolean): String =
    if (isCelsius) "${tempC.roundToInt()}°C"
    else "${(tempC * 9f / 5f + 32f).roundToInt()}°F"

/** Compact temperature without unit suffix, e.g. "21°" — for space-
constrained surfaces (widget). */
fun formatTempShort(tempC: Float, isCelsius: Boolean): String =
    if (isCelsius) "${tempC.roundToInt()}°"
    else "${(tempC * 9f / 5f + 32f).roundToInt()}°"

fun formatWind(speedKmh: Float, unit: WindUnit): String = when (unit) {
    WindUnit.KMH -> "${speedKmh.roundToInt()} km/h"
    WindUnit.MPH -> "${(speedKmh / 1.60934f).roundToInt()} mph"
    WindUnit.MS  -> "${(speedKmh / 3.6f).roundToInt()} m/s"
}

fun formatPressure(hpa: Float, unit: PressureUnit): String = when (unit) {
    PressureUnit.HPA  -> "${hpa.roundToInt()} hPa"
    PressureUnit.INHG -> "${"%.2f".format(hpa * 0.02953f)} inHg"
}

fun formatVisibility(km: Float, unit: VisibilityUnit): String = when (unit)
{
    VisibilityUnit.KM -> "${"%.1f".format(km)} km"
    VisibilityUnit.MI -> "${"%.1f".format(km * 0.621371f)} mi"
}
```

**File: util/HttpClientFactory.kt**

```
package com.example.aiweathermonitor.util

import com.example.aiweathermonitor.config.WeatherApiConfig
import okhttp3.OkHttpClient
import java.util.concurrent.TimeUnit

/**
 * Single source of truth for the app's [OkHttpClient] configuration. Both
the
 * Koin graph and the
[com.example.aiweathermonitor.ui.main.MainScreenViewModel]
 * default construct their client here so timeout/retry settings can never
drift.
 */
object HttpClientFactory {
    fun create(): OkHttpClient = OkHttpClient.Builder()
        .connectTimeout(WeatherApiConfig.API_TIMEOUT_SECONDS,
TimeUnit.SECONDS)
        .readTimeout(WeatherApiConfig.API_TIMEOUT_SECONDS, TimeUnit.SECONDS)
        .writeTimeout(WeatherApiConfig.API_TIMEOUT_SECONDS,
TimeUnit.SECONDS)
        .retryOnConnectionFailure(true)
        .build()
}
```

**File: util/OnboardingPrefs.kt**

```
package com.example.aiweathermonitor.util

import android.content.Context
import com.example.aiweathermonitor.config.WeatherApiConfig

/**
 * Small wrapper around the onboarding-completed flag in SharedPreferences
so the
 * key and prefs name live in exactly one place instead of being re-typed at
every
 * navigation call site.
 */
object OnboardingPrefs {
    private const val KEY_HAS_SEEN_ONBOARDING = "has_seen_onboarding"

    private fun prefs(context: Context) = context.getSharedPreferences(
        WeatherApiConfig.SharedPrefsKeys.PREFS_NAME, Context.MODE_PRIVATE
    )

    fun hasSeenOnboarding(context: Context): Boolean =
        prefs(context).getBoolean(KEY_HAS_SEEN_ONBOARDING, false)

    fun markOnboardingSeen(context: Context) {
        prefs(context).edit().putBoolean(KEY_HAS_SEEN_ONBOARDING,
true).apply()
    }
}
```

**File: util/ShareWeatherCard.kt**

```
package com.example.aiweathermonitor.util

import android.content.Context
import android.content.Intent
import android.graphics.Bitmap
import android.graphics.Canvas
import android.graphics.Paint
import android.graphics.RectF
import android.graphics.Typeface
import androidx.core.content.FileProvider
import com.example.aiweathermonitor.WeatherState
import com.example.aiweathermonitor.getAqiDescription
import com.example.aiweathermonitor.getUvDescription
import com.example.aiweathermonitor.getWeatherCodeDescription
import java.io.File

object ShareWeatherCard {

    /**
     * Draws a 1080×600 weather card bitmap and shares it via the system
share sheet.
     */
    fun share(context: Context, state: WeatherState) {
        val bmp = drawCard(state)
        val file = saveBitmap(context, bmp)
        val uri = FileProvider.getUriForFile(context, "$
{context.packageName}.provider", file)

        val intent = Intent(Intent.ACTION_SEND).apply {
            type = "image/png"
            putExtra(Intent.EXTRA_STREAM, uri)
            putExtra(
                Intent.EXTRA_TEXT,
                "Weather in ${state.selectedCity}: $
{state.temperature.toInt()}° · $
{state.weatherCode.getWeatherCodeDescription()}"
            )
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }
        context.startActivity(Intent.createChooser(intent, "Share weather"))
    }

    private fun drawCard(state: WeatherState): Bitmap {
        val w = 1080; val h = 600
        val bmp = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888)
        val canvas = Canvas(bmp)

        // Background
        val bgPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0xFF1A2744.toInt() }
        canvas.drawRoundRect(RectF(0f, 0f, w.toFloat(), h.toFloat()), 48f,
48f, bgPaint)

        // Accent circle (decorative)
        val accentPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0x220EA5E9.toInt() }
        canvas.drawCircle(w * 0.78f, h * 0.25f, 260f, accentPaint)

        val white = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0xFFFFFFFF.toInt() }
        val muted = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0xFF90B0C8.toInt() }
        val accent = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0xFF60A5FA.toInt() }
        val uvColor = Paint(Paint.ANTI_ALIAS_FLAG).apply { color =
0xFFFCD34D.toInt() }
```

```
        // City
        white.textSize = 52f; white.typeface = Typeface.DEFAULT_BOLD
        canvas.drawText(state.selectedCity, 60f, 100f, white)

        // Condition
        muted.textSize = 34f; muted.typeface = Typeface.DEFAULT
        canvas.drawText(state.weatherCode.getWeatherCodeDescription(), 60f,
150f, muted)

        // Big temperature
        white.textSize = 200f; white.typeface = Typeface.DEFAULT_BOLD
        val tempStr = if (state.isCelsius) "${state.temperature.toInt()}°C"
else "${((state.temperature * 9 / 5) + 32).toInt()}°F"
        canvas.drawText(tempStr, 60f, 380f, white)

        // H/L line
        val today = state.dailyForecast.firstOrNull()
        if (today != null) {
            val hi = if (state.isCelsius) "${today.tempMax.toInt()}°" else
"${((today.tempMax * 9 / 5) + 32).toInt()}°"
            val lo = if (state.isCelsius) "${today.tempMin.toInt()}°" else
"${((today.tempMin * 9 / 5) + 32).toInt()}°"
            muted.textSize = 36f
            canvas.drawText("H: $hi  ·  L: $lo", 60f, 440f, muted)
        }

        // Divider
        val divPaint = Paint().apply { color = 0x40FFFFFF.toInt();
strokeWidth = 1.5f }
        canvas.drawLine(60f, 480f, w - 60f, 480f, divPaint)

        // Bottom row: feels like, UV, AQI, Wind
        val metrics = buildList {
            add("Feels like" to "${state.feelsLike.toInt()}°")
            if (state.uvIndex > 0f) add("UV" to "${state.uvIndex.toInt()} ·
${state.uvIndex.getUvDescription()}")
            if (state.aqi > 0) add("AQI" to "${state.aqi} · $
{state.aqi.getAqiDescription()}")
            add("Wind" to "${state.windSpeed.toInt()} km/h $
{state.windDir}".trim())
        }.take(4)

        val colW = (w - 120f) / metrics.size
        metrics.forEachIndexed { i, (label, value) ->
            val x = 60f + i * colW
            muted.textSize = 24f
            canvas.drawText(label, x, 530f, muted)
            white.textSize = 30f; white.typeface = Typeface.DEFAULT_BOLD
            canvas.drawText(value, x, 568f, white)
        }

        // Source badge
        accent.textSize = 22f
        canvas.drawText("Open-Meteo · WeatherAPI.com", 60f, h - 20f, accent)

        return bmp
    }

    private fun saveBitmap(context: Context, bmp: Bitmap): File {
        val dir = File(context.cacheDir, "weather_shares").apply
{ mkdirs() }
        val file = File(dir, "weather_card.png")
        file.outputStream().use { bmp.compress(Bitmap.CompressFormat.PNG,
95, it) }
        return file
    }
}
```

**File: util/UrlBuilder.kt**

```
package com.example.aiweathermonitor.util

import java.net.URLEncoder
import com.example.aiweathermonitor.config.WeatherApiConfig

/**
 * Safe URL builder to prevent injection vulnerabilities and encoding
issues.
 * All parameters are properly URL-encoded.
 */
class SafeUrlBuilder(private val baseUrl: String) {
    private val queryParams = mutableMapOf<String, String>()

    /** Adds a query parameter with safe encoding. Skips blank values. */
    fun addParam(key: String, value: String): SafeUrlBuilder {
        if (value.isNotBlank()) {
            queryParams[key] = URLEncoder.encode(value, "UTF-8")
        }
        return this
    }

    fun addParam(key: String, value: Number): SafeUrlBuilder {
        queryParams[key] = value.toString()
        return this
    }

    fun addParam(key: String, value: Boolean): SafeUrlBuilder {
        queryParams[key] = value.toString()
        return this
    }

    fun build(): String {
        if (queryParams.isEmpty()) return baseUrl
        return baseUrl + "?" + queryParams.entries.joinToString("&") { (k,
v) -> "$k=$v" }
    }
}

// ── Type conversion helpers
─────────────────────────────────────────────

fun Double?.toFloatSafe(): Float = this?.toFloat() ?: 0f
fun Float?.toDoubleSafe(): Double = this?.toDouble() ?: 0.0
fun String?.orEmpty(): String = this ?: ""
fun <T> List<T>?.getOrNull(index: Int): T? = this?.getOrNull(index)
fun <T> List<T>?.getOrDefault(index: Int, default: T): T =
this?.getOrNull(index) ?: default

// ── URL factory
──────────────────────────────────────────────────────────

object UrlBuilder {

    /** Google News RSS search feed for an arbitrary query (HTTPS, US
English). */
    fun googleNewsSearch(query: String): String =
        "https://news.google.com/rss/search?q=" +
            URLEncoder.encode(query, "UTF-8") +
            "&hl=en-US&gl=US&ceid=US:en"

    /** Open-Meteo geocoding search. */
    fun openMeteoSearch(query: String, limit: Int = 10): String =
        SafeUrlBuilder(WeatherApiConfig.Endpoints.OPEN_METEO_GEOCODING)
            .addParam("name", query)
            .addParam("count", limit)
            .addParam("language", "en")
```

```
            .build()

    /**
     * Open-Meteo weather forecast.
     * Includes current, hourly, and daily blocks with UV index and
humidity.
     */
    fun openMeteoForecast(
        latitude: Float,
        longitude: Float,
        hourly: Boolean = true,
        daily: Boolean = true
    ): String =
SafeUrlBuilder(WeatherApiConfig.Endpoints.OPEN_METEO_FORECAST)
        .addParam("latitude", latitude)
        .addParam("longitude", longitude)
        .addParam("current",
"temperature_2m,relative_humidity_2m,apparent_temperature,surface_pressure,w
ind_speed_10m,wind_direction_10m,weather_code,uv_index")
        .apply {
            if (hourly) addParam("hourly",
"temperature_2m,weather_code,precipitation_probability,visibility")
            if (daily) addParam(
                "daily",

"weather_code,temperature_2m_max,temperature_2m_min,sunrise,sunset,uv_index_
max,precipitation_probability_max"
            )
        }
        .addParam("forecast_days", WeatherApiConfig.FORECAST_DAYS_LIMIT)
        .addParam("timezone", "auto")
        .build()

    /**
     * WeatherAPI.com forecast endpoint.
     * Returns current conditions + hourly/daily forecast + alerts in one
call.
     * Requires a free API key from https://www.weatherapi.com/
     */
    /** WeatherAPI.com city search — returns up to [limit] matches. */
    fun weatherApiSearch(query: String, apiKey: String, limit: Int = 10):
String =
        SafeUrlBuilder(WeatherApiConfig.Endpoints.WEATHER_API_SEARCH)
            .addParam("key", apiKey)
            .addParam("q", query)
            .build()

    fun weatherApiForecast(
        latitude: Float,
        longitude: Float,
        apiKey: String,
        days: Int = WeatherApiConfig.WEATHER_API_FORECAST_DAYS,
        aqi: Boolean = true,
        alerts: Boolean = true
    ): String =
SafeUrlBuilder(WeatherApiConfig.Endpoints.WEATHER_API_FORECAST)
        .addParam("key", apiKey)
        .addParam("q", "$latitude,$longitude")
        .addParam("days", days)
        .addParam("aqi", if (aqi) "yes" else "no")
        .addParam("alerts", if (alerts) "yes" else "no")
        .build()

    /** Open-Meteo air quality — hourly pollen concentrations (grains/m³).
*/
    fun openMeteoPollen(latitude: Float, longitude: Float): String =
        SafeUrlBuilder(WeatherApiConfig.Endpoints.OPEN_METEO_AIR_QUALITY)
            .addParam("latitude", latitude)
```

```
            .addParam("longitude", longitude)
            .addParam("hourly",
"alder_pollen,birch_pollen,grass_pollen,mugwort_pollen,olive_pollen,ragweed_
pollen")
            .addParam("forecast_days", 1)
            .addParam("timezone", "auto")
            .build()
}
```

**File: util/WeatherLogger.kt**

```
package com.example.aiweathermonitor.util

/**
 * DEPRECATED — use [AppLogger] instead.
 *
 * This file is a duplicate of AppLogger that was left over from a refactor.
 * It is intentionally empty. Delete this file once your VCS workflow allows
it.
 */
@Deprecated(
    message = "Use AppLogger instead. WeatherLogger is a dead duplicate.",
    replaceWith = ReplaceWith("AppLogger",
"com.example.aiweathermonitor.util.AppLogger"),
    level = DeprecationLevel.ERROR
)
object WeatherLogger
```